

Le Deep Learning et le hasard – Prédiction du Loto

L'étude mathématique de la loterie est aussi ancienne que les mathématiques. On peut citer les travaux de Leonard Euler qui ont conduit à la théorie de la probabilité de Kolmogorov [Kolmogorov, 1933]. À ce jour peu d'ouvrages existent sur la résolution du problème très difficile voire impossible de prédire des tirages de numéros de loterie à cause de l'indépendance des tirages (aléatoires) et donc de l'inexistence de patterns à apprendre pour les modèles.

Une grande question me vient alors en tête : et si malgré leur caractère aléatoire, les boules lors des tirages se retrouvent dans les mêmes positions que certains tirages passés, et sont mélangés à la même vitesse, ces informations ne peuvent-elles pas permettre à des modèles d'apprendre ces patterns et nous conduire à prédire les prochains numéros tirés ?

L'apprentissage profond a prouvé ses capacités dans la résolution de nombreux problèmes différents, qu'il s'agisse de la reconnaissance vocale, la reconnaissance d'images ou encore la reconnaissance d'écriture manuscrite. Ses algorithmes dont la structure est inspirée du fonctionnement du cerveau humain sont capables de capturer la structure sous-jacente de n'importe quel jeu de données avec des résultats très concluants.

Ainsi l'objectif de cet article est de montrer les résultats de l'apprentissage profond sur un cas concret de hasard.

Pour ce faire je présenterai dans cet article un modèle de prédiction des tirages du loto français en utilisant les tirages du passé comme données d'apprentissage.

Généralités et présentation du loto français

Le Loto est un jeu célèbre qui fait appel au hasard. Celui-ci est tellement répandu dans le monde entier que même les petits états comme le Vatican dont la papauté était initialement contre les jeux d'argent et de hasard, dispose aujourd'hui de leur propre loto permettant de lever des fonds pour des actions caritatives. La dynastie Han a même utilisé les bénéfices du loto pour financer la grande muraille de Chine.

Le principe de la loterie est très simple : les gens achètent un ticket qui correspond à un pari combiné sur un ensemble général de chiffres. Un tirage au sort sans remise est finalement

effectué à une date et une heure fixe. Les gains sont liés à la façon dont la combinaison correspond au tirage au sort. Le jackpot est gagné si la combinaison est correcte.

Pour jouer, il suffit de cocher 6 numéros :

- 5 numéros sur une grille de 49 numéros
- 1 numéro de chance sur une grille de 10 numéros

Le jackpot est remporté si on a parié les 5 bons numéros plus le numéro de chance.

Le prix d'une grille est de 2,20 € et le jackpot minimum est de 2 millions d'euros.

Collecte des données et calcul de features

Les données ont été *scrapées* sur le site de la française des jeux (fdj) qui recense tous les tirages du loto depuis 2008 à ce jour. Le dernier tirage à la date de rédaction de cet article datait du 21 octobre 2020 avec un total de 1921 tirages entre le 6 octobre 2008 et cette date.

```
In [4]: from scrap_archive import scrap_loto
df_tirage=scrap_loto()
df_tirage[['day','month_year','num0','num1','num2','num3','num4','chance']].head()
```

Out[4]:

	day	month_year	num0	num1	num2	num3	num4	chance
0	Mercredi	21 octobre 2020	15	21	33	35	48	1
1	Lundi	19 octobre 2020	7	25	28	35	48	1
2	Samedi	17 octobre 2020	9	20	23	24	28	4
3	Mercredi	14 octobre 2020	19	26	32	39	42	6
4	Lundi	12 octobre 2020	7	11	18	21	42	3

La prédiction des numéros de loto est une tâche supervisée : les tirages du passé sont utilisés comme données d'entrées en supposant les tirages dépendants même si ce n'est pas le cas dans la réalité. Ainsi en apprenant ces numéros du passé, on suppose que ce sont des numéros obtenus à partir d'une certaine position initiale des boules dans l'urne et un mélange à une certaine vitesse.

Les données que j'ai retenues après calcul pour chaque tirage sont :

- Le nombre de fois que chaque numéro est apparu dans les tirages précédents
- La présence de nombre pair ou impair
- La somme de la différence au carré entre chaque couple de numéros successifs dans le tirage
- Le nombre de numéros en dessous de 24
- Le nombre de numéros en dessous de 40

```
In [12]: df = df_tirage.iloc[::-1]#inversion du dataframe
df = df_tirage[['num0', 'num1', 'num2', 'num3', 'num4', 'chance']]
df['freq_num0'] = freq_val(df, 'num0')
df['freq_num1'] = freq_val(df, 'num1')
df['freq_num2'] = freq_val(df, 'num2')
df['freq_num3'] = freq_val(df, 'num3')
df['freq_num4'] = freq_val(df, 'num4')
df['freq_chance'] = freq_val(df, 'chance')#calcul des frequences
df['sum_diff'] = sum_diff(df)#somme de la différence au carré entre chaque couple de numéros successifs dans le tirage
df['pair_chance'] = is_pair_etoile(df)
df['impair_chance'] = is_impair_etoile(df)
df['pair'] = is_pair(df)
df['impair'] = is_impair(df)#verification de nombre pair et impair
df['is_under_24'] = is_under(df, 24) # Les numeros en dessous de 24
df['is_under_40'] = is_under(df, 40)# Les numeros en dessous de 40
df.head(6)
```

/anaconda3/lib/python3.6/site-packages/ipykernel_launcher.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

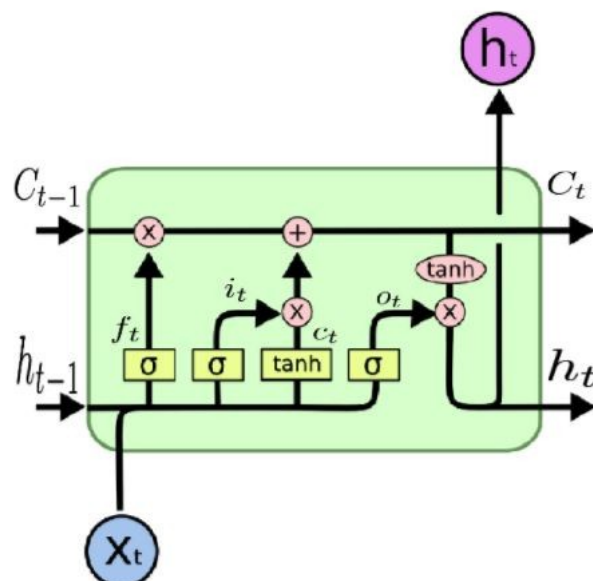
See the caveats in the documentation: <http://pandas.pydata.org/pandas-docs/stable/indexing.html#indexing-view-versus-copy>
This is separate from the ipykernel package so we can avoid doing imports until

```
Out[12]:
```

	num0	num1	num2	num3	num4	chance	freq_num0	freq_num1	freq_num2	freq_num3	freq_num4	freq_chance	sum_diff	pair_chance	impair_chance
0	15	21	33	35	48	1	1	1	1	1	1	1	353	0	1
1	7	25	28	35	48	1	1	1	1	2	2	2	551	0	1
2	9	20	23	24	28	4	1	1	1	1	1	1	147	1	0
3	19	26	32	39	42	6	1	1	1	1	1	1	143	1	0
4	7	11	18	21	42	3	2	1	1	1	2	1	515	0	1
5	26	31	42	47	48	8	1	1	1	1	3	1	172	1	0

Modèle

Les réseaux de neurones récurrents (RNN) se sont révélés comme étant l'un des modèles les plus puissants pour le traitement de données séquentielles. La mémoire à long court terme est l'une des architectures RNN la plus réussie. LSTM introduit la cellule mémoire, une unité de calcul qui remplace les neurones artificiels traditionnels dans la couche cachée du réseau. Avec ces cellules de mémoire, les réseaux sont capables d'associer efficacement des mémoires et d'entrer dans le temps, ce qui permet de saisir la structure des données de manière dynamique dans le temps avec une capacité de prédiction élevée.



Architecture d'un LSTM :

$$\begin{aligned} f_t &= \sigma(w_f[h_{t-1}, X_t] + b_f) & i_t &= \sigma(W_i[h_{t-1}, X_t] + b_i) & c_t &= \tan(W_c[h_{t-1}, X_t] \\ & + b_c) & O_t &= \sigma(W_o[h_{t-1}, X_t] + b_o) & C_t &= f_t * C_{t-1} + i_t * c_t & h_t &= O_t * \tanh(C_t) \end{aligned}$$

- f_t : Porte d'oubli, décide s'il faut remettre à 0 le contenu de la cellule.
- O_t : porte de sortie, décide si le contenu de la cellule doit influencer sur la sortie du neurone.
- i_t : Porte d'entrée, décide si l'entrée doit modifier le contenu de la cellule.
- C_t : Mise à jour de l'état de la cellule.
- h_t : Sortie de la cellule.

Formation du réseau

Notre modèle LSTM est composé d'une couche d'entrée séquentielle suivie d'une couche LSTM et d'une couche de sortie dense avec fonction d'activation linéaire.

```
01 import numpy as np
02 import pandas as pd
03 import matplotlib.pyplot as plt
04 import pickle
05 from sklearn.preprocessing import StandardScaler
06 from keras.models import Sequential
07 from keras.layers import LSTM, Dense, Bidirectional, TimeDistributed, RepeatVector, Flatten
08 from keras.callbacks import EarlyStopping
09
10 UNITS = 100
11 BATCHSIZE = 30
12 EPOCH = 1000
13 #ACTIVATION = "softmax"
14 OPTIMIZER = 'adam' # rmsprop, adam, SGD
15 LOSS = 'mae' # 'categorical_crossentropy' #mse
16 DROPOUT = 0.1
17 window_length = 12 #12
18 number_of_features = df.shape[1]
19
20 #Architecture du modèle
21 def define_model(number_of_features, nb_label_feature):
22     #initialisation du rnn
23     model = Sequential()
24     #ajout de la première couche lstm
25     model.add(LSTM(UNITS, input_shape=(window_length, number_of_features), return_sequences=True))
26     model.add(LSTM(UNITS, dropout=0.1, return_sequences=False))
27     #ajout de la couche de sortie
28     model.add(Dense(nb_label_feature))
29     model.compile(loss=LOSS, optimizer=OPTIMIZER, metrics=['acc'])
30     return model
```

Optimiseur

J'ai choisi d'utiliser l'optimiseur Adam. L'optimiseur Adam combine les avantages de deux autres optimiseurs : ADAGRAD et RMSprop. Cette méthode calcule les taux d'apprentissage adaptatif pour chaque paramètre en considérant la moyenne en décroissance exponentielle des

gradients carrés passés et la moyenne en décroissance exponentielle des gradients passés. Cela peut être représenté par :

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{avec} \quad v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{avec} \quad m_t = \beta m_{t-1} + (1 - \beta) g_t$$

Cela nous amène à la règle finale de mise à jour du gradient :

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \zeta} \cdot \hat{m}_t$$

Où v et m peuvent être considérés comme les estimations du premier et du deuxième moment des gradients respectivement, d'où le nom d'estimation de moment adaptatif.

Les avantages de cet optimiseur sont résumés comme suit :

- Le taux d'apprentissage est différent pour chaque paramètre et chaque itération
- L'apprentissage ne diminue pas comme avec l'ADAGRAD
- Une descente plus solide d'un point de vue statistique.

Entraînement :

Avant d'entraîner un modèle LSTM, il faut une préparation minimum de la donnée afin de s'accommoder à la dimension voulue par les RNNs en particulier le LSTM. En effet, la donnée en entrée doit avoir trois dimensions (*samples, timesteps, feature*).

- Samples désigne le nombre d'échantillons ou d'observations
- Timesteps est le pas de temps
- Feature est le nombre de caractéristiques ou de variables à étudier

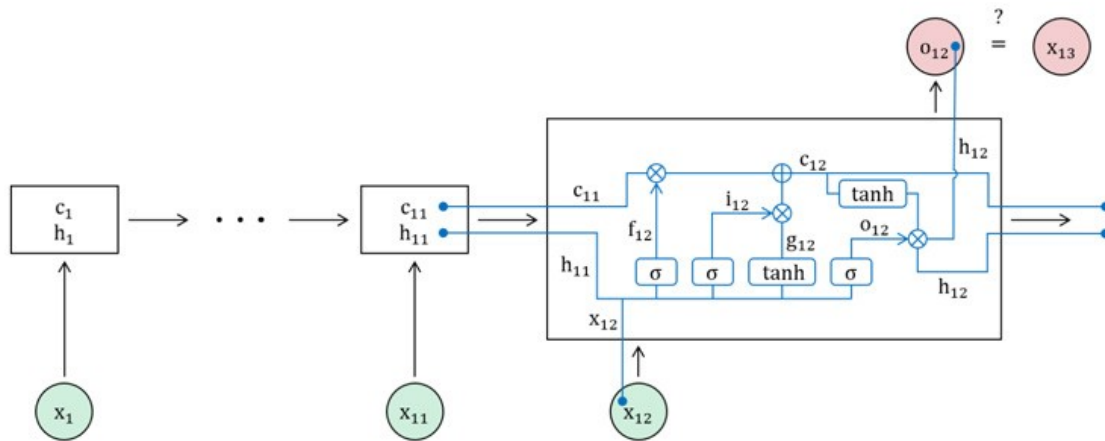
Ainsi pour notre suite de tirage, j'ai dû implémenter une fonction permettant de transformer la séquence initiale à la forme attendue en entrée du LSTM en choisissant un *timesteps* sur un mois soit 12 tirages car trois tirages sont réalisés par semaine pour le loto français.

Par exemple pour la première séquence de 13 tirages $S=[t_1, t_2, t_3, \dots, t_{13}]$

L'entrée $X=[[t_1], [t_2], [t_3], [t_4], \dots, [t_{12}]]$ avec X de la forme (1,12,19) car ayant 19 features calculés au total pour chaque tirage

La sortie $Y=[[t_{13}]]$ avec Y de la forme (1,1,6) car l'on désire ici prédire les bons numéros du tirage 13.

On peut donc concrètement représenter les entrées et sorties à partir des 13 premiers tirages avec des LSTM comme suit :



*Architecture des entrées et sorties LSTM des 13 premiers tirages
où les x_i représentent les différents tirages.*

Une fois l'architecture de mon modèle LSTM prête, il ne reste qu'à entraîner le modèle.

Pour ce faire, mon dataset d'entraînement est constitué de tous les tirages *scrapés* et ma donnée de test représente le prochain tirage.

C'est à dire que j'entraîne le modèle sur toutes les données disponibles, et j'effectue une prédiction sur le prochain tirage qui représente ma donnée de test. Je procède de la sorte afin de m'assurer qu'à chaque tirage, le modèle ait appris du tirage précédent. Ainsi à chaque nouvelle date de tirage, un nouveau modèle est entraîné. Cette méthode a fait ces preuves, dans la mesure où j'arrive très souvent à prédire 3 numéros parmi les 6 tout en étant proche des autres numéros non trouvés. Voyons un exemple concret.

Résultats d'apprentissage sur le tirage du 21 octobre 2020

Pour ce test j'ai commencé par supprimer du dataset d'entraînement le tirage du 21 octobre afin de l'utiliser comme donnée de test.

Le modèle converge après 830 *epochs* avec une précision de 92% sur les données de train.

```
tom = df.tail(window_length).iloc[:,0:6] # les n derniers tirages qui est la valeur du window_length
scaler = StandardScaler().fit(df.iloc[:,0:6])
scaled_to_predict = scaler.transform(tom)
print(scaler.inverse_transform(scaled_predicted_output_1).astype(int)[0])
```

[5 15 21 36 43 1]

```
#Training
train, label, model, scaler1 = create_lstm_dataset(df, window_length, nb_label_feature)
history = model.fit(train, label, batch_size=BATCHSIZE, epochs=EPOCH, verbose=2, callbacks=[es])
```

Epoch 822/1000

- 1s - loss: 0.0779 - acc: 0.9271

Epoch 823/1000

- 1s - loss: 0.0779 - acc: 0.9135

Epoch 824/1000

- 1s - loss: 0.0764 - acc: 0.9266

Epoch 825/1000

- 1s - loss: 0.0780 - acc: 0.9088

Epoch 826/1000

- 1s - loss: 0.0781 - acc: 0.9209

Epoch 827/1000

- 1s - loss: 0.0790 - acc: 0.9261

Epoch 828/1000

- 1s - loss: 0.0784 - acc: 0.9161

Epoch 829/1000

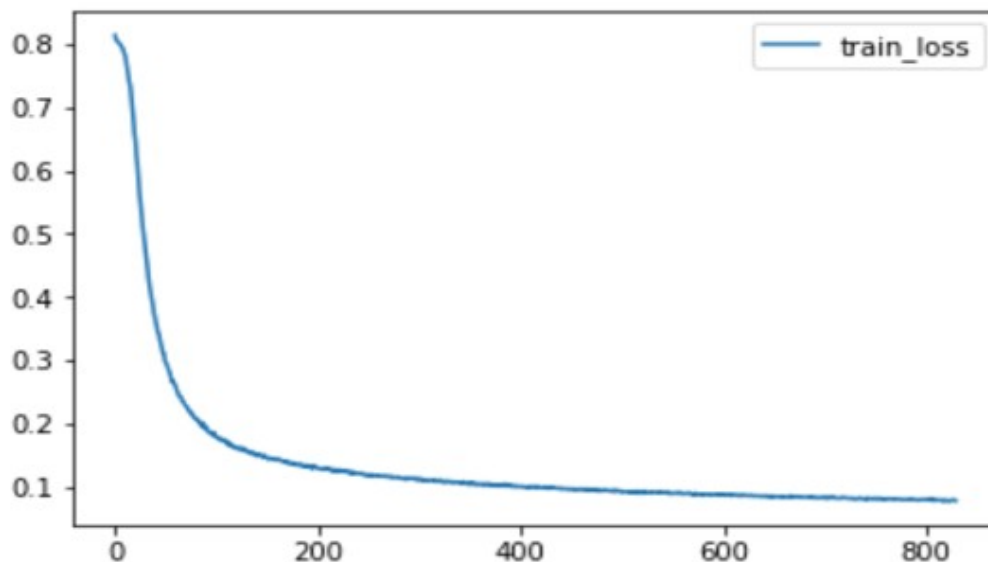
- 1s - loss: 0.0789 - acc: 0.9135

Epoch 830/1000

- 1s - loss: 0.0774 - acc: 0.9256

Epoch 00830: early stopping

```
from matplotlib import pyplot
pyplot.plot(history.history['loss'])
pyplot.legend(['train_loss'])
pyplot.show()
```



Résultats de prédiction pour le tirage du 21 octobre 2020

```
tom = df.tail(window_length).iloc[:,0:6] # les n derniers tirages qui est la valeur du window_length
scaler = StandardScaler().fit(df.iloc[:,0:6])
scaled_to_predict = scaler.transform(tom)
print(scaler.inverse_transform(scaled_predicted_output_1).astype(int)[0])
```

[5 15 21 36 43 1]

Le résultat officiel du tirage du 21 octobre 2020 sur le site de la FDJ est donné par les numéros suivants : [15 21 33 35 48 01].

Et comme vous pouvez le constater le modèle prédit bien 3 numéros à savoir le 15 le 21 et 1 pour le numéro de chance.

Résultats de prédiction pour le tirage du 24 octobre 2020

```
tom = df.tail(window_length).iloc[:,0:6] # les n derniers tirages qui est la valeur du window_length
scaler = StandardScaler().fit(df.iloc[:,0:6])
scaled_to_predict = scaler.transform(tom)
print(scaler.inverse_transform(scaled_predicted_output_1).astype(int)[0])

[ 3  8 18 36 46  6]
```

Le résultat officiel du tirage du 24 octobre 2020 sur le site de la FDJ est donné par les numéros suivants : [4 8 12 25 45 02].

Comme vous pouvez le constater pour ce tirage le numéro 8 a été bien prédit cependant deux autres numéros dont le 3 et le 46 sont très proches du vrai résultat respectivement à une somme et différence près de 1.

Jusque-là mon modèle n'arrive qu'à prédire au maximum pour certains tirages 3 bons numéros sur 6 ou s'en rapproche ce qui permet d'évaluer sa précision globale sur le loto français à ce jour à 50%.

Remarque

Le réapprentissage du modèle sur les mêmes données d'apprentissage risque de donner une prédiction différente s'approchant encore plus du vrai tirage ou même s'en éloignant. Quelqu'un me dira mais pourquoi ne pas effectuer une cross-validation sur le training set afin d'avoir un modèle robuste qui prédira toujours la même sortie même après plusieurs entraînements du modèle sur les mêmes données ?

Le principe même de la validation croisée qui consiste à diviser le dataset en plusieurs portions et d'utiliser de manière itérative chaque petite portion comme valeur de test et les grandes comme valeurs de train en supposant que les échantillons sont indépendants n'est pas adapté aux LSTM qui apprennent des valeurs historiques pour prédire l'avenir. Je laisse cette question ouverte à tout commentaire.

Conclusion

En dépit de l'indépendance des tirages, le modèle LSTM arrive très souvent à prédire certains bons numéros en supposant que ceux-ci sont dépendants. Ces résultats proviennent-ils du hasard ? non je ne pense pas. Plus de travail est nécessaire pour optimiser le modèle afin de pouvoir éventuellement remporter le jackpot un de ces jours. J'envisage par exemple de personnaliser le calcul de l'erreur en prenant en compte les erreurs de prédictions qui se traduisent par de petites différences de 1 ou 2 en plus ou en moins.

Chapitre 1

Dans ce chapitre, nous allons découvrir les cellules à mémoire interne : les LSTM et comprendre le mécanisme des portes de contrôle. Ensuite, nous allons assimiler l'utilisation des LSTM en couches et comprendre l'intérêt des LSTM bidirectionnels et multidimensionnels, et leur fonctionnement.

La cellule LSTM: Long Short Term Memory

Comme nous l'avons vu dans le chapitre précédent, afin de modéliser des dépendances à très long terme, il est nécessaire de donner aux réseaux de neurones récurrents la capacité de **maintenir** un état sur une longue période de temps.

C'est le but des cellules LSTM (Long Short Term Memory), qui **possèdent une mémoire interne appelée cellule** (ou *cell*). La cellule permet de **maintenir un état aussi longtemps que nécessaire**. Cette cellule consiste en une valeur numérique que le réseau peut piloter en fonction des situations.

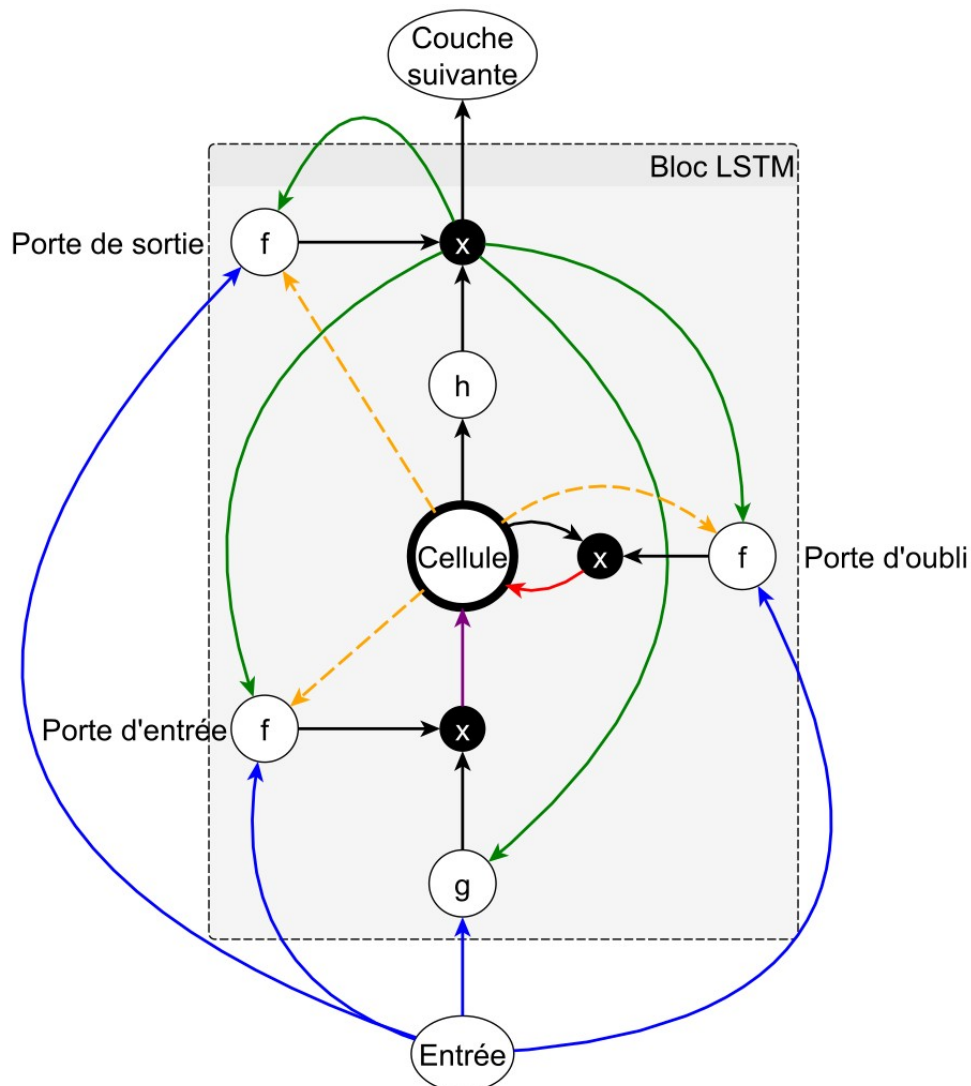


Schéma d'un neurone LSTM comprenant une mémoire interne (cell) contrôlée par les trois portes d'entrée (input), d'oubli (Forget) et de sortie (output). Schéma issu de la thèse de Bruno Stuner (2018).

Comme on peut le constater sur le schéma, la cellule mémoire peut être pilotée par **trois portes** de contrôle qu'on peut voir comme des vannes :

- **La porte d'entrée** décide si l'entrée doit **modifier le contenu** de la cellule ;
- **La porte d'oubli** décide s'il faut **remettre à 0** le contenu de la cellule ;
- **La porte de sortie** décide si le contenu de la cellule doit **influencer sur la sortie** du neurone.

Le mécanisme des trois portes est strictement similaire. L'ouverture/la fermeture de la vanne est modélisée par une fonction f qui est généralement une sigmoïde. Cette sigmoïde est appliquée à la somme pondérée des entrées (en bleu), des sorties (en vert) et de la cellule (en orange), par des poids W_{ab} spécifiques à chaque connexion entre les signaux a et b .

Soient X_i et Z_h les entrées et les sorties de la cellule, S_c la valeur de la cellule et i, ω, c les indices décrivant respectivement les signaux issus de la porte d'entrée, d'oubli et de la cellule. W_{ii} sont donc les poids reliant les entrées à la cellule d'entrée, $W_{h\phi}$ les poids reliant les sorties à la porte d'oubli, etc.

Les équations régissant les trois portes de contrôle sont donc les suivantes ; elles sont l'application de la somme pondérée suivie de l'application d'une activation f , typiquement la sigmoïde :

Porte d'entrée :

$$a_{ti} = \sum_{i=1}^I W_{ii} X_{ti} + \sum_{h=1}^H W_{hi} Z_{t-1h} + \sum_{c=1}^C W_{ci} S_{t-1c} b_{ti} = f(a_{ti})$$

Porte d'oubli :

$$a_{t\phi} = \sum_{i=1}^I W_{i\phi} X_{ti} + \sum_{h=1}^H W_{h\phi} Z_{t-1h} + \sum_{c=1}^C W_{c\phi} S_{t-1c} b_{t\phi} = f(a_{t\phi})$$

Porte de sortie :

$$a_{t\omega} = \sum_{i=1}^I W_{i\omega} X_{ti} + \sum_{h=1}^H W_{h\omega} Z_{t-1h} + \sum_{c=1}^C W_{c\omega} S_{t-1c} b_{t\omega} = f(a_{t\omega})$$

Attention, les signaux sont toujours indicés par la variable temporelle t . Remarquons que les signaux de sortie sont toujours pris à l'instant précédent dans les sommes pondérées.

Concernant la cellule mémoire, son contenu S_c peut être mis à jour ou remis à 0.

On applique dans un premier temps une somme pondérée des entrées et des sorties :

$$a_{tc} = \sum_{i=1}^I W_{ic} X_{ti} + \sum_{h=1}^H W_{hc} Z_{t-1h}$$

Le contenu de la cellule à l'instant t est ensuite recalculé comme une somme de deux termes :

$$S_{tc} = b_{t\phi} S_{t-1c} + b_{t\omega}(a_{tc})$$

- Le premier terme reprend la valeur précédente de la cellule S_{t-1c} , qui peut être annulée par la valeur de la porte d'oubli $b_{t\phi}$;

- Le deuxième terme est l'influence de la somme pondérée, pilotée par la valeur de la porte d'entrée b_{ti} .

Pour terminer, la sortie du neurone LSTM est calculée par une activation h appliquée à la valeur de la cellule mémoire St_c , pilotée par la valeur de la porte de sortie b_{to} :

$$b_{tc} = b_{to} h(St_c)$$

Si le comportement complet d'une cellule LSTM peut sembler compliqué, signalons que nous ne manipulons que des sommes pondérées et des fonctions d'activation, comme c'est le cas pour tous les réseaux de neurones, d'ailleurs !

Bien évidemment, l'ensemble des poids W_{ab} sont appris lors de la phase d'apprentissage.

Couches de LSTM

Comme n'importe quel neurone, les neurones LSTM sont généralement utilisés en couches.

Dans ce cas, **les sorties de tous les neurones sont réinjectées en entrée de tous les neurones.**

Compte tenu de toutes les connexions nécessaires au pilotage de la cellule mémoire, les couches de neurones de type LSTM sont deux fois plus "lourdes" que les couches récurrentes simples, qui elles-mêmes sont deux fois plus lourdes que les couches denses classiques.

Les couches LSTM sont donc à utiliser avec parcimonie !

Afin de réduire le nombre de paramètres des modèles, il existe une variante des LSTM appelée **GRU (Gated Recurrent Unit)**, qui supprime la porte d'oubli. Il a été montré qu'on obtenait des performances quasiment similaires avec moins de paramètres.

Les couches LSTM sont donc généralement combinées avec des couches denses ou des couches convolutionnelles.

Apprentissage des LSTM

Comme nous venons de le constater, les LSTM sont constitués de sommes pondérées, de fonctions d'activations différentiables et de connexions récurrentes.

Ils peuvent donc s'apprendre avec l'algorithme **BPTT** (Backpropagation Through Time, voir chapitre précédent), comme pour les réseaux récurrents classiques, en dépliant le réseau récurrent à travers le temps.

LSTM bidirectionnels : les BLSTM (Bidirectional - LSTM)

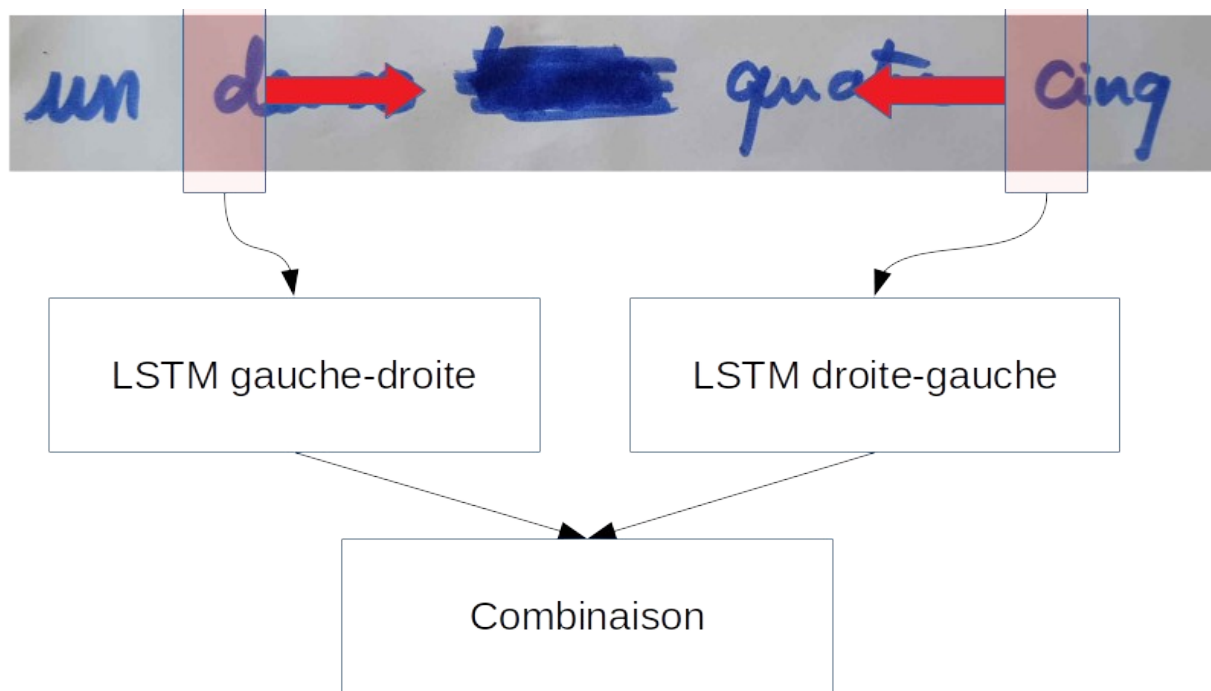
Dans la plupart des problèmes d'analyse de séquence, il est intéressant d'avoir une mémoire du passé pour prendre de bonnes décisions à l'instant t . Mais dans certains problèmes de reconnaissance, il est parfois intéressant de regarder les observations futures, lorsque celles-ci sont disponibles !

Si l'on reprend l'exemple suivant, on peut deviner le texte caché en analysant le contexte passé, mais aussi le contexte futur.



Pour découvrir le texte raturé, on peut se baser sur les mots passés comme sur les mots futurs !

Pour ces problèmes, il est possible d'utiliser un réseau appelé *BLSTM*, qui consiste à dédoubler une couche LSTM, l'une étant apprise pour parcourir le signal de gauche à droite, et l'autre de droite à gauche :



LSTM bidirectionnel

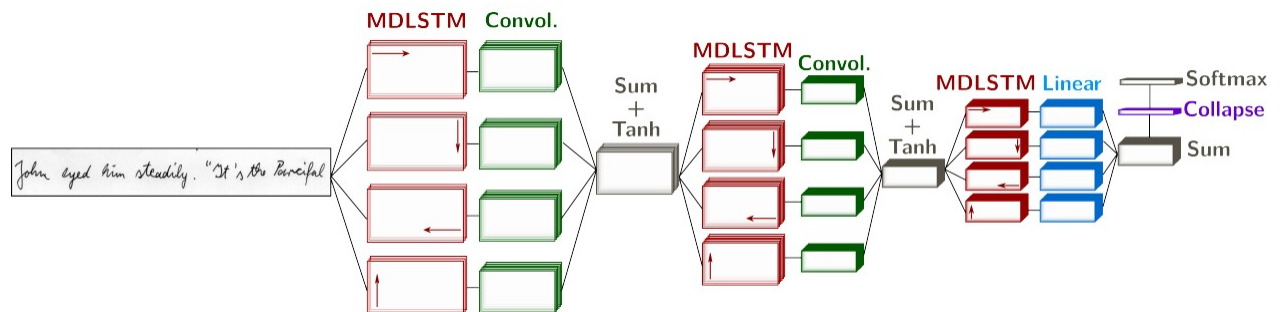
Les deux couches sont combinées afin de prendre les meilleures décisions locales en ayant "vu" ... l'intégralité du signal !

Remarque : l'utilisation de ce type d'architecture n'est **pas possible dans le cas d'applications fonctionnant en temps réel**, comme la reconnaissance de la parole au fur et à mesure que la personne parle, par exemple. En revanche, sur un signal enregistré, c'est adapté.

Les MDLSTM : Multi-dimensional LSTM

Les Bidirectional-LSTM consistent à parcourir un signal à une dimension selon ses deux directions. Les MDLSTM généralisent ce comportement aux signaux à plusieurs dimensions. Un signal à deux dimensions, tel qu'une image, va donc être parcouru selon 4 directions (deux par dimension) :

- Haut -> bas ;
- Bas -> gauche ;
- Gauche -> droite ;
- Droite -> gauche.



Exemple d'utilisation d'un MDLSTM pour la reconnaissance d'écriture, présenté dans l'article L'image est parcourue dans les 4 directions.

Théodore Bluche, Jérôme Louradour, Ronaldo Messina (2017) Scan, Attend and Read: End-to-End Handwritten Paragraph Recognition with MDLSTM Attention. In *14th International Conference on Document Analysis and Recognition (ICDAR)*, 2017.

Les MDLSTM permettent ainsi de **modéliser les dépendances dans des signaux, quelle que soit leur dimension**.

Comme on peut le constater sur la figure précédente, des couches de LSTM sont souvent combinées avec d'autres types de couches neuronales (ici, des couches convolutionnelles et des couches denses).

Aller plus loin

Alex Graves: *Supervised sequence labelling with recurrent neural networks*. Technical University Munich 2008, pp. 1-117.

Alex Graves, Jürgen Schmidhuber: Offline Handwriting Recognition with Multidimensional Recurrent Neural Networks. *NIPS* 2008: 545-552.

Kyunghyun Cho, Bart van Merriënboer, Çaglar Gülçehre, Fethi Bougares, Holger Schwenk, Yoshua Bengio: *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. CoRR abs/1406.1078 (2014).

Dans ce chapitre, nous avons présenté une version améliorée des réseaux de neurones récurrents simples : les **LSTM**, capables de modéliser des dépendances à très long terme.

Les LSTM reposent sur un mécanisme de mémoire interne piloté par des **portes de contrôle**. Tous les éléments étant **différentiables**, ces réseaux s'apprennent par une **rétropropagation** à travers le temps classique.

Nous avons également présenté l'utilisation des LSTM dans des architectures **bidirectionnelles** et **multidimensionnelles**, afin de modéliser les dépendances dans **plusieurs** directions au sein d'un même modèle.

Chapitre 2

Dans ce chapitre, nous allons constater que les réseaux de neurones sont des architectures modulaires, permettant d'empiler des couches diverses, entraînées avec un seul algorithme. Nous allons identifier le rôle de chaque type de couche et présenter des architectures profondes classiques sur des problèmes réels.

Jouez aux LEGO avec des réseaux de neurones

Tout au long de ce cours "**Initiez-vous au Deep Learning**", nous avons présenté différents types de couches de neurones, telles que les couches **denses**, les couches **convolutionnelles**, les couches **récurrentes**, etc.

Quel que soit leur type (dense, CNN, RNN), toutes ces couches sont composées de sommes pondérées et de fonctions d'activations non linéaires, et rien d'autre !

L'empilement d'un nombre important de couches neuronales va permettre d'apprendre des fonctions potentiellement très complexes !

Dans tous les chapitres de ce cours, vous avez peut-être remarqué que pour tous les types de couches, on revenait toujours à l'algorithme d'apprentissage de la **rétropropagation du gradient** pour apprendre les paramètres du réseau (les poids, voir premier cours, chapitre 2).

Il est donc possible d'empiler des couches, quelle que soit leur nature, les unes au-dessus des autres ; l'apprentissage complet du modèle sera effectué par une descente de gradient appliquée à l'ensemble du réseau.

On peut donc jouer aux LEGO, c'est-à-dire empiler les couches en fonction des besoins.

Généralisation de la rétropropagation du gradient

Afin de bien comprendre qu'on peut empiler les couches sans se préoccuper de leur nature pour l'apprentissage, il est important de faire un point sur la généralisation de la rétropropagation du gradient.

La figure suivante permet de s'affranchir de la nature de la couche, comme présenté dans le [cours de Y. Lecun au collège de France](#). Les fonctions F_i sont soit des sommes pondérées (couches linéaires paramétrées par W_i), soit des fonctions d'activation (ReLU, Tanh, etc. ; dans ce cas elles n'ont pas de paramètres W_i) :

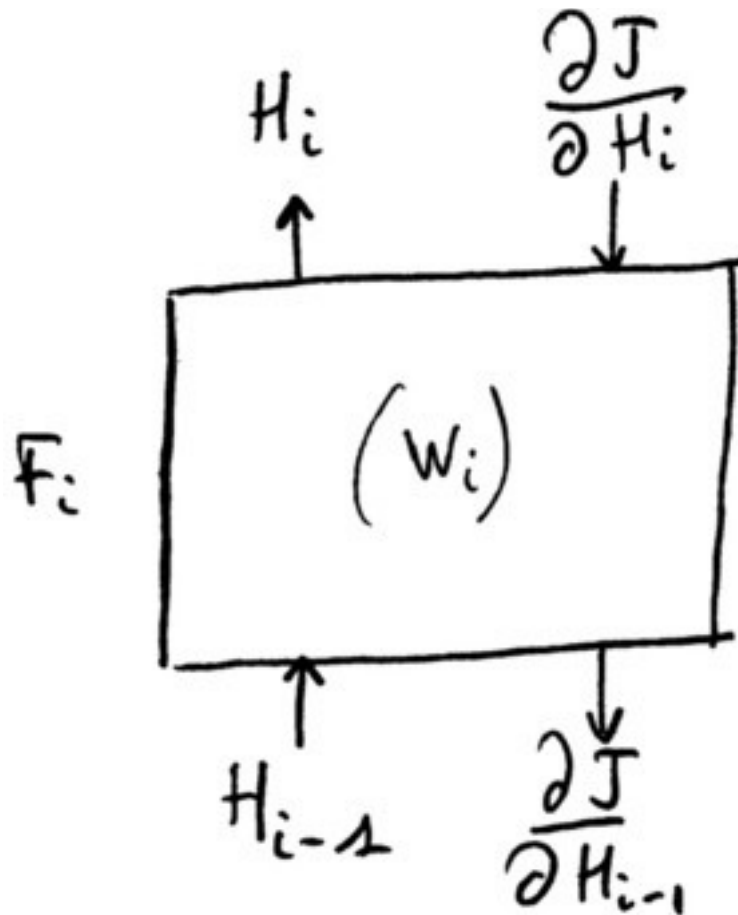


Schéma générique d'une couche de neurones appliquant la fonction F_i à l'entrée H_{i-1} pour calculer la sortie H_i .

En phase de décision (de bas en haut sur le schéma), on propage l'information des entrées vers les sorties, en calculant simplement les H_i successifs :

$$H_i = F_i(H_{i-1})$$

En phase d'apprentissage, on calcule successivement les gradients de l'erreur depuis la sortie vers l'entrée (du haut vers le bas sur le schéma) :

$$\frac{\partial J}{\partial H_{i-1}} = \text{bleu} \frac{\partial J}{\partial H_i} \times \text{magenta} \frac{\partial H_i}{\partial H_{i-1}}$$

Le terme bleu provenant de la couche précédente, et le terme magenta étant facilement calculable en fonction du type de couche considéré.

Rappelons que la seule contrainte pour pouvoir appliquer cet algorithme est que toutes les fonctions F_i utilisées soient différentiables, ce qui est bien sûr le cas des sommes pondérées et des fonctions d'activations communément employées (tanh, softmax, ReLU, etc.).

Maintenant que nous savons "empiler des couches" et les apprendre, faisons le point sur les différents types de couches et identifions le contexte dans lequel il faut les utiliser.

Intérêt des différents types de couches

Couches denses, pour décider

Les couches denses sont les couches **"historiques"** des réseaux de neurones feedforward. Elles consistent en une interconnexion totale entre une couche de neurones et la suivante.

Comme leur nom l'indique (elles sont aussi appelées *fully connected*), ces couches sont assez "lourdes", car elles comportent beaucoup de paramètres.

Elles sont donc assez difficiles à apprendre, et nécessitent un nombre assez important de données pour être correctement entraînées.

Les couches denses doivent être utilisées afin de **prendre une décision**, qu'il s'agisse d'une classification ou d'une régression. Elles sont donc généralement **utilisées comme dernière(s) couche(s) d'un modèle**.

Couches convolutionnelles, pour caractériser

Les couches convolutionnelles (voir cours 1, chapitre 3) sont beaucoup plus légères, puisqu'elles ne comportent que les poids liés aux filtres (dont la taille est généralement réduite, par exemple 3×3 ou 5×5), multipliés par quelques dizaines de filtres.

La tendance est donc à l'utilisation intensive de couches convolutionnelles, au détriment de couches plus lourdes.

Il est donc fréquent de rencontrer des architectures profondes comprenant des "piles" de plusieurs dizaines de couches convolutionnelles.

Rappelons que les couches convolutionnelles sont souvent utilisées avec des couches de pooling, afin de concentrer l'information.

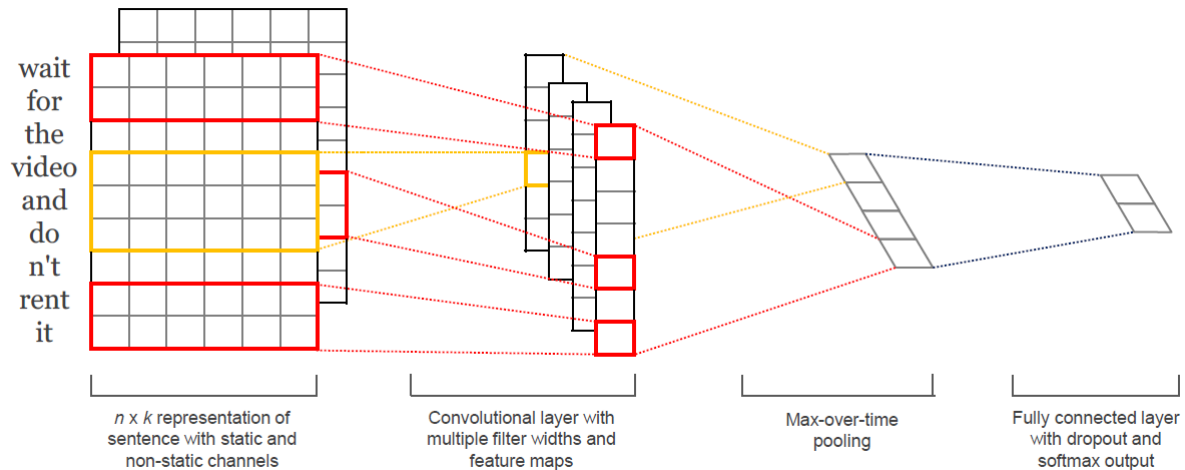
Les filtres appris par les couches convolutionnelles **peuvent être considérés comme des extracteurs de caractéristiques** ; ils sont donc souvent utilisés dans les **couches basses des réseaux**, afin d'**apprendre des représentations** adaptées à la fonction de décision considérée.

De par leur conception, les couches convolutionnelles sont naturellement adaptées au traitement des images, et permettent de caractériser efficacement des contours ou des textures.

Mais **on peut également appliquer des couches convolutionnelles sur n'importe quel type de signal**, tel que du texte ou des signaux sonores.

À titre illustratif, la figure suivante présente un modèle convolutionnel dédié au texte présenté dans l'article "Convolutional neural networks for sentence classification" [1]. Le modèle comporte 4 parties :

- La première consiste à encoder les mots sous la forme d'une représentation numérique de taille fixe.
- La seconde est constituée des fameux filtres convolutionnels.
- La troisième réalise une opération de pooling sur l'axe temporel.
- La dernière est une couche dense traditionnelle permettant de classifier le texte.



Exemple de CNN appliqué à du texte pour la classification de phrases courtes

Couches récurrentes, pour modéliser les dépendances...

Comme nous l'avons vu tout au long de ce chapitre, les RNN permettent de modéliser les dépendances au sein d'un signal.

Ils sont donc adaptés pour tous les problèmes impliquant des séquences. En revanche, les couches récurrentes sont lourdes à cause **des connexions récurrentes**, notamment dans le cas de l'utilisation de cellule LSTM plus performante.

On **limitera donc le nombre de couches récurrentes ainsi que le nombre de neurones dans ces couches au strict nécessaire**. Par exemple, pour traiter un signal sonore, on va typiquement enchaîner un certain nombre de couches convolutionnelles avant d'appliquer une ou deux couches récurrentes.

... ou pour générer des séquences

Rappelons qu'il existe une version "générative" des RNN (voir chapitre 2 de ce cours). Ces RNN permettent de générer itérativement des séquences. Pour cela, le RNN est utilisé dans son mode "classification de séquence", où les observations passées et courantes sont utilisées pour prédire l'élément suivant. La génération de texte pourra par exemple être effectuée mot après mot, ou caractère après caractère.

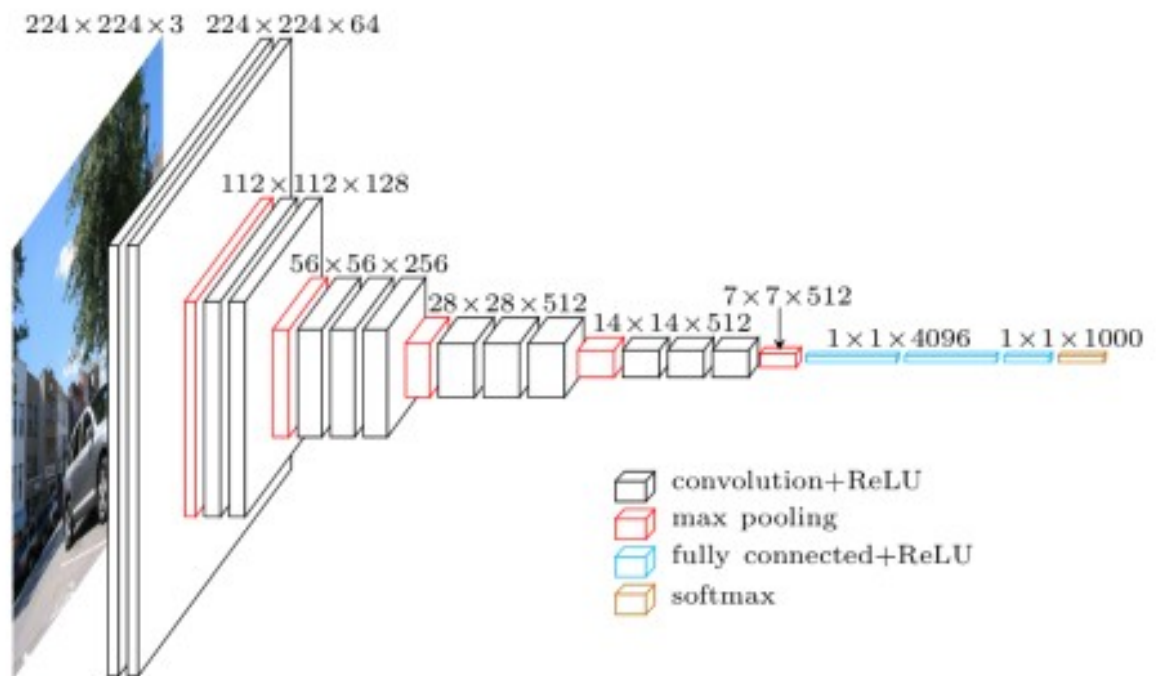
Exemple d'architectures "LEGO"

Dans cette section, nous allons analyser certains modèles profonds "classiques" pour traiter des problèmes donnés. Ils sont tous constitués des types de couches standard résumés ci-dessus.

Exemple de modèle profond pour la classification d'images

Plusieurs architectures profondes ont été proposées pour traiter l'immense base "**ImageNet**", qui contient plus de 14 millions d'images de scènes naturelles (animaux, véhicules, personnes, etc.).

L'une des plus connues s'appelle VGG, proposée dans l'article en anglais "Very deep convolutional networks for large-scale image recognition" [2], destinée à la classification d'images et représentée ci-dessous.



Modèle profond proposé pour la classification d'images.

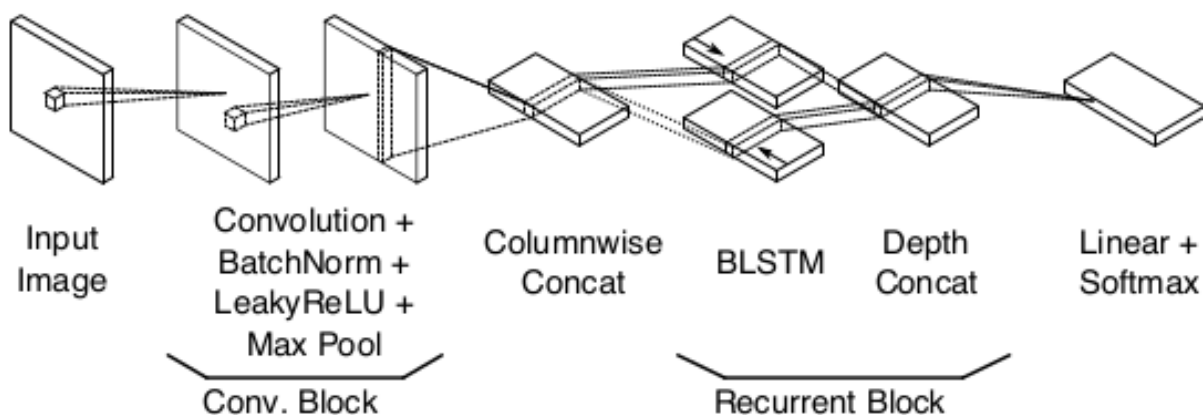
Compte tenu de la tâche à effectuer, les choix suivants ont été faits :

- Les couches basses sont des **couches convolutionnelles (en noir dans la figure ci-dessus)** pour caractériser les images. Chaque couche possède plusieurs filtres, conduisant à plusieurs cartes de caractéristiques (souvent appelés *channels* en anglais). Dans la notation $X \times Y \times Z$, les deux premiers nombres désignent la taille des cartes, et le troisième désigne le nombre de cartes. La version VGG16 comporte 16 couches convolutionnelles ;
- Les couches convolutionnelles sont alternées avec des **couches de max pooling (en rouge)** (ici 2×2) qui réduisent la taille des cartes de caractéristiques pour concentrer l'information spatiale ;

- Afin de classifier les images, 3 **couches denses (fully connected, en bleu)** sont appliquées en sortie des couches convolutionnelles. La dernière fonction d'activation est une "softmax" (en marron) qui permet d'estimer des probabilités *a posteriori*.

Exemple de modèle profond pour la reconnaissance d'écriture

Un autre exemple d'application est la reconnaissance d'écriture. Dans l'article en anglais "Are Multidimensional Recurrent Layers Really Necessary for Handwritten Text Recognition ?", les auteurs proposent un modèle assez classique pour cette tâche, présenté ci-dessous.



Modèle proposé dans la reconnaissance d'écriture manuscrite.

Le modèle comporte quatre parties :

- L'image d'entrée (Input image) est soumise à des **blocs convolutifs** avec batch normalization, activation leaky ReLU et max pooling (Convolution + BatchNorm + leaky ReLU + Max Pool) ;
- Les sorties du bloc précédent sont ensuite **concaténées par colonne** (Columnwise concat)
- Puis soumises à des **blocs récurrents** (BLSTM + depth concat) ;
- La dernière couche du modèle est naturellement une couche dense (appelée *linear* sur le schéma) associée à une fonction d'activation **softmax** pour réaliser la classification en caractères des séquences d'entrée.

L'architecture proposée n'est pas très éloignée du modèle précédent pour la classification d'images. Le modèle est également composé de **couches convolutionnelles** alternées avec des **couches de max pooling** sur les couches basses, destinées à extraire des caractéristiques.

Remarquons la présence de "Batch Normalization" (voire partie précédente) pour normaliser les représentations numériques, et des activations de type "leaky ReLU", une variante des ReLU.

La réelle différence avec le modèle VGG est la présence d'une **couche BLSTM** destinée à **traiter l'aspect séquentiel** de l'écriture manuscrite.

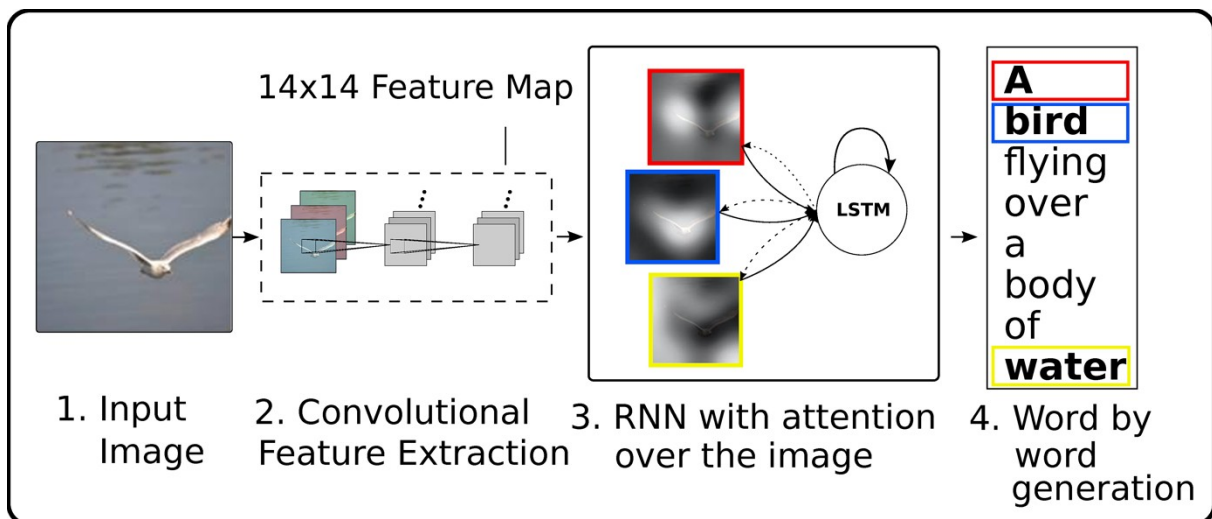
Exemple de modèle profond pour la génération de légende

Le dernier modèle présenté est le plus complexe. Il est destiné à réaliser la tâche suivante :

« Étant donné une image, on cherche à générer une légende décrivant la scène ».

L'architecture proposée sur le schéma suivant est composée de plusieurs modules :

- 1) Image d'entrée,
- 2) Extraction de caractéristiques convolutionnelles,
- 3) Réseau récurrent avec modèle d'attention sur l'image, et 4) génération mot par mot.



Architecture proposée dans l'article de Xu et al. pour la génération de légende.

- 1) Le premier module est un module caractérisant l'image. Il est naturellement composé de **couches convolutionnelles et de pooling**, comme dans les autres modèles présentés ci-dessus ;
- 2) Le dernier module est un module permettant de générer la description textuelle de l'image. Il s'agit d'un **réseau de neurones récurrent de type LSTM, utilisé en mode génératif**. Il génère itérativement chaque nouveau mot à ajouter à la légende. Ce réseau réalise donc une tâche de classification parmi tous les mots de la langue, anglaise en l'occurrence. Un mot "fin de légende" est ajouté à cette liste pour décider quand la légende est terminée ;
- 3) Le troisième module permet de "relier" les deux autres modules : il s'agit d'un **modèle d'attention**. Il permet de présenter au modèle génératif la meilleure partie de l'image, en fonction du contexte, pour chaque nouveau mot. Par exemple, le modèle d'attention va se porter sur les ailes d'un oiseau au moment de sortir le mot "flying" (voler).

- 4) L'implémentation du modèle d'attention est réalisée par un modèle récurrent, généralement de type LSTM. Nous renvoyons à l'article en anglais "Show, attend and tell : Neural image caption generation with visual attention" [4] pour une description plus détaillée du modèle d'attention.

Concernant l'apprentissage du modèle, il est réalisé en plusieurs étapes. Le modèle convolutionnel est un modèle pré appris sur la base *ImageNet*, et le modèle génératif est également pré appris sur une très grosse base de texte telle que *Wikipédia*. L'apprentissage du système complet, y compris le modèle d'attention, est finalisé sur une base de couples (image, légende) annotés.

Allez plus loin

[1] Kim, Y. (2014). Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*.

[2] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv :1409.1556*.

[3] Joan Puigcerver: Are Multidimensional Recurrent Layers Really Necessary for Handwritten Text Recognition? *ICDAR 2017*: 67-72

[4] Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhudinov, R., ... & Bengio, Y. (2015, June). Show, attend and tell: Neural image caption generation with visual attention. In *International conference on machine learning* (pp. 2048-2057).

En résumé

Dans ce chapitre, nous avons synthétisé le rôle de chaque type de couche. Nous avons montré qu'une architecture adaptée pouvait être mise en place pour traiter des problèmes variés. Il suffit pour cela :

- **d'empiler** les couches adéquates ;
- **d'entraîner** le modèle ainsi construit par la rétropropagation du gradient.

Nous avons ensuite présenté trois modèles classiques pour la reconnaissance d'image, la reconnaissance d'écriture manuscrite et la génération de légende à partir d'une image.

La question légitime qui interpelle en début de présentation est la suivante : comment une machine peut-elle apprendre ? Nous allons répondre à cette question dans ce chapitre et pour cela nous allons présenter les concepts essentiels vous permettant de comprendre comment un réseau de neurones peut apprendre

■ Une IA = une fonction mathématique

Comment sont représentées les informations dans un problème d'apprentissage. En sortie, le réseau peut estimer un âge, des scores, des probabilités... Toutes ces informations peuvent être encodées sous la forme d'un nombre ou d'un vecteur de nombres. Pour les données en entrée, on trouve aussi des informations codées avec des valeurs numériques comme :

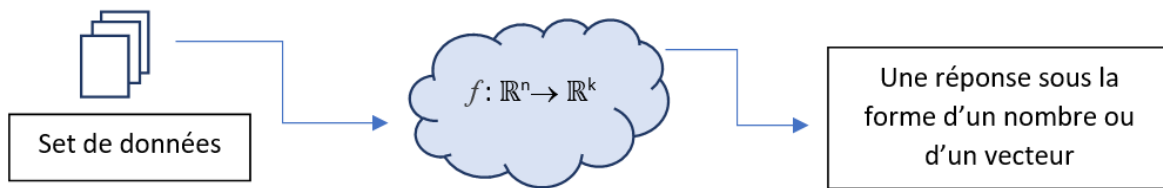
- Une image qui se représente sous la forme d'un tableau de valeurs 8 bits allant de 0 et 255.
- Un son qui se modélise comme une suite de valeurs issues de l'enregistrement depuis un micro.
- Dans un jeu comme Pac-Man, la position du héros, la position des fantômes, le score de partie et la carte des murs et des Pac-gommes forment un vecteur de valeurs décrivant entièrement l'état du jeu à un instant donné.
-

Ainsi, en entrée comme en sortie, nous avons des valeurs numériques. Construire une IA prenant des données en entrée et fournissant une réponse en sortie s'apparente à mettre en place une **fonction mathématique** f de $R_n \rightarrow R_k$. Ceci a un impact immédiat sur le fonctionnement du système, car si nous modélisons notre IA par une fonction mathématique, cela implique que les entrées et les sorties doivent toujours être de même taille.

De cette façon :

- Pour un set d'images, il faut retailler et redimensionner les images pour qu'elles aient toutes la même résolution et la même profondeur comme : 32x32x3 octets.
- Pour des enregistrements sonores, il faut faire en sorte que tous les échantillons fassent la même durée avec la même vitesse d'échantillonnage en allongeant la durée des plus courts par exemple.
- Pour une analyse de texte, il faut que toutes les phrases contiennent le même nombre de mots quitte à ajouter des mots « vides » pour compléter les fins de phrase.

Voici donc notre premier système :

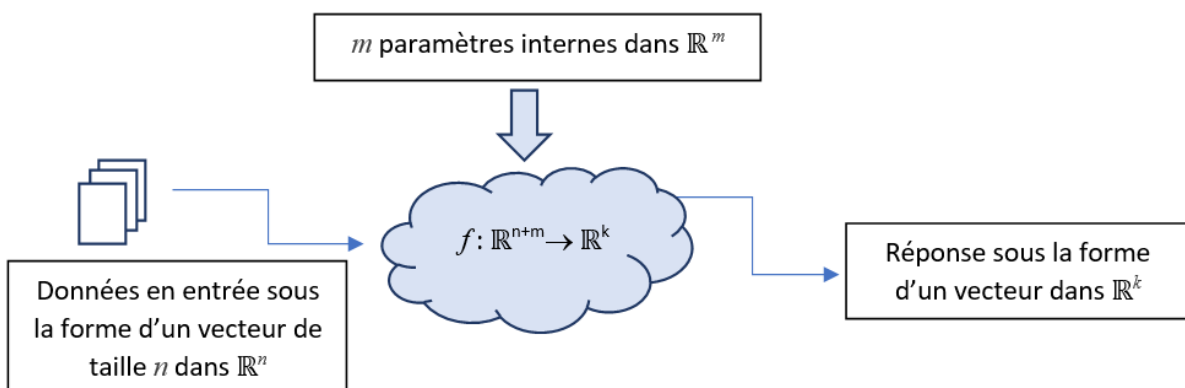


Note

Les réseaux de neurones, que nous présenterons plus tard, permettent de générer ce type de fonctions.

▪ Les paramètres d'apprentissage

Nous disposons d'une fonction f fournissant une réponse pour des informations fournies en entrée. Maintenant, nous voulons faire en sorte que cette fonction puisse évoluer sinon aucun apprentissage ne sera possible. Pour cela, nous allons ajouter des paramètres supplémentaires, dit **paramètres internes** ou **paramètres d'apprentissage** aussi appelés **poids** du réseau (**weights** en anglais). Ils vont être utilisés comme des entrées supplémentaires de la fonction f au même titre que les données en entrée, ainsi nous obtenons la modélisation suivante:



L'ajout de ces poids dans le réseau va permettre l'apprentissage : en effet, si les réponses fournies par la fonction f sont incorrectes, on peut faire évoluer ces paramètres internes pour tenter d'obtenir une réponse éventuellement meilleure.

Ainsi, durant la phase d'apprentissage, les données en entrée sont des données « constantes » au sens où elles ne sont pas modifiées durant cette phase. La fonction f est choisie et

conservée telle quelle durant toute la phase d'apprentissage. **Les seuls éléments pouvant évoluer durant la phase d'apprentissage sont les paramètres internes.**

Prenons un exemple basique. Supposons que nous avons une seule valeur en entrée notée x . Prenons comme fonction f un polynôme du second degré de la forme $f(x,a,b,c)=ax^2+bx+c$. Cette fonction f possède ainsi trois paramètres internes a , b et c . Différentes valeurs pour ces paramètres internes vont fournir des réponses différentes pour une même valeur d'entrée $x=2$:

- Pour $a=1, b=3$ et $c=5$, la réponse est égale à : $f(x)=2^2+3.2+5=15$
- Pour $a=3, b=1$ et $c=3$, la réponse est égale à : $f(x)=3.2^2+1.2+3=17$

Note

Comment initialiser les paramètres internes en début de d'apprentissage ? Impossible de répondre à cette question. Les paramètres seront donc initialisés aléatoirement et ainsi deux apprentissages avec des valeurs initiales différentes peuvent donner des IA très différentes.

■ L'apprentissage supervisé

Il nous manque encore une notion essentielle pour que l'apprentissage puisse exister : il faut que l'on soit capable d'évaluer la qualité de la réponse. Dans l'**apprentissage supervisé**, nous mettons en place une approche utilisant des vérités terrain : une liste d'échantillons dont on connaît exactement la réponse. Ainsi, durant la phase d'apprentissage, on utilise des bases de données fournissant des entrées ainsi que les valeurs attendues en sortie du réseau :

- Pour l'évaluation de l'âge, nous traitons une base de photos de personnes et pour chaque photo nous connaissons l'âge exact de la personne.
- Pour la classification : on dispose d'un set d'images et pour chacune d'entre elles, on va donner la catégorie associée : chien, chat, grenouille...
- Pour la description automatique d'images : pour chaque image, on dispose d'une phrase décrivant le contenu de l'image.
- Pour la transcription automatique (Speech to Text), nous disposons de plusieurs phrases enregistrées ainsi que leurs transcriptions.

L'apprentissage non supervisé est une thématique de recherche consistant à catégoriser, par exemple, des photos d'animaux sans connaissance à priori des espèces présentes sur les images.

▪ L'erreur d'une estimation

Une fois que nous avons des échantillons et les réponses attendues, nous pouvons évaluer la qualité du réseau. Pour cela, nous mettons en place une **fonction d'erreur** indiquant si la réponse de la fonction f est éloignée de la réponse attendue. Par convention, la fonction d'erreur doit être positive et sa valeur doit augmenter lorsque la réponse s'éloigne de la valeur recherchée.

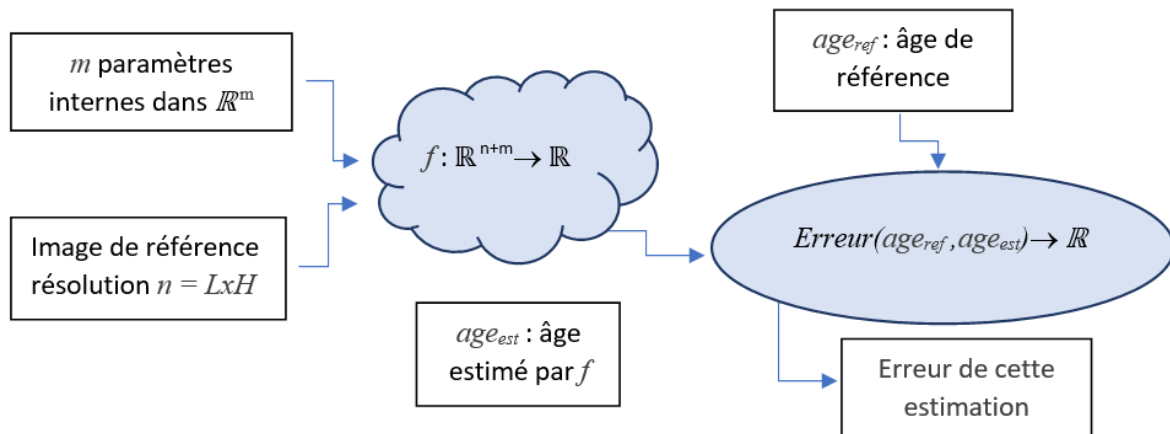
Prenons comme exemple le problème de la détection de l'âge d'une personne. En utilisant une base d'images, nous connaissons, pour chaque image, l'âge de référence $ageref$ de la personne présente sur l'image. Nous notons l'âge estimé $ageest$ par la fonction f . Nous pouvons donc utiliser la fonction d'erreur suivante :

$$\text{Err}(ageref, ageest) = \text{abs}(ageref - ageest)$$

Voici différentes erreurs pour une même image en entrée où la personne a 37 ans :

- Age estimé : 37, $\text{Erreur}(...) = |37 - 37| = 0$ car on a trouvé la bonne réponse.
- Age estimé : 39, $\text{Erreur}(...) = |37 - 39| = 2$ faible erreur, 2 ans trop vieux.
- Age estimé : 30, $\text{Erreur}(...) = |37 - 30| = 7$ l'erreur augmente car l'écart augmente.
- Age estimé : 87, $\text{Erreur}(...) = |37 - 87| = 40$ soit 40 ans trop vieux, l'erreur est forte.

Dans cet exemple, la valeur de l'erreur correspond à un nombre d'années. Mais rarement l'erreur aura une unité, nous ne saurons pas si 0.1 ou 150 correspond à une erreur importante ou non. Cependant, **si la valeur de l'erreur baisse cela suggère que la qualité de la réponse augmente et réciproquement**. Intégrons maintenant la fonction Erreur dans notre modèle :



Durant la phase d'apprentissage, nous utilisons principalement la valeur de l'erreur. Ainsi, nous étudions la fonction Err définie ainsi :

$$\text{Err}(\mathbf{I}, \text{refI}, \mathbf{f}, \mathbf{w}) = \text{Erreur}(\mathbf{f}(\mathbf{I}, \mathbf{w}), \text{refI})$$

Avec :

- **I** : donnée en entrée (une image par exemple).
- **refI** : réponse exacte associée.
- **f** : fonction d'apprentissage.
- **w** : paramètres d'apprentissage (weights).

L'apprentissage supervisé, pour être efficace, doit disposer de plusieurs milliers d'échantillons en entrée. Plus la taille de l'échantillon de référence est importante, plus vous augmentez la qualité de l'apprentissage. On peut citer quelques bases annotées utilisées régulièrement dans la formation et la recherche :

- [MNIST](#) : base d'images correspondant aux chiffres manuscrits de 0 à 9 ; utilisée à l'origine pour construire les machines automatisées de tri postal ; 60 000 images de résolution de 28x28 en niveaux de gris.
- [CIFAR10](#) : base d'images pour la classification d'images. 60 000 images, résolution de 32x32 en couleur, 10 classes : avion, voiture, oiseau, chat, daim, chien, grenouille, cheval, navire et camion.
- [CIFAR100](#) : 100 classes cette fois et 600 images par classe.

- [ImageNet](#) : base d'images qui sert de référence pour le problème de classification : 1000 classes, plus d'un million d'images en couleur de taille variable.
- [COCO](#) : base d'images pour la segmentation et la description d'images : 80 000 images accompagnées de leurs descriptions.

▪ L'erreur totale

Pour estimer l'erreur totale sur l'ensemble de la base d'apprentissage, il nous suffit de sommer les erreurs induites par chaque entrée de la base :

$$\text{ErrTot}(\mathbf{f}, \mathbf{w}, \text{DataSet}) = \sum_{(\mathbf{I}, \text{refI}) \in \text{DataSet}} \text{Err}(\mathbf{I}, \text{refI}, \mathbf{f}, \mathbf{w})$$

▪ Comment apprendre ?

Si nous examinons la fonction $\text{ErrTot}(\dots)$, elle prend en entrée une fonction $\mathbf{f}$, des paramètres internes $\mathbf{w}$ et une base d'apprentissage. Sur ces trois entrées, deux sont fixes : la base et la fonction $\mathbf{f}$ car la base ainsi que la fonction ne changent pas durant la phase d'apprentissage. Les seules entrées susceptibles d'évoluer sont les paramètres d'apprentissage $\mathbf{w}$. Par conséquent, lorsque nous avons choisi une fonction $\mathbf{f}$ et la base à traiter, le problème de l'apprentissage se ramène à cette formulation :

$$\min_{\mathbf{w}} \text{ErrTot}(\mathbf{w}, \mathbf{f}, \text{DataSet})$$

Vous devez ici reconnaître un problème d'optimisation : la recherche du minimum d'une fonction. Si vous n'êtes pas familier avec ce domaine, voici le code d'une méthode d'optimisation naïve pour vous prouver que toute fonction d'erreur peut être minimisée :

Function $\text{OptimNaif}(\text{ErrTot}, \mathbf{w}, \mathbf{f}, \text{DataSet})$:

```

ErreurMin = +∞
Tant qu'il me reste du temps :

    # on génère des poids w' dans le voisinage de w
    pour i allant de 0 à len(w) :
        w'[i] = w[i] + pas * rand(-1,1)

```

```
Erreur = ErrTot(w,f,DataSet)
```

```
# si les poids génèrent une erreur moindre, on les garde
```

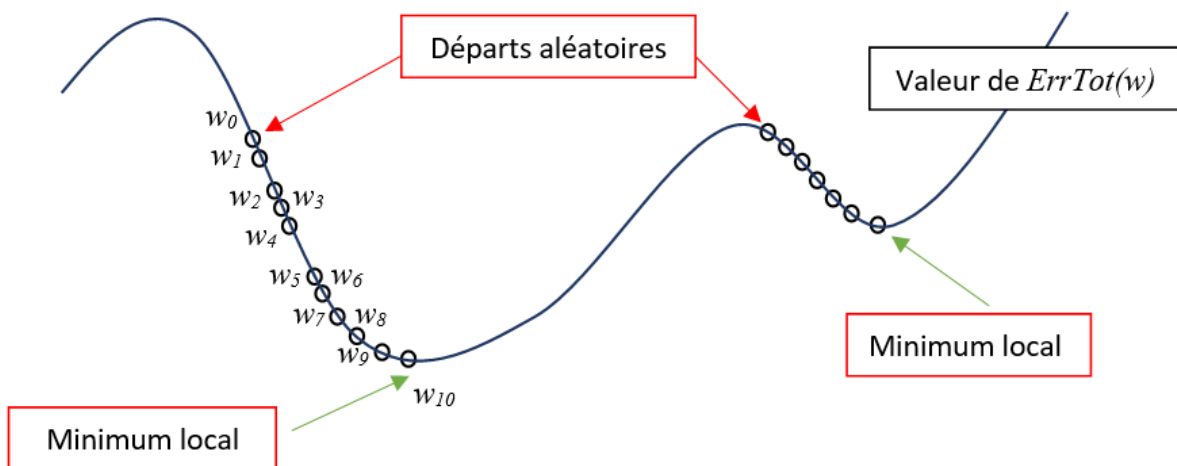
```
Si Erreur < ErreurMin :
```

```
    w=w'
```

```
    ErreurMin=Erreur
```

Successivement, nous testons de nouveaux poids pour le réseau dans le voisinage des poids courants. Si l'erreur totale sur l'ensemble des échantillons s'avèrent moindres, nous conservons ces nouveaux paramètres. Ainsi, en itérant, l'erreur continue à diminuer. L'apprentissage est opérationnel !

Néanmoins, ni cette méthode d'optimisation naïve, ni aucune autre méthode d'optimisation peut nous garantir de converger vers l'erreur minimale. En effet, il existe le problème des minimas locaux qui piégeront tout algorithme de minimisation. Voici un exemple :



Dans la partie gauche du graphique, les paramètres internes sélectionnés itérativement : w_i , positionnent la fonction $ErrTot()$ dans une pente qui descend vers un minimum local. En utilisant notre algorithme naïf, nous descendons le long de cette pente chaque fois que de meilleurs paramètres internes w_i sont trouvés.

En commençons à droite, nous minimisons, de la même manière, l'erreur totale à chaque changement de paramètres internes. Cependant, en suivant cette pente, nous atteignons cette fois un minimum local moins intéressant que le premier. Il n'existe pas de moyen évident pour « s'échapper » de ce minimum. Bien souvent, on va relancer la phase d'apprentissage plusieurs fois à partir d'une autre position pour espérer trouver un meilleur optimum.

Note

Vous pouvez reprendre une phase d'apprentissage, même plusieurs jours après l'arrêt de la phase initiale. Pour cela, il vous suffit de relancer l'algorithme d'optimisation avec la fonction et les derniers paramètres internes choisis pour essayer d'obtenir de nouveaux meilleurs paramètres.

▪ Les écueils de l'apprentissage

Bien que les réseaux de neurones aient permis de grandes avancées en quelques années, il faut rester humble, car les réseaux d'aujourd'hui sont les résultats des travaux de plusieurs milliers de chercheurs ayant effectués des milliers de tentatives avant d'arriver à une solution fonctionnelle. On l'oublie trop souvent. Voici les différents écueils que vous pouvez rencontrer lors de la mise en place d'une méthode d'apprentissage :

La non-convergence : une phase d'apprentissage peut ne pas converger ! En effet, il se peut que la courbe de l'erreur oscille fortement à chaque itération montrant que les poids trouvés sont associés à un réseau fournissant des réponses instables.

Un mauvais score : votre réseau peut converger mais l'erreur peut rester haute traduisant des prédictions globalement mauvaises. Ne soyez pas impressionnés par un pourcentage de bonnes réponses : un réseau qui prédit le sexe d'une personne avec un taux de réussite de 60% ne fait en fait que 10% de mieux qu'une réponse aléatoire fournissant de base une performance de 50 %.

Le surapprentissage : vous avez entraîné un réseau qui obtient un taux de bonnes prédictions de 99%, c'est un miracle ! Non pas forcément. En effet, si trop de neurones sont présents dans le réseau, il peut apparaître une sorte de phénomène d'apprentissage par cœur. Ainsi, le réseau arrive à identifier chaque échantillon en entrée et à fournir la réponse attendue. Pour détecter ce phénomène, on sépare la base d'apprentissage en deux : on garde 80% des données pour la phase d'apprentissage et 20% des données pour la phase de validation. On ne présente jamais le groupe de validation durant l'apprentissage. Une fois cette dernière terminée, les performances du réseau sont évaluées uniquement sur la base de validation : des données « nouvelles » que le réseau n'a jamais traitées. Si un écart de performance se fait ressentir entre les deux échantillons, on peut supposer qu'il y a eu surapprentissage. Cette approche représente une option efficace pour détecter le surapprentissage.

L'évidence : vous avez entre les mains un système capable d'apprendre et si vous le mettez dans des situations évidentes, il ne va pas s'embêter il va aller directement à la conclusion la plus rapide ! Ainsi, si vous lancez un problème de classification sur des images de grenouilles, de dauphins et de lions, en quelques secondes, vous allez arriver à un score de 100% de

prédictions justes. Avez-vous construit un réseau hyper-performant ? Avez-vous découvert un algorithme d'optimisation exceptionnel ? Non rien de cela... Inutile d'analyser précisément le contenu de l'image : en effet, la couleur la plus présente dans l'image permet de classifier facilement ces trois catégories. Ainsi le réseau associe une image plutôt verte à la catégorie grenouille, une image plutôt jaune à la catégorie lion et une image plutôt bleue à la catégorie dauphin.

Des ensembles déséquilibrés : si vous cherchez à classifier des images de chats et de chiens et que vous fournissez 1000 photos de chats et 10 photos de chiens, le réseau, quel qu'il soit, va afficher un très bon score de 99% de prédictions exactes ! Le réseau va vite détecter qu'en répondant toujours « chat » à la question, il a ainsi 99% de chance de voir juste car vous avez une majorité de photos de chats dans votre base.

Note

Lorsque vous manquez d'échantillons pour votre apprentissage, une technique consiste à utiliser du *data augmentation* pour simuler des données supplémentaires. On peut par exemple citer pour l'image : une légère translation / rotation, l'ajout de bruit ou de tâches dans l'image d'origine. Ces techniques ont aussi pour objectif de rendre le réseau plus robuste aux perturbations.

Mod 2. La méthode de descente du gradient

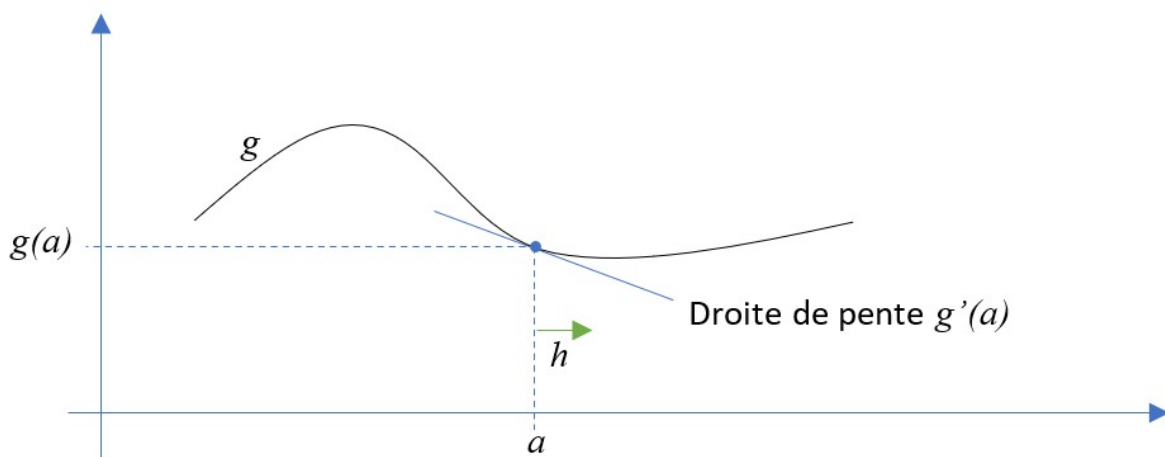
Nous avons vu que l'apprentissage requiert la minimisation d'une fonction d'erreur de $\mathbb{R}^n \rightarrow \mathbb{R}$. Pour cela, une méthode très répandue existe : la méthode du gradient. Cette dernière permet itérativement de diminuer la valeur de l'erreur jusqu'à converger vers un minimum local. Dans ce chapitre, nous introduisons son principe, tout d'abord en traitant une fonction de $\mathbb{R} \rightarrow \mathbb{R}$, puis une fonction de $\mathbb{R}^2 \rightarrow \mathbb{R}$ pour terminer en généralisant la méthode aux fonctions de $\mathbb{R}^n \rightarrow \mathbb{R}$.

■ Pour une fonction de $\mathbb{R} \rightarrow \mathbb{R}$

Durant le processus d'apprentissage, nous cherchons à minimiser l'erreur totale. Supposons que nous traitons une fonction de $\mathbb{R} \rightarrow \mathbb{R}$. Dans ce cas particulier, la notion de gradient correspond à la notion de dérivée. Nous rappelons ici la formule d'approximation d'une fonction :

$$g(a+h) = g(a) + h \cdot g'(a) + \epsilon(h) \quad \text{avec} \quad \lim_{h \rightarrow 0} \epsilon(h) = 0$$

Ce qui signifie qu'autour de la valeur a , pour une valeur de h « petite », la fonction g se comporte comme une droite de pente $g'(a)$:



En conclusion, pour minimiser la valeur $g(a)$, il faut se déplacer dans la direction opposée du signe de $g'(a)$. En effet, si la dérivée $g'(a)$ est négative, la fonction g décroît localement autour de a . Par conséquent, il faut augmenter la valeur de a , en se déplaçant vers la droite, pour diminuer la valeur courante de la fonction g . Nous utilisons cette approche pour mettre en place l'algorithme de descente du gradient :


```

Function GradientDescent(g,a) :
  pas = 0.01
  tant que du temps est disponible :
    grad = g'(a)    # dans notre exemple, la dérivée donne le gradient
    a = a - pas * grad # le signe - fait progresser dans la direction opposée au signe de grad

```

Note

La méthode de descente du gradient ne garantit pas que la valeur de la fonction diminue à chaque itération. Par exemple, lorsque nous sommes proches d'un minimum, suivant la valeur du *pas*, on peut « sauter » par-dessus ce minimum pour atteindre finalement une valeur courante plus importante.

Avertissement

Dans la suite, nous utilisons la notation x pour désigner une variable libre alors que la notation $x\text{---}$ va désigner la valeur prise par cette variable à un moment du programme. Ainsi, lorsque $f(x)=3x^2+2$, nous pouvons écrire $f'(x)=6x$ et pour $x\text{---}=5$, nous écrirons : $f'(x\text{---})=30$. Cette notation permet de lever certaines ambiguïtés.

■ Pour une fonction de $\mathbb{R}^2 \rightarrow \mathbb{R}$

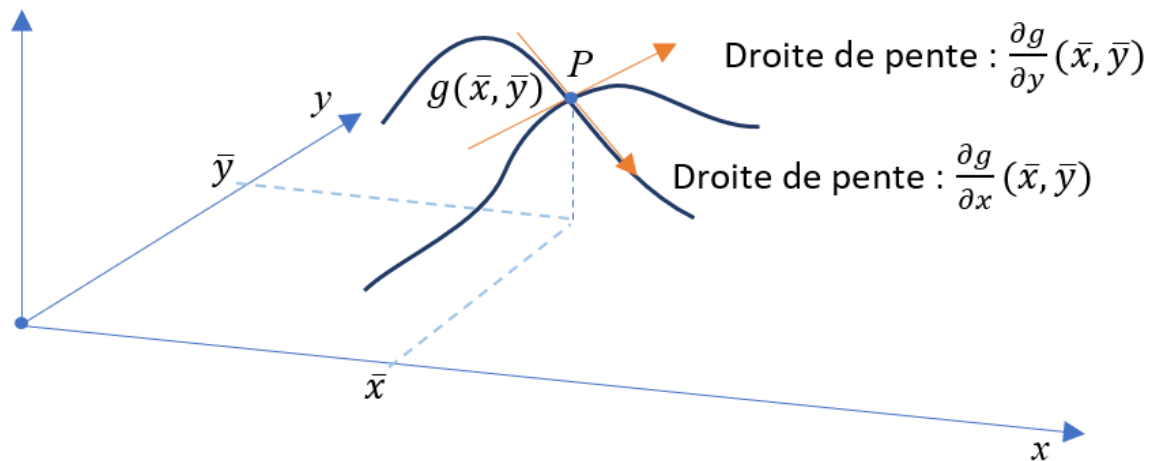
Pour minimiser une telle fonction, nous utilisons la notion de dérivée partielle qui étend la notion de dérivée d'une fonction de $\mathbb{R} \rightarrow \mathbb{R}$ au cas général d'une fonction de $\mathbb{R}^n \rightarrow \mathbb{R}$. Prenons l'exemple d'une fonction à deux variables $g(x,y) \rightarrow \mathbb{R}$. En ajoutant un paramètre de perturbation h , nous obtenons des formules d'approximation similaires au cas précédent :

$$g(x+h,y) = g(x,y) + h \cdot \partial g \partial x(x,y) + \epsilon(h) \quad \text{avec} \quad \lim_{h \rightarrow 0} \epsilon(h) = 0$$

et

$$g(x,y+h) = g(x,y) + h \cdot \partial g \partial y(x,y) + \epsilon(h) \quad \text{avec} \quad \lim_{h \rightarrow 0} \epsilon(h) = 0$$

Comment s'interprètent les deux dérivées partielles ? Si nous faisons uniquement évoluer la valeur de x , nous obtenons comme dans le cas 1D une droite tangente au point courant P . De la même manière, si nous faisons évoluer y , nous obtenons une autre droite tangente en ce point. En combinant ces deux directions tangentes, nous obtenons un plan tangent qui approche le comportement de la fonction g autour du point P :



Nous utilisons la notation ∇g pour désigner le **gradient** de la fonction g . Ce vecteur correspond à l'ensemble des dérivées partielles de la fonction g :

$$\nabla g = [\partial g / \partial x, \partial g / \partial y]$$

Grâce à cette nouvelle notation, on peut regrouper les deux formules d'approximation précédentes en une seule. Ainsi, avec $h = [h_0, h_1]$ et ∇g deux vecteurs colonnes, on peut écrire :

$$g(x+h_0, y+h_1) = g(x, y) + t h \cdot \nabla g + \epsilon(h)$$

Ainsi pour minimiser la valeur de g , il suffit de choisir $h = -\gamma \cdot t \nabla g$ avec $\gamma > 0$. En effet, nous obtenons :

$$g(x+h_0, y+h_1) \sim g(x, y) - \gamma t \nabla g \cdot \nabla g$$

Or comme $\gamma \nabla g \cdot \nabla g$ donne un réel positif, ce choix nous garantit que la valeur courante de la fonction g va diminuer dans cette direction (si γ reste suffisamment petit).

■ Le cas général

Nous étudions maintenant le cas général avec une fonction g de $\mathbb{R}^n \rightarrow \mathbb{R}$. L'utilisation du gradient permet d'étendre cette formule facilement. Ainsi, avec X et H désignant deux vecteurs de $\mathbb{R}^n$, nous avons :

$$g(X+H) = g(X) + t H \cdot \nabla g + \epsilon(H)$$

Comme dans le cas précédent, il suffit de prendre $H = -\gamma \nabla g$ avec $\gamma > 0$ suffisamment petit pour garantir que nous minimisons la valeur courante de la fonction. Nous présentons ci-dessous l'algorithme de descente du gradient construit en appliquant itérativement cette formule :

Function GradientDescent($g, \bar{X}$) :

$\gamma = 0.01$ # valeur du pas
 tant que du temps est disponible :
 $\text{grad} = \nabla g(\bar{X})$ # calcule les valeurs du gradient ∇g en $\bar{X}$
 $\bar{X} = \bar{X} - \gamma * \text{grad}$ # le signe - fait progresser dans la direction opposée au gradient

Pour une fonction de g de $\mathbb{R}^n \rightarrow \mathbb{R}$, le gradient ∇g est un vecteur de taille n égale à la taille du vecteur X . En effet, le gradient est un vecteur avec n paramètres correspondant aux n dérivées partielles de g .

Par conséquent, on peut effectuer le calcul $X = X - \gamma * \nabla g(X)$ car X et $\nabla g(X)$ sont deux vecteurs de même dimension. A noter que le paramètre γ est un réel positif et que le signe $*$ dans $\gamma * \nabla g(X)$ désigne la multiplication entre un réel et un vecteur.

Nous obtenons donc par dérivation :

- Vecteur de $\mathbb{R}^n$

$$e^{-i\omega t} = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$$

- Vecteur $\mathbb{R}^n$

$$I_n = 1 \text{ } I_m(X_n \cap Y_n) \cap \sum_{i=0}^n 1$$

Ainsi si nous dérivons l'Axiome $\mathbb{R}^{nn}$ nous pouvons alors en déduire que les termes majorés de valeur R et R'' pourront se composer de la manière suivante :

$$\sin \alpha \pm \sin \beta = 2 \sin \frac{1}{2}(\alpha \pm \beta) \cos \frac{1}{2}(\alpha \mp \beta) e^{-i\omega t} \rightarrow \max_{0 \leq x \leq 1} x e^{-x^2}$$

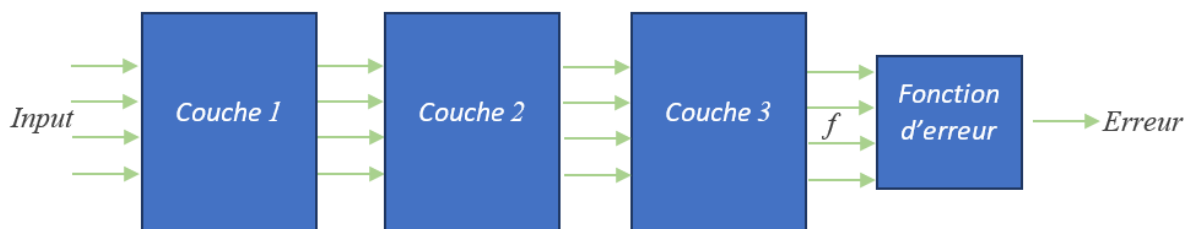
$$\cos \alpha + \cos \beta = 2 \cos \frac{1}{2}(\alpha + \beta) \cos \frac{1}{2}(\alpha - \beta)$$

$$\begin{bmatrix} \cos & \cdots & Rn \\ \vdots & \ddots & \vdots \\ Rn' & \cdots & \cos n \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Mod 3. La rétropropagation

■ Un réseau séquentiel

Les réseaux de neurones **séquentiels**, avec lesquels nous allons travailler, sont décrits sous forme de **couches** (layer). Chaque couche prend en entrée les informations en sortie de la couche précédente, puis effectue ses traitements pour transmettre ses résultats à la couche suivante. L'ensemble des couches constituent la fonction f correspondant à la prédiction de l'IA. Ensuite vient l'application de la fonction d'erreur qui permet d'obtenir comme résultat final un unique nombre réel correspondant à l'erreur.



Cette partie présente la méthode utilisée par les librairies d'apprentissage pour calculer le gradient d'un réseau y compris les plus complexes. Cette méthode, appelée rétropropagation (Back Propagation), a été inventée par Yann le Cun et ses coauteurs ce qui leur valut le prix Turing en 2018. Aujourd'hui Yann le Cun est devenu directeur du centre de recherche en IA de Facebook appelé FAIR « Facebook Artificial Intelligence Research ». **Pour la petite histoire, Yann le Cun est un ancien élève d'ESIEE Paris** diplômé en 1983.

Avertissement

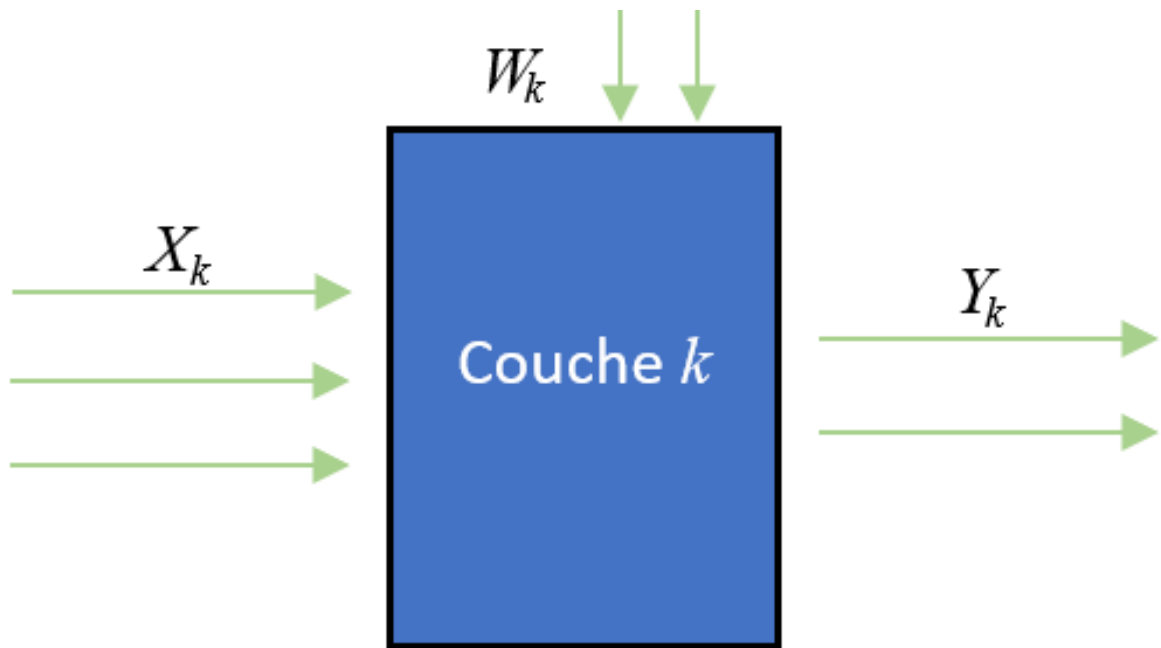
Nous utilisons une notation en majuscules pour désigner une variable correspondant à un vecteur.

Nous notons:

- X_k le vecteur des entrées de la couche k
- Y_k le vecteur des sorties de la couche k , à noter que $Y_k = X_{k+1}$.

- W_k le vecteur des paramètres internes de la couche k .

Nous rappelons ces notations dans le schéma suivant :



▪ Principe général

Pour appliquer l'algorithme de descente du gradient, il faut déterminer le gradient de la fonction erreur par rapport aux paramètres internes de chaque couche.

▪ Démarrage

Nous démarrons le traitement à partir de la sortie du réseau au niveau de la fonction d'erreur. Supposons que le réseau ait $n+1$ couches, la fonction d'erreur prend donc comme entrée le vecteur Y_n . Par conséquent nous avons $Err(Y_n) = F_{nt}Err(Y_n)$. Ainsi, nous obtenons le premier gradient qui va initier la rétropropagation :

$$\nabla Y_n Err = \nabla Y_n F_{nt} Err$$

Avertissement

Le symbole ∇X indique que le gradient est construit à partir des dérivées partielles associées aux composantes du vecteur X .

▪ Exemple

Supposons que $Y_n = (y_0, y_1)$.

Considérons la fonction d'erreur $F_{ntErr}(y_0, y_1) = (y_0 - \text{ref})^2 + (y_1 - \text{ref})^2$.

Nous cherchons à déterminer l'expression de $\nabla Y_n \text{Err}$:

$$\nabla Y_n \text{Err} = [\partial F_{ntErr} / \partial y_0 \quad \partial F_{ntErr} / \partial y_1] = [2(y_0 - \text{ref}) \quad 2(y_1 - \text{ref})]$$

▪ Cascadage

Supposons que le gradient $\nabla Y_k \text{Err}$ ait été déterminé à l'itération précédente. A cette nouvelle itération, nous traitons la couche k et nous allons déterminer :

- Le gradient $\nabla W_k \text{Err}$ utilisé dans l'algorithme de descente du gradient.
- Le gradient $\nabla X_k \text{Err}$ utilisé pour propager le calcul du gradient vers la couche inférieure.

Nous rappelons que $Y_{k-1} = X_k$. Ainsi, lorsque nous connaissons le gradient $\nabla X_k \text{Err}$, nous connaissons aussi $\nabla Y_{k-1} \text{Err}$ et nous avons assez d'informations pour passer à l'itération suivante et traiter la couche $k-1$.

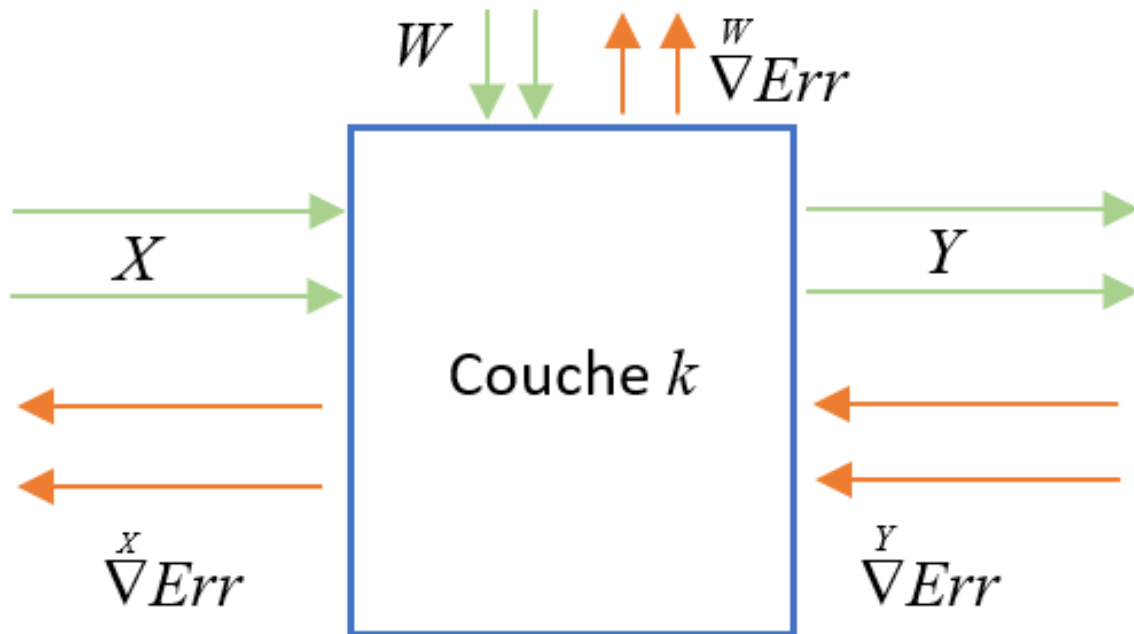
De cette façon, les itérations successives cascaden les informations des différents gradients $\nabla X_k \text{Err}$.

▪ Traiter la couche courante

Nous traitons maintenant la k -ième couche.

Pour alléger les notations, les vecteurs X_k , Y_k et W_k sont notées : X , Y et W .

Regardons comment traiter la couche courante :



Le **théorème général de dérivation** énonce :

$$\nabla X \text{Err} = t J_{Y/X} \cdot \nabla Y \text{Err}$$

où $J_{Y/X}$ désigne la matrice jacobienne de la fonction $Y(X, W)$ par rapport aux variables $x_0, x_1, \dots$ de X .

$$J_{Y/X} = [\partial y_0 / \partial x_0 \quad \partial y_0 / \partial x_1 \quad \partial y_0 / \partial x_2 \quad \dots \quad \partial y_1 / \partial x_0 \quad \partial y_1 / \partial x_1 \quad \partial y_1 / \partial x_2 \quad \dots \quad \partial y_2 / \partial x_0 \quad \partial y_2 / \partial x_1 \quad \partial y_2 / \partial x_2 \quad \dots]$$

De la même manière, ce théorème nous indique :

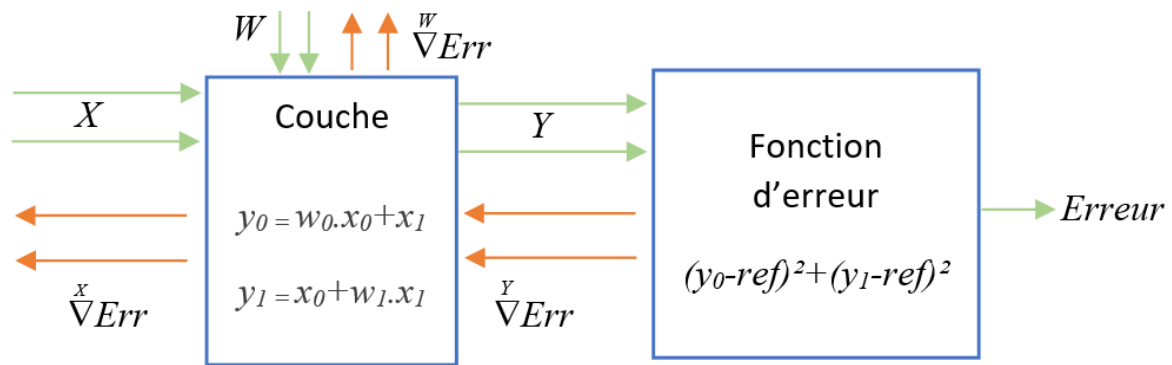
$$\nabla W \text{Err} = t J_{Y/W} \cdot \nabla Y \text{Err}$$

où $J_{Y/W}$ désigne la matrice jacobienne de la fonction $Y(X, W)$ par rapport aux variables $w_0, w_1, \dots$ de W .

$$J_{Y/W} = [\partial y_0 / \partial w_0 \quad \partial y_0 / \partial w_1 \quad \partial y_0 / \partial w_2 \quad \dots \quad \partial y_1 / \partial w_0 \quad \partial y_1 / \partial w_1 \quad \partial y_1 / \partial w_2 \quad \dots \quad \partial y_2 / \partial w_0 \quad \partial y_2 / \partial w_1 \quad \partial y_2 / \partial w_2 \quad \dots]$$

▪ Exemple

Nous utilisons un réseau de taille réduite composé d'une seule couche suivie d'une fonction d'erreur :



Nous calculons l'expression des deux matrices jacobiniennes JY/X et JY/W :

$$JY/X = [w_0 \ 1 \ w_1] \quad \text{et} \quad JY/W = [x_0 \ 0 \ x_1]$$

et nous rappelons que :

$$\nabla Y Err = [\partial Err / \partial y_0 \ \partial Err / \partial y_1] = [2(y_0 - ref) \ 2(y_1 - ref)]$$

Dans cette hypothèse la manière la plus directe de résoudre le développement de cette fonction dans une matrice jacobienne sera de décomposer les termes de la manière suivante :

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, -\infty < x < \infty$$

$$\cos \alpha + \cos \beta = 2 \cos \frac{1}{2}(\alpha + \beta) \cos \frac{1}{2}(\alpha - \beta)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Mod 4. L'algorithme d'apprentissage

Après avoir présenté la méthode de descente du gradient et le principe de la rétropropagation, il est maintenant possible de concevoir dans sa totalité l'algorithme d'apprentissage.

▪ L'instanciation du réseau

La première étape consiste à instancier le réseau et ses différentes couches en utilisant les différentes classes fournies par la librairie. Pour chaque type de couche, la librairie dispose des matrices jacobiniennes associées.

La création du réseau se traduit par la mise en place d'un graph de calcul. Ce graph symbolise le flux de traitements à effectuer d'une couche à l'autre. Il sert aussi à conserver certaines données comme les valeurs calculées en sortie de chaque couche.

▪ La boucle sur les échantillons

Nous avons vu précédemment que l'erreur totale du réseau par rapport à l'ensemble des échantillons présents dans la base d'apprentissage est donnée par la formule suivante :

$$\text{ErrTot}(\mathbf{W}) = \sum_{(\mathbf{I}, \text{refl}) \in \text{DataSet}} \text{Err}(\mathbf{W}, \mathbf{I}, \text{refl})$$

Comme l'opérateur ∇ est linéaire, nous obtenons :

$$\nabla \text{ErrTot}(\mathbf{W}) = \sum_{(\mathbf{I}, \text{refl}) \in \text{DataSet}} \nabla \text{Err}(\mathbf{W}, \mathbf{I}, \text{refl})$$

Ainsi, le gradient de l'erreur totale s'obtient en cumulant les valeurs des gradients associés à chaque échantillon. Nous obtenons donc l'algorithme de principe suivant :

```
Instanciation du réseau
Tant qu'il reste du temps
  Mise à 0 dans le graph de calcul de  $\nabla \text{ErrTot}(\mathbf{W})$ 

  # estimation de  $\nabla \text{ErrTot}(\mathbf{W})$ 
  Pour chaque échantillon dans la base
    Calculez  $\nabla \text{Err}(\mathbf{W})$  associé à cet échantillon
```

$\nabla \text{ErrTot}(W) += \nabla \text{Err}(W)$ #formule ci-dessus

Minimisation de l'erreur / descente du gradient

$W = W - \text{pas} * \nabla \text{ErrTot}(W)$

Le calcul de $\nabla \text{Err}(W)$ s'effectue en deux passes :

- Dans la première passe appelée **Forward**, on calcule itérativement les valeurs en sortie de chaque couche jusqu'à obtenir la valeur de l'erreur en sortie du réseau. Dans le graph de calcul sont conservées les valeurs obtenues en sortie de chaque couche.
- Dans la deuxième passe appelée **Backward**, la librairie va parcourir le graph de calcul en sens inverse. Pour chaque couche, on calcule ainsi les valeurs des matrices jacobiniennes et on détermine successivement le gradient de l'erreur en fonction des paramètres internes de chaque couche.

■ Algorithme de principe

Nous sommes maintenant capables de construire l'algorithme complet d'apprentissage.

Nous notons:

- X_k le vecteur des entrées de la couche k
- W_k le vecteur des paramètres internes de la couche k .

Avertissement

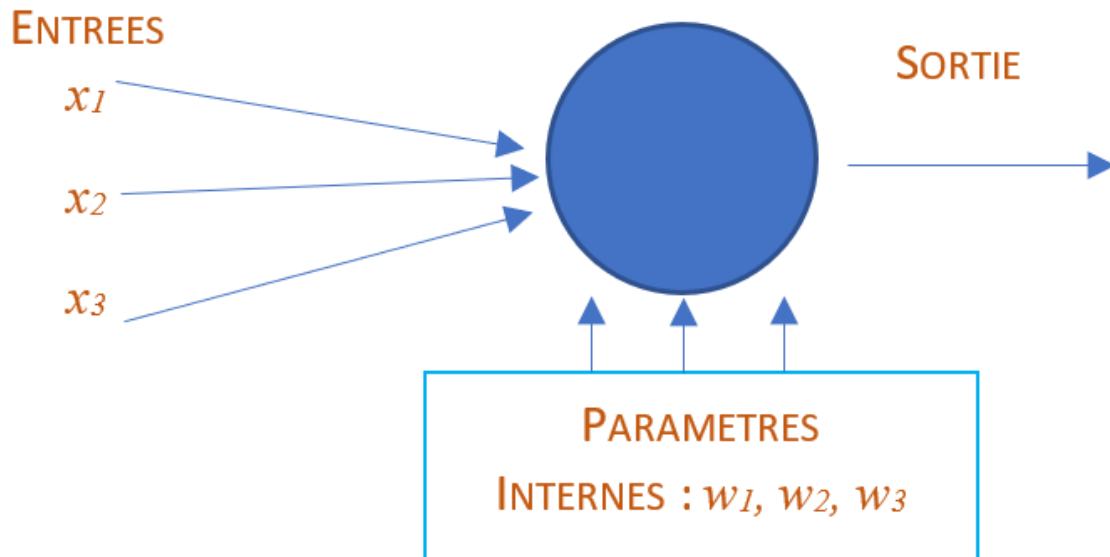
Nous utilisons la notation X pour désigner un vecteur de variables libres alors que la notation $X\text{---}$ va désigner un vecteur de valeurs. Cette notation permet de lever certaines ambiguïtés.

Calcul d'une image pour 1 échantillon d'entrée :

*éée# Calcul de ∇Err Fonction Grad($X0\text{---}$, $\text{Ref}\text{---}$)
Passe Forward Pour i de $0, \dots, n-1$: $X_{i+1}\text{---} = \text{Couche}_i(X_i\text{---})$
n couches dans le réseau $\text{Err} = \text{FntErr}(X_n\text{---}, \text{Ref}\text{---})$
Passe Backward # D : variable utilisée pour la
backpropagation $D\text{---} = \nabla X_n \text{FntErr}(X_n\text{---}, \text{Ref}\text{---})$
calcule la valeur du gradient de la fonction d'erreur Pour i de $n-1, \dots, 0$:
calcul des valeurs des jacobiens $JX\text{---} = JY_i/X_i(X_i\text{---}, W_i\text{---})$
 $JW\text{---} = JY_i/W_i(X_i\text{---}, W_i\text{---})$ $\nabla W_i \text{Err} = tJW\text{---}.D\text{---} - D\text{---} = tJX\text{---}.D\text{---}$
rétropropagation : D contient les valeurs de $\nabla X_k \text{Err}$ Retourner $\text{Err}, (\nabla W_k \text{Err})_{0 \leq k < n}$*

Mod 5. Réseaux de neurones – Introduction

▪ Le neurone informatique



Un neurone dispose :

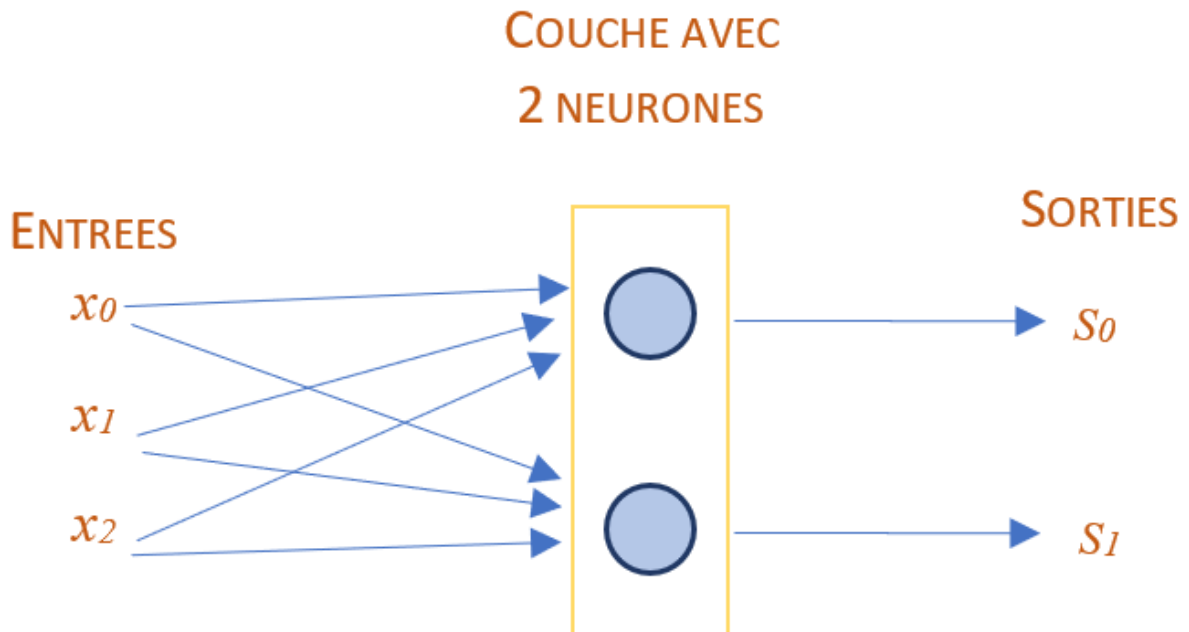
- d'une UNIQUE sortie notée s .
- de n entrées notées x_i .
- de n paramètres internes notés w_i .
- éventuellement d'un paramètre interne supplémentaire appelé biais et noté b .

Un neurone permet d'effectuer le calcul suivant :

$$s = \sum_{i=0}^{n-1} x_i \cdot w_i + b$$

On peut utiliser plusieurs neurones qui partagent les mêmes entrées. Dans ce cas, ils font partie de la même couche. Cette couche produit autant de valeurs de sortie que de neurones présents dans la couche.

Voici le schéma correspondant :



Ce type de couche est appelée **Fully Connected** car chaque neurone de la couche en question est connecté à toutes les entrées. On trouve aussi le terme de **couche Linear** ou encore de **couche FC**.

Voici donc ce que nous obtenons lorsque nous avons n neurones et k valeurs d'entrée pour une couche donnée, cette couche peut être décrite ainsi :

- La sortie de cette couche contient n valeurs
- Cette couche contient n valeurs de biais b : une par neurone
- Cette couche contient $n.k$ poids : n neurones avec k poids chacun.

▪ Topologie des réseaux

Une couche Linear peut être représentée graphiquement à partir d'un rectangle vertical, contenant des ronds correspondant aux neurones.

Certains auteurs dessinent une **couche d'entrée**, « **input layer** » en anglais, même si cette couche ne contient aucun neurone. Cette couche « virtuelle » a pour fonction de « charger » les données, elle a donc un rôle « actif » ce qui explique sa présence dans certains schémas, même si, à proprement parler, elle n'effectue aucun calcul.

Les entrées sont généralement situées sur la gauche et les couches successives sont empilées de gauche à droite, la sortie finale du réseau se situe ainsi à l'extrémité droite :

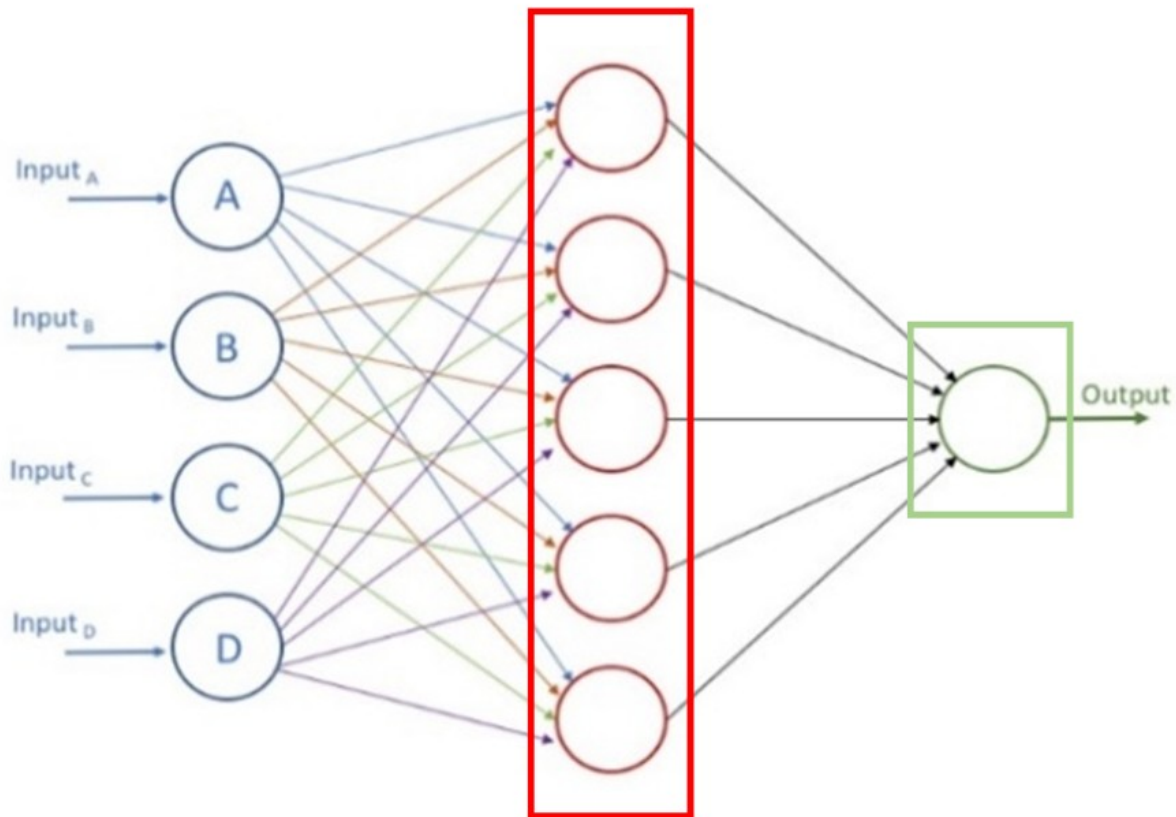


Schéma d'un « petit » réseau : une entrée avec quatre valeurs suivies d'une couche Linear au centre comportant 5 neurones et une deuxième couche Linear à droite ayant un seul neurone. Ce réseau donne une unique valeur de sortie.

▪ La fonction d'activation

Suffit-il d'empiler des couches pour construire un réseau ? Pas si sûr. En effet, prenons le cas de trois couches Linear sans biais. En l'absence de biais, chaque couche effectue l'équivalent d'une multiplication matricielle. Notons A_1 , A_2 et A_3 les matrices associées à chaque couche. Ainsi, la valeur de sortie du réseau peut être calculée de la manière suivante :

$$\text{Sortie} = A_3 \cdot A_2 \cdot A_1 \cdot \text{Input}$$

En prenant $M = A_3 \cdot A_2 \cdot A_1$, l'équation ci-dessus se réécrit comme :

$$\text{Sortie} = M \cdot \text{Input}$$

Ce qui permet d'affirmer que tout réseau à plusieurs couches est équivalent à un réseau à une seule couche. Ainsi, en empilant plusieurs couches Linear, on ne crée pas un réseau plus avancé qu'un réseau à une couche. Pour créer quelque-chose de nouveau, il faut donc ajouter entre chaque couche une « non-linéarité » et c'est ici le rôle des fonctions d'activation. Une **fonction d'activation** est une fonction de $\mathbb{R} \rightarrow \mathbb{R}$ non-linéaire qui se connecte à la sortie de chaque neurone.

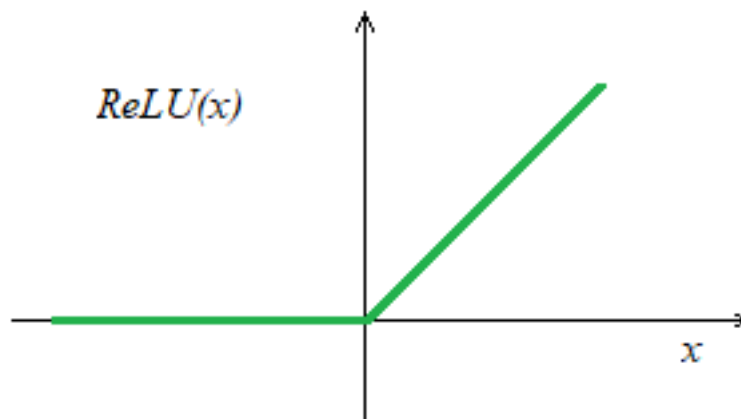
Généralement :

- Dans une même couche, tous les neurones utilisent la même fonction d'activation.
- Les fonctions d'activation n'ont pas de paramètres internes.

Une fonction assez populaire aujourd'hui est la fonction ReLU (Rectified Linear Unit) définie ainsi :

$\text{ReLU}(x) = \max(0, x)$

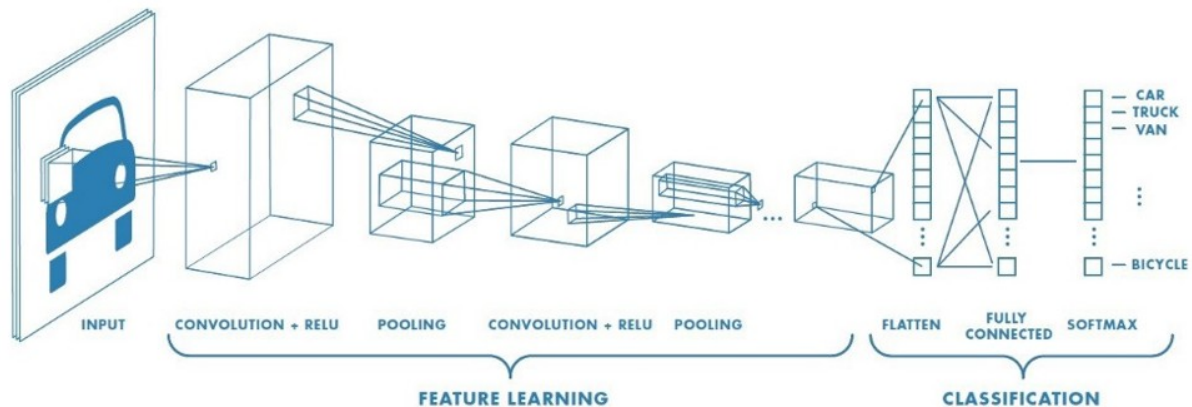
Dont le graph est :



Il s'agit d'une fonction continue, dérivable quasiment partout et peu coûteuse en calcul ! Dans les schémas des réseaux de neurones, vous trouverez ainsi des commentaires sous chaque couche du type LINEAR+ReLU pour indiquer le type de couche utilisée ainsi que sa fonction d'activation. Avant que la fonction ReLU soit popularisée, on trouvait d'autres fonctions d'activation comme tanh dont les valeurs de sortie sont entre -1 et 1 ou encore la fonction sigmoïde $\sigma(x)$ dont les valeurs de sortie sont entre 0 et 1.

■ Réseau complexe

Il existe d'autres couches que les couches Linear qui effectue des traitements plus complexes ou plus simples. Mais le principe reste le même : le réseau se construit par empilage de plusieurs couches. Voici un exemple de réseau utilisant plusieurs couches et traitant des images en entrée:



Dans ce schéma, on remarque que les couches ne sont pas dessinées. Comme il existe une association 1-1 entre une couche et son tableau des données en sortie, on préfère représenter uniquement le tableau sous la forme d'un bloc pour indiquer ses dimensions.

En dessous de chaque bloc est donné le type de la couche correspondante. Ainsi dans ce schéma, on peut voir que l'on utilise successivement une couche Convolution puis une couche Pooling suivie d'une couche Convolution et d'une couche Flatten et enfin d'une couche Fully Connected.

Chaque fois qu'une fonction d'activation est utilisée en sortie d'une couche, son type est précisé à côté du nom de la couche. Ainsi dans ce réseau, les deux couches Convolution utilisent la fonction d'activation Relu. La dernière couche de type Fully Connected utilise la fonction SoftMax.

En sortie du réseau on obtient plusieurs valeurs, chaque valeur étant associée à un type de véhicules (Car, Truck, Van, ..., Bicycle). Il semble donc que ce réseau calcule pour une image en entrée une sorte de score / probabilité pour chaque type de véhicules.

Mod 5. Les réseaux convolutionnels

Nous présentons dans ce chapitre les **réseaux convolutionnels (Convolutional Neural Network - CNN)**. Pour parler de réseau convolutionnel, il faut qu'au moins une couche de type Conv soit présente dans le réseau. Un réseau à base de couches Linear ne permet pas d'obtenir un réseau convolutionnel ! Historiquement, l'apparition de cette technique a permis aux réseaux de neurones d'atteindre un tout autre niveau de performance.

La couche Conv2D

▪ Principe

Pour calculer le résultat d'une couche conv2D utilisant un **noyau** de taille (u,v) et une image représentée par un tableau de taille (n,m) , on parcourt chaque pixel de l'image et lorsqu'on peut extraire une sous-image de taille (u,v) depuis le pixel courant, on effectue la convolution entre cette portion d'image et le noyau, tous deux de même taille. Pour cela, on effectue une multiplication membre à membre de chaque valeur et on cumule l'ensemble de ses résultats. Le résultat obtenu en sortie de la couche Conv2D est un tableau de taille $(n-u+1, m-v+1)$.

Voici le code de principe correspondant :

```
def Conv2D(Tnoyau, TImage) :
    sizeNoyau = Tnoyau.shape
    sizeImage = TImage.shape

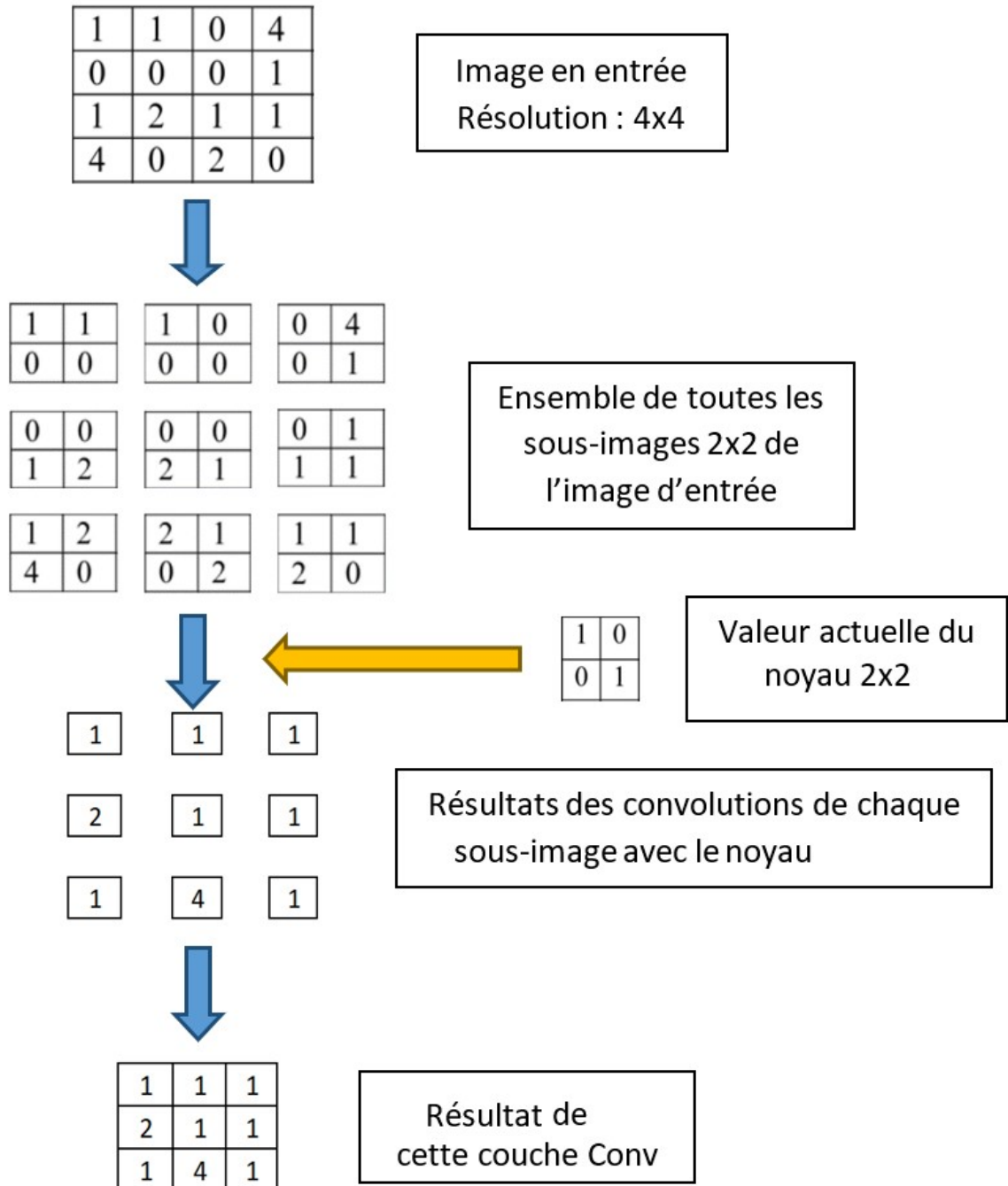
    # tableau résultat :
    size = (sizeImage[0]-sizeNoyau[0]+1, sizeImage[1]-sizeNoyau[1]+1)
    R = NouveauTensor(size)

    # Calcul
    for x in range(0, size[0]):
        for y in range(0, size[1]):
            exImage = ExtractImage(TImage,x,y,sizeNoyau)
            value = Conv(exImage,TImage) # multiplication membre à membre et sommation
            R[x,y] = value
```


Avec :

$$\text{Conv}(N,S)=\sum_{i,j} N_{i,j} * S_{i,j}$$

Voici un exemple utilisant une image 4x4 et un noyau 2x2 :



Dans une image, les zones de transition sont souvent « douces ». Il n'est pas toujours nécessaire de calculer la convolution en chaque point de l'image, mais seulement tous les k pixels. Ce saut est appelé **pas** ou **stride**. Il permet de réduire la taille du tableau de sortie d'un facteur k^2 .

La taille du tableau de sortie diminue légèrement en fonction de la taille du noyau. Pour éviter cela, on peut élargir virtuellement l'image d'entrée avec des 0. Ce paramètre de marge ou **zero padding** permet ainsi d'ajuster la taille du tableau de sortie.

▪ Les noyaux de taille complexe

Nous allons d'abord considérer une base d'images. Une image se caractérise par :

- le nombre de lignes (Rows) aussi appelé sa hauteur (Height)
- le nombre de colonnes (Columns) aussi appelé sa largeur (Width)
- le nombre de canaux (Channels) aussi appelé sa profondeur (Depth)

Pour une image RGB, les trois canaux RGB sont présents et pour une image en niveaux de gris, un seul canal existe. Il peut exister d'autres configurations de canaux, notamment dans le cas d'images multimodales.

Afin d'uniformiser les tailles des images, les images en niveaux de gris seront décrites comme des images en 3 dimensions, la dimension associée à la profondeur étant toujours égale à 1. Il existe deux conventions pour représenter les images dans un tableau :

- Channels Last : la dernière dimension du tableau correspond au canal des couleurs : (Height,Width,Channels).
- Channels First : la première dimension du tableau correspond au canal des couleurs : (Channels,Height,Width).

Dans tous les cas, l'ordre Height/Width est identique, seul change la position de l'indice des canaux. La fonction Conv2D() des bibliothèques d'IA disposent d'un paramètre permettant de régler cette option. Dans la suite, **nos exemples seront donnés en Channel First** :

(Channels,Height,Width).

Les couches Conv2D ont la capacité de traiter une série d'images. On ne transmet pas les images une à une, mais en lot, afin d'occuper toutes les ressources CPU/GPU disponibles. Ainsi, l'entrée d'une couche Conv2D correspond à un tableau 4 dimensions :

Taille des tableaux en entrée des couches Conv2D : **(N,C,H,W)**, avec :

- N : Nombre d'images.
- C : Nombre de canaux.
- H : Hauteur / Height.
- W : Width / Largeur.

Commençons par le cas des images en niveaux de gris. Lorsque l'on effectue la convolution 2D par un noyau de taille (u,v) , on obtient en sortie un tableau de taille $(H',W')=(H-u+1,W-v+1)$. Une couche Conv2D peut traiter plusieurs convolutions en parallèle ceci en utilisant plusieurs noyaux, de 4 à 32 généralement. De cette façon la sortie associée à 1 image de taille (H,W) aura une taille (K,H',W') où K désigne le nombre de noyaux (kernels) présents dans la couche Conv2D. Pour terminer, si nous traitons un lot de N images en entrée, nous obtenons finalement en sortie un tableau de taille (N,K,H',L') . En conclusion :

Pour une entrée de taille (N,C,H,W) et K noyaux de taille (u,v) :

- N : nombre d'images
- C : nombres de canaux
- (H,W) : Height/Width des images

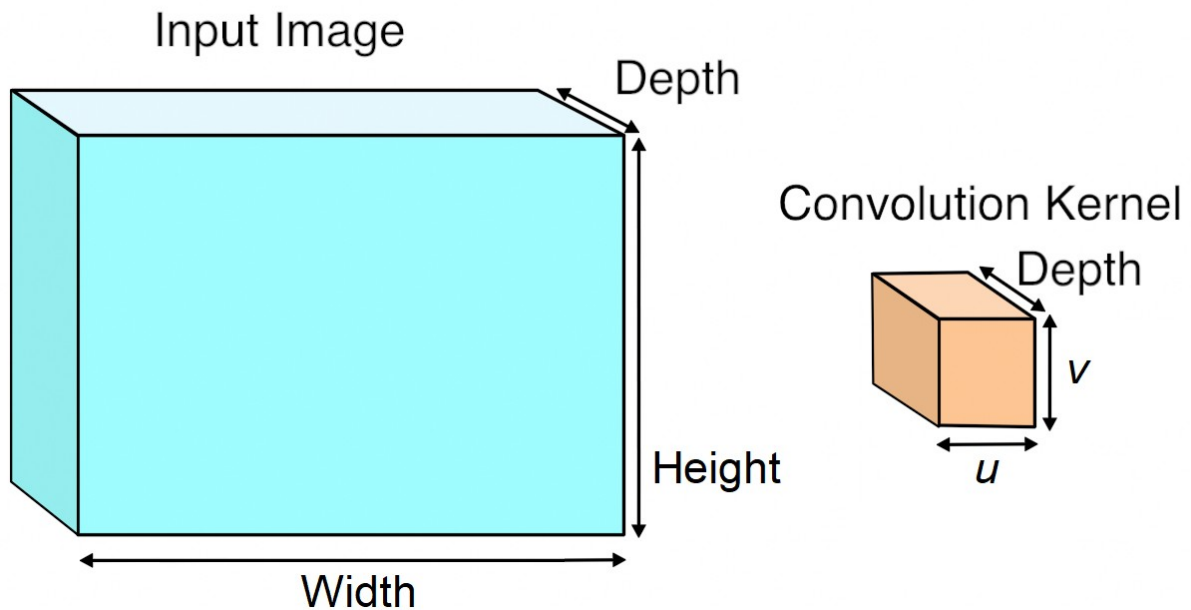
La taille de la sortie de la couche Conv2D est : (N,K,H',W') avec $H'=H-u+1$ et $W'=W-v+1$.

Examinons maintenant le cas des images RVB. Elles sont représentées par un tableau de taille $(C,H,L)=(3,H,L)$. Dans ce cas précis, le noyau N de la convolution aura une taille $(3,u,v)$. Ainsi, le noyau s'étend à une troisième dimension. La formule de la convolution 2D devient :

$$\text{Conv}(N,S)=\sum_{i,j,k} N_{i,j,k} * S_{i,j,k}$$

Note

A ce niveau il existe une petite subtilité. La taille réelle des noyaux n'est pas (u,v) mais plus exactement (C,u,v) où C correspond aux nombres de canaux. En effet, cette extension est nécessaire pour que la formule de la convolution 2D fonctionne comme énoncé.



▪ Terminologie

Une couche Conv gère plusieurs noyaux encore appelés **caractéristiques** (**feature** en anglais, ou encore **kernel**). Le nombre de features s'appelle la **profondeur de la couche Conv**.

Le pas, la marge et la profondeur de la couche CONV sont des **hyperparamètres d'apprentissage** du réseau : ils sont choisis par l'utilisateur en début d'apprentissage et n'évoluent pas durant l'apprentissage.

La couche Conv participe aux calculs Forward du réseau et son résultat contribue à l'erreur finale. Ainsi les valeurs contenues dans ses features correspondent à des poids qui vont ainsi être optimisés par la méthode du gradient.

▪ Interprétation des features

Mais à quoi peuvent correspondre les features une fois appris ? Il s'agit ici d'une question difficile. Cependant, on a réussi à déterminer que pour la couche traitant l'image d'entrée, ces features correspondent à des détecteurs de contours. Voici un exemple de features appris pour le problème de la classification des chiffres manuscrits :



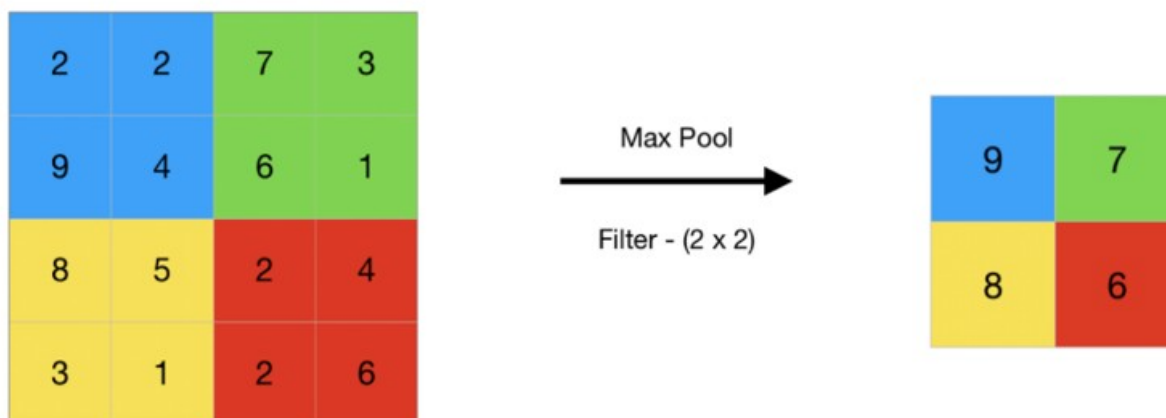
Ainsi les features cherchent à détecter des « caractéristiques » permettant de décrire l'information présente dans l'image. Si une couche Conv a seulement 5 features, elle va devoir en choisir 5 qui représentent au mieux l'information recherchée dans l'image.

La couche POOL

▪ Objectif

L'objectif d'une couche Pool est de réduire la quantité d'informations qui transitent d'une couche à l'autre. Par exemple, pour un tableau 2D, on peut choisir de le diviser en blocs de taille 2x2 et d'effectuer pour chaque bloc une opération du type : moyennage, min ou max... Le tableau résultat aura donc une taille 4 fois moindre.

Nous présentons un exemple de MaxPool-2D avec une réduction 2x2 :



■ La couche DropOut

Cette couche permet de mettre à zéro aléatoirement certaines valeurs d'un tableau ceci avec une probabilité donnée. Il n'y a pas réduction de la taille des données, mais simplement une simulation d'une perte d'information. L'intérêt est d'améliorer l'indépendance entre les différentes features. En effet, supposons que le réseau doive trouver deux features A et B . Sans la technique de DropOut, il pourrait aussi bien sélectionner les features A et $A+B$, car les traitements sur les couches suivantes pourraient reconstruire les features A et B idéaux. En désactivant de temps à autre le feature A ou B , cela force le réseau à converger sans tenir compte des valeurs des features voisins.

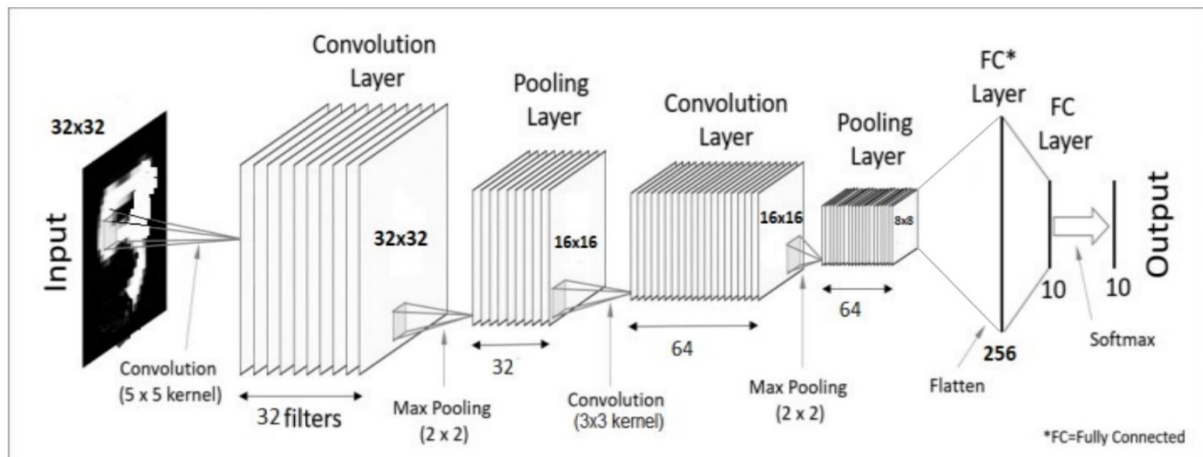
Les couches DropOut ne sont utiles que durant la phase d'apprentissage et non durant la phase de validation. Ainsi, elles seront activées durant la phase d'apprentissage et ignorées pendant la phase de validation.

On donne pour cette couche une probabilité p d'effacer une donnée. Cela sous-entend que la somme des valeurs en sortie est abaissée (en moyenne) d'un facteur $1-p$ par rapport à l'entrée. Pour compenser cette perte, un facteur $1/(1-p)$ est appliquée aux valeurs conservées pour préserver le niveau moyen des valeurs d'entrée. Voici un exemple de l'application d'une couche Dropout avec une probabilité de 0.5 sur un tableau rempli de valeurs 1. Vous remarquez que sur les 10 valeurs présentes 6 ont été mises aléatoirement à zéro. Les valeurs de sortie sont multipliées par $2 = 1/p$ pour compenser le niveau d'entrée. En relançant ce test, on obtient des résultats différents.

- Input : tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
- Output : tensor([0., 0., 0., 0., 2., 2., 0., 0., 2., 2.])

■ Lecture d'illustration

Étudions l'exemple suivant :

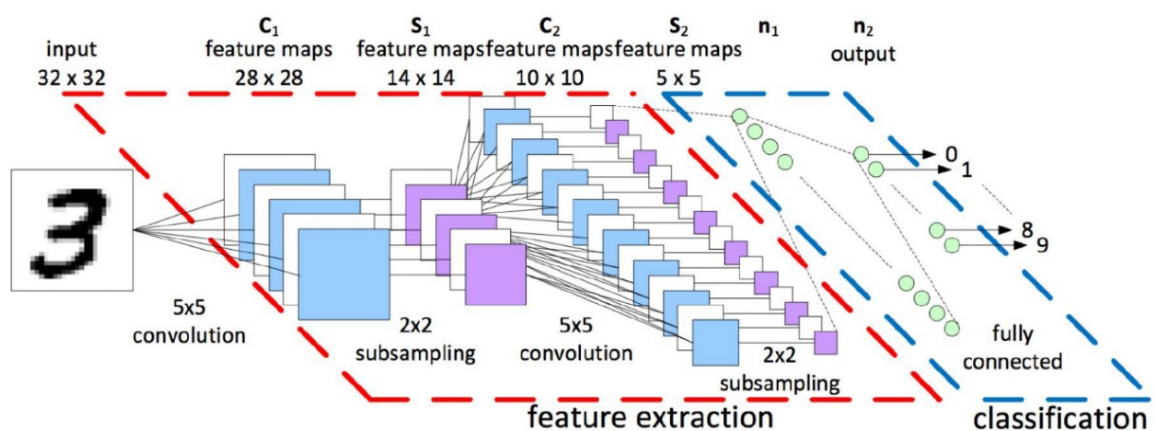


Nous analysons ce réseau de gauche à droite. Sur ce schéma, les tableaux en entrée/sortie des différentes couches sont représentés par des empilements de rectangle. Les opérations effectuées par chaque couche sont notées en bas des illustrations.

- En entrée, on trouve une image 2D de résolution 32x32 en niveaux de gris.
- La sortie de la première couche est le résultat d'une convolution 2D donnant un tableau de taille (32,32,32). Il y a donc 32 features dans cette couche Conv. Les features étant de taille 5x5, la résolution de sortie aurait dû être de 28x28 et non 32x32, un paramètre de zero padding a dû être utilisé pour compenser la perte de résolution.
- La couche de Max-Pooling réduit la résolution par 2, ce qui donne un tableau (32,16,16). Il y a toujours 32 canaux car l'opération de MaxPooling2D ne s'applique qu'aux deux premières dimensions.
- Une nouvelle couche Conv de 64 features est appliquée. Le tableau d'entrée ayant une taille de (32,16,16), celui de sortie une taille de (64,16,16) et celui du noyau 3x3, le noyau de la couche Conv2D a été construit avec un tableau (64,32,3,3). Cette couche traite donc le tableau d'entrée comme une image 16x16 de 32 canaux.
- La couche de Max-Pooling suivante réduit la résolution par 2, ce qui donne un tableau (64,8,8) en sortie. Le nombre de canaux reste inchangé.
- Le tableau [64,8,8] subit un « flatten » pour être aplati sous la forme d'un tableau 1D de taille $64 \times 8 \times 8 = 4096$.

- On trouve ensuite une couche Linear (aussi appelée Fully Connected). La grandeur 256 indique sûrement le nombre de neurones présents dans cette couche. La fonction d'activation n'est pas précisée, on peut supposer qu'il s'agit d'une fonction ReLU. Chaque neurone de cette couche est connecté aux 4096 entrées de la couche précédente
- La dernière couche correspond à une deuxième couche Linear. Elle contient 10 neurones qui calculent 10 scores correspondant aux 10 catégories étudiées.
- Pour finir, les valeurs obtenues étant quelconques, négatives comme positives, on applique la fonction Softmax qui transforment ces 10 valeurs en 10 probabilités de somme 100%.

▪ Les deux parties d'un réseau



La partie gauche du réseau, proche de l'entrée, cherche à évaluer des features permettant à la partie droite du réseau de faciliter sa décision. Ainsi, les réseaux utilisés en classification d'images ont tendance à se séparer en deux parties :

- La partie gauche, proche des données d'entrée, a pour objectif d'estimer et de trouver un ensemble de critères « feature extraction » ceci en empilant plusieurs couches Conv.
- La partie droite doit construire sa réponse et apprendre à classifier l'image à partir des critères évalués par la partie gauche du réseau. Pour mettre en place cette partie, on utilise plusieurs couches Linear empilées.

On peut ainsi transférer des réseaux déjà entraînés vers d'autres problèmes. Par exemple pour un réseau entraîné sur la classification d'images, on peut reprendre ce réseau tel quel, garder sa partie gauche car on suppose qu'elle contient des détections de features dans une image et remettre à zéro la partie droite afin d'apprendre sur un nouveau set d'images spécialisées : bâtiments, bactéries, véhicules. Cela permet d'économiser un temps précieux d'apprentissage.

Mod 6. Les optimiseurs

Nous avons présenté la méthode de descente du gradient (gradient descend) pour optimiser les paramètres internes du réseau afin de réduire l'erreur totale. Mais ce n'est pas la seule méthode, des améliorations et des variations de ces améliorations ont été mises en place pour accélérer et améliorer la convergence. Ainsi, ces différentes méthodes sont regroupées sous le terme d'**optimiseurs (optimizers)**. Leurs noms sont donnés en anglais car elles apparaissent ainsi dans les librairies et dans le code source des programmes.

Les classiques

▪ Gradient Descent

La descente du gradient utilise un **pas fixe** aussi appelé **Learning rate**. Pour rappel, les paramètres du réseau évoluent en fonction du gradient de la fonction d'erreur :

$$W \leftarrow W - \text{pas} * \nabla \text{Err}$$

Avantages :

- Facile à comprendre.
- Facile à programmer.

Inconvénient :

- Nécessite d'évaluer l'erreur et son gradient sur l'ensemble des données en entrée. Lorsque le Dataset est important, cela ralentit considérablement chaque itération.

En pratique, la méthode de descente du gradient est peu utilisée telle quel car elle requiert trop de temps. La variante suivante est la version plus commune.

▪ Stochastic Gradient Descent (SGD)

Que signifie le terme stochastique ajouté au nom de la méthode du gradient ? Lorsque la taille des données en entrée devient importante, évaluer l'erreur totale et son gradient devient une opération très coûteuse. Nous rappellerons de plus qu'au départ, les paramètres internes sont choisis aléatoirement et qu'il n'est donc pas utile d'avoir un gradient calculé de manière précise au vue de la piètre performance du réseau. Ainsi, pour améliorer la descente du gradient, on sélectionne aléatoirement un sous-ensemble des données en entrée pour estimer l'erreur et le gradient. Cette sélection étant faite aléatoirement à chaque itération, on qualifie le gradient de **stochastique** : fruit du hasard.

Avantages :

- Les mises à jour sont plus fréquentes, c'est toujours bon pour le moral de voir le réseau évoluer régulièrement !

- Dans la pratique, diminue le temps de convergence.

Inconvénients :

- Du bruit apparaît dans le gradient et par conséquent dans la courbe de l'erreur aussi.
- Lorsque l'on approche d'un optimum, la convergence est plus hasardeuse comparée à la méthode classique. On peut corriger cela en réduisant le Learning rate.

▪ **Mini-Batch Gradient Descent**

L'idée est la même que pour SGD, mais ici nous allons partitionner le Dataset des données en entrée en batches de taille identique. Ainsi plutôt que de tirer au hasard les données à utiliser pour le calcul du gradient, on va itérativement traiter chaque minibatch. Ainsi, lorsque tous les batches ont été traités, on dit que l'algorithme a terminé une *epoch*. L'apprentissage continue en repartant du premier batch dans la liste.

Lors de la création des batches, il faut qu'ils soient générés aléatoirement ou équitablement. Ainsi pour un problème de classification, on ne peut pas faire un batch de chats puis un batch de chiens et ainsi de suite. Cela conduirait à une évaluation du gradient très biaisée. Il est donc recommandé d'avoir des batches représentatifs avec une représentation équivalente de chaque classe. Ainsi, comparé à SGD, l'évaluation du gradient est moins bruité.

▪ **Momentum**

On ajoute une technique à la méthode SGD pour réduire son bruit élevé. On accélère ainsi la convergence vers la direction pertinente et on réduit la fluctuation provenant des directions non pertinentes. Cette technique consiste à pondérer le gradient courant par la pondération des gradients précédents afin de lisser des changements de direction trop important. Pour cela, on utilise un nouvel hyperparamètre appelé le **Momentum** symbolisé par gamma. Voici la formule utilisée :

$$V(t) = \gamma V(t-1) + (1-\gamma) \nabla \text{Err}$$

Cette valeur $V(t)$ est alors injectée dans la méthode de descente du gradient :

$$W \leftarrow W - \text{pas} * V(t)$$

Par défaut, dans les librairies le Momentum est fixé à 0.9.

Avantage :

- Réduit le bruit et les oscillations présents dans SGD.

Désavantage :

- Rajoute un hyperparamètre supplémentaire à gérer.

▪ Adagrad

Dans la méthode de descente du gradient, vous remarquez que l'on utilise un pas constant ceci pour tous les paramètres internes. Cependant, certains paramètres internes peuvent être plus activement impliqués que d'autres, il faut donc les faire évoluer plus rapidement. Ainsi, l'optimiseur Adagrad introduit l'idée de mise à jour des paramètres internes avec un pas propre à chacun :

Gradient descent: $\mathbf{w}_{i,t+1} = \mathbf{w}_{i,t} - \text{pas} * \partial \text{Err} \partial \mathbf{w}_{i,t}$

devient

Adagrad: $\mathbf{w}_{i,t+1} = \mathbf{w}_{i,t} - \frac{\text{pas}}{\sqrt{SSG_{i,t}}} * \partial \text{Err} \partial \mathbf{w}_{i,t}$

L'indice i dans l'équation désigne le i -ème paramètre interne et l'indice t représente le numéro de l'itération. La grandeur SSG est la somme des gradients au carré pour le i -ème paramètre à partir de l'itération 0 jusqu'à l'itération t . La grandeur ϵ est utilisée pour ajouter une petite valeur à SSG pour éviter une division par zéro. En divisant le *pas* par la racine carrée de SSG , on s'assure que le taux d'apprentissage pour les paramètres changeant fréquemment diminue plus rapidement que le taux d'apprentissage pour les paramètres rarement mis à jour.

Avantages :

- Le pas change et s'adapte pour chaque paramètre.
- Cela supprime le besoin de régler/bidouiller le Learning rate durant la convergence.

Désavantage :

- Le Learning rate de chaque paramètre ne fait que diminuer au fil du temps, ce qui ralentit la convergence.

▪ Les optimiseurs avancés

Il existe d'autres optimiseurs plus avancés ayant pour objectif d'apporter une amélioration par rapport à un autre optimiseur. L'inconvénient est souvent l'apparition de multiples paramètres de réglage plus ou moins faciles à interpréter et à exploiter. La différence entre deux optimiseurs est parfois subtile, chacun étant une variation ou un mélange d'autres optimiseurs.

Dans l'approche Adagrad, on peut facilement remarquer que le dénominateur sous le terme *pas* a une valeur qui ne cesse d'augmenter au fil des itérations, car le terme $SS_{Git} + \epsilon$ cumulent des valeurs positives. Cela peut ainsi faire tendre le pas pondéré vers une valeur très faible.

- **RMSProp**

Pour diminuer l'effet de ralenti se produisant au fil des itérations, la méthode RMSProp propose de pondérer le facteur d'amorti de l'itération précédente avec le carré du gradient courant pour construire le facteur d'amorti.

$SS_{Git} = \beta \cdot SS_{Git-1} + (1-\beta) \cdot (\nabla Err_t)^2$
--

- **Adadelta**

Pour diminuer l'effet de ralenti se produisant au fil des itérations, Adadelta ne tient compte que des k derniers gradients pour calculer son facteur d'amortissement par paramètre.

- **Adam**

Adam (Adaptive Moment Estimation) a repris les améliorations apportées par Adadelta et RMSProp en apportant une gestion des momentums au premier et au second ordre. Cet optimiseur est reconnu pour fournir de très bonnes performances et une vitesse de convergence accrue.

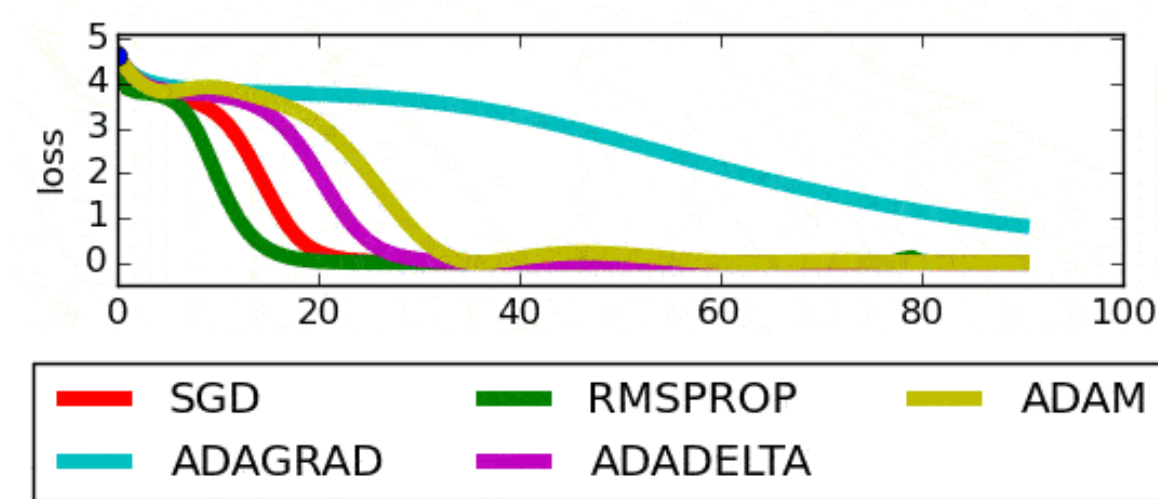
- **Quel optimiseur choisir ?**

Les optimiseurs, c'est un peu comme les chevaux de course, il faut parier sur le bon ! Si vous êtes en mode recherche & développement, l'objectif est de trouver au moins un optimiseur qui converge. En cas de réussite, c'est une victoire.

Dans un deuxième temps, une fois que l'on sait résoudre le problème d'apprentissage avec un optimiseur, on peut tester d'autres variantes et d'autres hyperparamètres pour obtenir une

convergence avec un temps optimal. C'est généralement le cas des optimiseurs que vous trouvez dans les exemples de code sur internet. L'optimiseur a été testé et comparé avec les autres optimiseurs et il s'est avéré fournir la meilleure convergence.

Pour un problème donné, le graphique ci-dessous présente les performances des différents optimiseurs :



A partir de cette comparaison, il ne faut jamais conclure que tel ou tel optimiseur est le plus performant en général. La conclusion n'est valable que pour ce problème avec ces données. Cependant, certains optimiseurs sont connus pour se retrouver souvent dans le top 3.

Quel est l'intérêt de rechercher un optimiseur et des hyperparamètres fournissant une convergence optimale si un optimiseur a déjà fonctionné ? Les temps de convergence sont longs et se comptent en heures / jours / mois, gagner 20% ou 30% est donc important !

Mod 7. La fonction Softmax

Introduction

▪ Définition

Dans le problème de la classification d'images, le réseau de neurones associe à chaque image plusieurs valeurs dans $\mathbb{R}$. Ces paramètres indiquent une sorte de score pour chaque catégorie étudiée. Il est cependant difficile d'interpréter les valeurs obtenues. On peut cependant intuitivement dire qu'une valeur supérieure aux autres correspond à une catégorie plus probable, mais que dire des nombres négatifs ou nuls ? Il existe une fonction très répandue dans le monde de

l'apprentissage : la fonction *Softmax* qui permet de transformer toute série de valeurs en une série de probabilités. Nous donnons sa définition ci-après pour des valeurs $X=(x_j)_{0 \leq j < n}$ en entrée :

$$\text{Softmax}(X)=[\exp(x_0),\dots,\exp(x_i),\dots,\exp(x_{n-1})].1/\sum_{j=0}^{n-1}\exp(x_j)$$

Remarques :

- Les valeurs en sortie de la fonction *Softmax* sont positives (propriété de la fonction exponentielle).
- Les valeurs en sortie sont inférieures ou égale à 1 (le dénominateur étant supérieur ou égal au numérateur).
- La somme des valeurs en sortie de la fonction *Softmax* est égale à 1.

Par conséquent, nous pouvons énoncer la propriété suivante :

La fonction *Softmax* associe à une série de valeurs quelconques une distribution de probabilités.

Voici un exemple de calcul dans le tableau ci-dessous :

Input	$\exp(x_i)$	Total	$\exp(x_i)/\text{Total}$			
x0	-10,3681	59%	x1	12,7182	3,1721	1,73%
x2	20,0886	6,68%				

Propriétés

▪ Propriété 1

La fonction *Softmax* présente certains avantages :

- qu'elle que soit le signe des valeurs en entrée,
- qu'elle que soit l'ordre de grandeur des valeurs d'entrée (autours de 1/100 ou d'un million),
- qu'elle que soit l'écart de grandeur entre les différents paramètres,

→ la fonction *Softmax* construit toujours une distribution de probabilités.

▪ Propriété 2

La fonction exponentielle est une fonction croissante. Ainsi, plus une valeur d'entrée est grande par rapport aux autres, plus la probabilité associée à sa catégorie augmente, les autres probabilités se voyant diminuer. Prenons un exemple sur un cas simple :

$\begin{array}{l} \text{Input} \exp(x_i) \\ \text{Total Prob} \end{array} \begin{array}{l} -10,36 \\ 11,59\% \end{array} \begin{array}{l} 12,71 \\ 23,17\% \end{array} \begin{array}{l} 21,17 \\ 32,08\% \end{array} \begin{array}{l} 23,81 \\ 36,68\% \end{array}$

Si la deuxième valeur en entrée passe à 2, nous obtenons une nouvelle série de probabilités :

$\begin{array}{l} \text{Input} \exp(x_i) \\ \text{Total Prob} \end{array} \begin{array}{l} -10,36 \\ 11,32\% \end{array} \begin{array}{l} 27,38 \\ 27,84\% \end{array} \begin{array}{l} 22,65 \\ 22,54\% \end{array} \begin{array}{l} 32,08 \\ 32,14\% \end{array}$

En passant de 1 à 2, la probabilité augmente de 11,75% à 26,54%. Les deux autres probabilités dont les valeurs d'entrée sont inchangées diminuent, il y a eu une forme de rééquilibrage. Lorsque l'on utilise la fonction *Softmax*, on peut voir les valeurs d'entrée comme des intensités. Plus l'intensité est importante, plus la probabilité associée est importante. Ainsi la *Softmax* s'adapte et construit intelligemment une liste de probabilités cohérentes.

▪ Propriété 3

Nous pouvons énoncer une propriété importante de la fonction *Softmax* :

Les probabilités issues de la fonction *Softmax* sont invariantes par ajout d'une constante.

Cette propriété vient de la définition de la fonction *Softmax*, en effet, en posant $x'_i = x_i + c$, nous avons pour la i-ème valeur en sortie :

$\text{Softmax}_i(X') = \frac{\exp(x_i + c)}{\sum_j \exp(x_j + c)} = \frac{\exp(x_i) \cdot \exp(c)}{\sum_j \exp(x_j) \cdot \exp(c)} = \text{Softmax}_i(X)$
--

Pour tester cette propriété, reprenons l'exemple précédent :

Inputexp(xi)TotalProbi 10,3681,59%12,71823,17211,73%320,0886,68%

En décalant de 10 les valeurs d'entrée, nous remarquons que les probabilités obtenues restent inchangées :

Inputexp(xi)TotalProbi 98103,11,59%1159874,1510390,611,73%13442413,486,68%

▪ Remarque

Peut-être pensez-vous qu'une formule plus simple comme :

$$S(X)=[x_0, \dots, x_i, \dots, x_{n-1}].1 \sum_{j=0}^{n-1} x_j$$

aurait largement suffi pour construire une liste de valeurs de somme égale à 1. Mais en pratique, des problèmes persistent :

- Avec cette formule simplifiée, les valeurs en entrée doivent être positives sinon on pourrait obtenir des probabilités négatives, ce qui est impossible.
- Avec cette formule simplifiée, il faut être sûr que les valeurs d'entrée soient toutes non nulles sinon on obtient une division par zéro. Il n'est pas facile de gérer cette contrainte. En comparaison, la fonction *Softmax* associe à une série de valeurs nulles des probabilités équiprobables.
- Avec la formule simplifiée, les valeurs (1,1,2) donnent les probabilités en sortie de 25%, 25% et 50%. Les valeurs (100,100,101) produisent des résultats proches de l'équiprobabilité. Une constante additive présente dans les valeurs d'entrée influence donc beaucoup les probabilités construites. En comparaison, la fonction *Softmax* ne souffre pas de ce défaut. Ainsi, la fonction *Softmax* rejette tout biais dans les valeurs d'entrée, ce qui est pratique car les valeurs en entrée ne sont pas forcément centrées sur 0.

Si la formule simplifiée semble plus facile à mettre en œuvre au premier abord, son utilisation s'avère bien plus contraignante.

▪ Instabilité numérique

La fonction *Softmax* peut être sujette à des instabilités numériques. Pour rappel, voici la formule donnant la i-ème composante de la fonction *Softmax* :

$$\text{Softmax}_i(\mathbf{X}) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

La première source de problème provient des exponentielles présentes dans la fonction. Car avec une fonction exponentielle, les grandeurs peuvent grimper vite, très vite. Or, les calculs dans le réseau sont faits en Float32 pouvant au maximum représenter la valeur 3.402823E+38. Ainsi, il suffit qu'une entrée de la fonction *Softmax* soit supérieure à $\ln(3.402823E+38)$ pour que le résultat de l'exponentielle soit égal à $+\infty$. La suite des calculs sera forcément erronée. Voici un exemple :

$$\frac{\exp(x_i)}{\sum_j \exp(x_j)} \text{ en } \text{Float32} \rightarrow \frac{3.2805540622E+34}{5.540622E+34 + 9.4894489613E+38 + \dots} \rightarrow \text{inf}$$

Pour éviter le problème de saturation à l'infini, nous allons soustraire à l'ensemble des valeurs x_i le maximum des x_i noté c pour obtenir une série de nouvelles valeurs notées x'_i . Ce décalage, suivant les propriétés de la fonction *Softmax*, ne change pas les valeurs des probabilités. Ainsi, nous évitons, avec cette astuce, toute saturation :

$$x'_i = x_i - c \quad \text{TotalProb} = \sum_j \exp(x'_j) \rightarrow 1.0$$

Une fois cette première astuce utilisée, nous allons rencontrer un autre problème pour les exposants avec une valeur négative forte. En effet, en Float32, le plus petit nombre positif représentable est $m = 1,4E-45$. Ainsi, lorsqu'il existe une valeur x_i telle que $x_i < c - \ln(m) = c - 103$, nous allons obtenir un résultat nul. Voici un exemple :

$$x'_i = x_i - c \rightarrow \text{TotalProb} = \sum_j \exp(x'_j) \rightarrow 1.0$$

En soit, ce résultat n'est pas incohérent car la probabilité est quasi nulle. Cependant, nous injectons ces probabilités dans la fonction d'erreur du réseau et certaines d'entre elles contiennent des fonctions logarithmes qui en recevant une valeur nulle vont produire une erreur d'exécution.

Pour gérer ce problème, nous allons utiliser une deuxième astuce en appliquant directement la fonction $\ln$ sur la formule de la fonction *Softmax* :

$$\ln(\text{Softmax}_i(\mathbf{X})) = \ln(\exp(x'_i) / \sum_j \exp(x'_j))$$

En développant cette expression, nous obtenons :

$$\ln(\text{Softmax}_i(X')) = x'_i - \ln(\sum_j \exp(x'_j))$$

Les valeurs $\exp(x'_j)$ sont strictement positives et par conséquent le résultat du grand sigma est lui aussi positif strict. Ainsi, le logarithmique ne reçoit jamais une valeur nulle ou négative et son évaluation n'est plus problématique.

Nous verrons plus tard, que la fonction d'erreur utilisée à la suite d'une fonction *Softmax* est souvent la fonction :

$$\text{Cross Entropy Loss}(p, \text{icat}) = -\ln(p_{\text{icat}})$$

Ainsi, vous pourrez parfois rencontrer dans les réseaux, le remplacement du couple : fonction *Softmax* / fonction d'erreur *Cross Entropy Loss* par le couple fonction *Log Softmax* / fonction d'erreur *Negative Log Likelihood Loss* correspondant à la fonction *Cross Entropy Loss* privée de son logarithme :

$$\text{Negative Log Likelihood Loss}(X, \text{icat}) = -x_{\text{icat}}$$

Il ne s'agit pas d'une nouvelle fonction d'erreur ou d'une nouvelle fonction *Softmax* mais de variantes offrant une meilleure stabilité numérique.

Mod 8. Les fonctions d'erreur

Critère géométrique

- **MeanSquaredError**

Cette fonction d'erreur calcule la moyenne des carrés des différences entre chaque valeur de référence et chaque valeur estimée par le réseau. Ainsi, la formule de calcul de la *MSE* pour deux vecteurs y_{true} et y_{pred} de n valeurs est égale à :

$$\frac{1}{n} \sum_{i=0}^{n-1} (y_{\text{true}}[i] - y_{\text{pred}}[i])^2$$

Par exemple, pour $X=[6, 5, 1, 3, 5]$ et $Y=[10, 2, 1, 4, 5]$, on obtient :

$$\frac{1}{5} \sum_{i=0}^{4} |x_i - y_i|^2 = \frac{1}{5} (4 + 9 + 0 + 1 + 0) = 2$$

Ainsi la somme des carrés des éléments de X et Y est égale à 25. Comme il y a 5 éléments dans X et Y , nous obtenons finalement une MSE de 5. Si les données en entrée ont plus d'une dimension, la *MSE* ignore ces dimensions et calcule la MSE des valeurs présentes. Ainsi, pour $X = \begin{bmatrix} 1 & 9 \\ 2 & 6 \\ 9 & 4 \end{bmatrix}$ et $Y = \begin{bmatrix} 4 & 8 \\ 12 & 3 \\ 10 & 7 \end{bmatrix}$, nous obtenons :

$$\frac{1}{6} \sum_{i=0}^{5} |x_i - y_i|^2 = \frac{1}{6} (9 + 1 + 16 + 9 + 4 + 9) = 12.5$$

La somme des différences au carré est ainsi égale à 129. Comme 6 éléments sont présents, on obtient une *MSE* égale à $129/6=21.5$.

▪ Keras

La fonction d'erreur MSE est disponible dans Keras sous le nom *MeanSquaredError()* :

```
import tensorflow as tf
y_true = [0, 0, 1, 5]
y_pred = [10, 0, 1, 5]

mse = tf.keras.losses.MeanSquaredError()
mse(y_true, y_pred)
>> 25
```

Critère d'entropie : CrossEntropyLoss

▪ Présentation

Nous introduisons une notion importante pour la suite : l'entropie permettant de caractériser le niveau de désorganisation ou d'imprédictibilité d'un système. Pour une distribution de probabilité discrète $X(\Omega)=(x_i)_{0 \leq i < n}$ associée aux probabilités $(p_i)_{0 \leq i < n} > 0$, l'entropie est définie ainsi :

$$H(X) = -\sum_i p_i \ln(p_i)$$

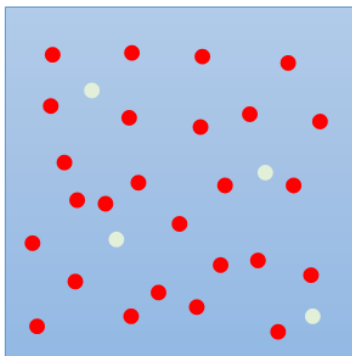
L'entropie est une grandeur positive : en effet, les probabilités étant des valeurs dans l'intervalle $]0,1[$, les termes en log sont tous négatifs. Ainsi le signe $-$ avant le grand sigma donne un résultat positif.

L'entropie n'est pas une valeur bornée : en effet, si nous avons n valeurs équidistribuées, nous obtenons :

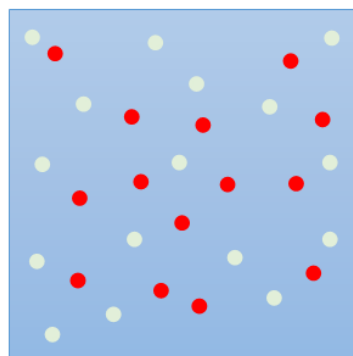
$$H(X) = -n \ln(1/n) \text{ ce qui donne } H(X) = \ln(n).$$

L'entropie atteint son maximum lorsque les probabilités sont équidistribuées. Prenons le cas $n=2$, ainsi nous pouvons écrire : $H(X) = -p \ln(p) - (1-p) \ln(1-p)$. En dérivant par rapport à p , nous obtenons $H' = -\ln(p) + \ln(1-p)$, dérivée qui s'annule pour $p=1/2$.

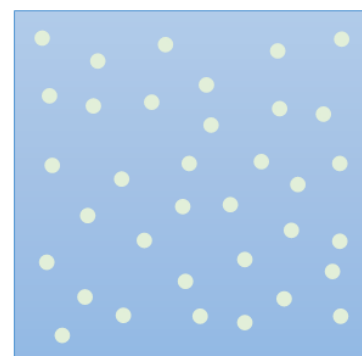
Examinons un cas pratique avec trois boîtes contenant des billes rouges et des billes vertes :



R = 26 V = 4



R = 14 V = 16



R = 0 V = 30

Dans la boîte de gauche, la probabilité de tirer une boule rouge est de 26/30 et celle de tirer une bille verte est de 4/30. Ainsi, la probabilité de tirer une boule rouge est plus certaine.

Dans la boîte au centre, la probabilité de tirer une boule rouge est de 14/30 et celle de tirer une bille verte de 16/30. Il y a presque une chance sur deux de tirer une boule rouge ou une boule verte. Il devient difficile d'être sûr de la couleur tirée.

Dans la boîte de droite, nous sommes certains que la boule tirée sera de couleur verte. La probabilité de tirer une boule verte est de 1 contre 0 pour une boule rouge.

Calculons les valeurs de l'entropie pour chacune de ces situations :

- Boîte de gauche : $-26/30 \cdot \ln(26/30) - 4/30 \cdot \ln(4/30) = 0.3926$
- Boîte au centre : $-14/30 \cdot \ln(14/30) - 16/30 \cdot \ln(16/30) = 0.6909 < \ln(2) = 0.6931$
- Boîte de droite : $-30/30 \cdot \ln(30/30) = 0$

L'entropie de la boîte de gauche est inférieure à la boîte du centre car en tirant une boule dans cette boîte nous sommes presque sûr qu'elle sera rouge. Dans la boîte au centre, nous ne sommes sûr de rien, elle peut être rouge comme verte et l'entropie calculée est proche de l'entropie maximale possible : $\ln(2)$. La boîte de droite correspond à une situation idéale où nous sommes absolument certains que toute boule tirée sera verte, en conséquence l'entropie est nulle.

▪ CrossEntropyLoss

Nous allons utiliser la définition de l'entropie croisée pour construire la fonction d'erreur **Cross-Entropy Loss**. L'entropie croisée pour deux distributions discrètes p et q est définie de la façon suivante :

$$\text{CrossEntropyLoss}(p, q) = -\sum_{p_i > 0} p_i \cdot \ln(q_i)$$

Quand on compare une distribution de probabilités q avec une distribution de probabilités p , l'entropie croisée atteint son minimum lorsque les deux distributions sont égales i.e. $p_i = q_i$. Ce minimum correspond à la valeur $H(p) = H(q)$.

Dans le processus d'apprentissage, les vérités terrain correspondent à une distribution de probabilités p connue. Le réseau doit lui chercher à construire la distribution de probabilités q afin qu'elle corresponde au mieux à la distribution de référence. Si nous calculons l'entropie croisée entre la distribution p (fixée) et q (estimée par le réseau) et que nous cherchons par le processus de descente du gradient à minimiser cette grandeur, alors cela équivaut à faire tendre la distribution q vers la distribution p .

Dans le problème de classification, lorsque nous savons que la catégorie donnée par le réseau doit correspondre à la catégorie $icat$, cela équivaut à dire que les probabilités obtenues par la fonction *Softmax* devraient être toutes nulles à l'exception de la i -ème composante qui devrait être égale à 1 (100%). Ainsi, la distribution de probabilités p de référence correspond au vecteur suivant :

$$\mathbf{picat}=[0,\dots,0,1_{icat},0,\dots,0]$$

L'expression de l'erreur associée à l'entropie croisée se réécrit ainsi :

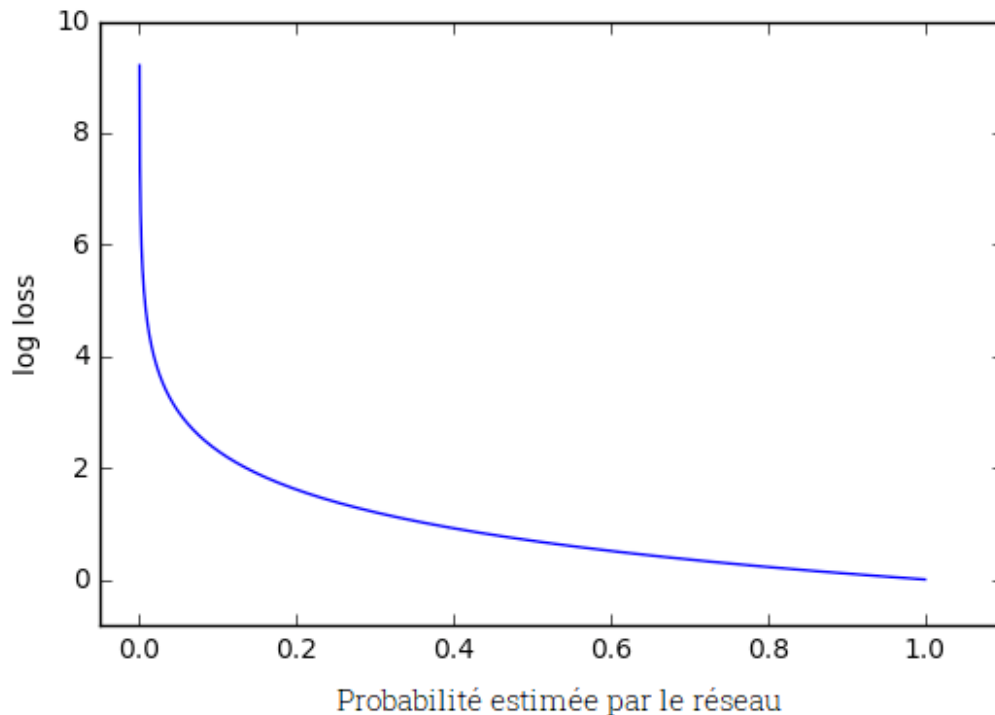
$\text{CrossEntropyLoss}(\mathbf{picat},\mathbf{q})=-\ln(q_{icat})$

De l'expression précédente, il ne reste qu'un seul terme correspondant à la catégorie $icat$. Ainsi en minimisant la fonction *CrossEntropyLoss*, les poids du réseau s'optimisent pour que les probabilités en sortie de la fonction *Softmax* donnent une probabilité de 1 pour les catégories désirées.

Note : la fonction d'entropie croisée est souvent confondue avec la fonction **logistic loss** aussi appelée **log loss**. Bien que ces deux mesures soient des notions différentes, utilisées comme fonction d'erreur dans le problème de classification, elles calculent en fait la même grandeur et deviennent dans ce contexte équivalentes.

▪ Comportement

La fonction d'erreur *Cross-Entropy* ou *log loss*, mesure la performance de prédiction du réseau. En effet, l'erreur est nulle lorsque la probabilité de la bonne catégorie atteint 100%. D'autre part, plus la probabilité s'éloigne de 1, plus l'erreur augmente. Voir l'exemple dans le graph ci-dessous :



Lorsque la probabilité estimée s'approche de 1, la pénalité associée décroît lentement. Cependant, lorsque la probabilité estimée est inférieure à 20% et qu'elle continue à diminuer, alors la pénalité augmente très vite. Ainsi, cette fonction d'erreur est particulièrement sensible aux mauvaises prédictions car la valeur de l'erreur est très importante.

▪ Keras

Différentes fonctions d'entropie croisée sont disponibles dans Keras et toutes calculent la même grandeur. Cependant, elles diffèrent par le format choisi pour représenter les vérités terrain :

- Si les vérités terrain sont encodées sous la forme d'une série de 0 avec un seul 1 pour indiquer la classe correcte, on utilise la fonction d'erreur **CategoricalCrossentropy**. Exemple d'encodage pour un problème de classification avec 3 classes : [1,0,0] , [0,1,0], [0,0,1].
- Si les vérités terrain sont encodées avec le numéro de la catégorie, on utilise la fonction d'erreur **SparseCategoricalCrossentropy**. Exemple d'encodage pour un problème de classification avec 3 classes : [0], [1] , [2].
- Dans le cas où la sortie du réseau fournit une seule valeur, cette valeur, grâce à un threshold fixé par défaut à 0.5, traduit deux catégories. Le problème de

classification ne peut alors traiter que deux classes. Les vérités terrains s'encodent sous la forme : 0 ou 1 et dans ce cas, on utilise la fonction **BinaryCrossEntropy**.

- **Les métriques**

Durant la phase d'apprentissage, les fonctions d'erreur permettent d'évaluer la qualité de la prédiction d'un réseau par rapport aux réponses exactes. Les fonctions de métrique, ou les métriques, ont un objectif similaire mais sont cependant utilisées uniquement durant la phase de validation du réseau. Ainsi, une fonction d'erreur peut être utilisée comme métrique alors qu'une métrique ne peut pas être utilisée comme fonction d'erreur. Pourquoi ? Simplement parce que les métriques ne sont pas adaptées à la méthode de descente du gradient. En effet, on peut citer l'exemple d'une métrique évaluant le pourcentage de réponses correctes, cette fonction est dérivable par morceaux mais sur chaque zone où elle est dérivable, sa dérivée est nulle, ce qui empêche l'algorithme de rétropropagation de fonctionner.

- **Pourcentage de prédictions réussies**

Cette métrique évalue le pourcentage de bonnes prédictions. Cette métrique est intuitive et permet d'évaluer le taux de succès du réseau.

- **Logit**

Il s'agit d'un terme ayant un sens spécifique dans le monde de l'apprentissage. Le logit désigne les valeurs brutes (non-normalisé) en sortie du réseau. Ainsi, dans les problèmes de classification, le logit est le vecteur de valeurs qui va ensuite être transmis à la fonction Softmax pour normalisation. Le logit est un vecteur de valeurs quelconques (non-normalisées) alors que le vecteur en sortie de la fonction Softmax est un vecteur de probabilités dont les valeurs sont dans l'intervalle $[0,1]$ et dont la somme vaut 1.

- **Keras**

Cette métrique existe dans Keras sous trois formes :

SparseCategoricalAccuracy , CategoricalAccuracy et BinaryAccuracy
--

- *SparseCategoricalAccuracy* demande l'indice de la catégorie réelle comme référence.
- *CategoricalAccuracy* demande un vecteur `[0 0 0 1 0 0 0]` (one_hot) comme référence.
- *BinaryAccuracy* s'utilise lorsqu'une seule donnée est présente en sortie du réseau, le threshold (0.5 par défaut) fournissant la prédiction correspondante. Le vecteur des références contient des valeurs 0 ou 1.

Note

Les métriques *SparseCategoricalAccuracy* et *CategoricalAccuracy* prennent comme prédiction du réseau aussi bien des données sous forme de logit que sous forme de probabilités. En effet, la catégorie retenue dans les deux cas correspond à l'indice de la valeur maximale (argmax) du vecteur de prédiction.

Dans les deux exemples suivants, la première et la dernière prédiction s'avèrent correctes ce qui fait un ratio de 66% de bonnes réponses.

```
import tensorflow as tf

sparse = [[0], [1], [2]]
logits = [[.8, .1, .1], [.5, .3, .2], [.2, .2, .6]]

sparse_cat_acc = tf.metrics.SparseCategoricalAccuracy()
sparse_cat_acc(sparse, logits)

>> <tf.Tensor: shape=(), dtype=float32, numpy=0.6666667>
onehot = [[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]]
logits = [[.8, .1, .1], [.5, .3, .2], [.2, .2, .6]]

cat_acc = tf.metrics.CategoricalAccuracy()
cat_acc(sparse, logits)

>> f.Tensor: shape=(), dtype=float32, numpy=0.6666667>
```

Les trois premières prédictions s'avèrent correctes ce qui fait un ratio de 75% de bonnes réponses :

```
y_true = [1, 1, 0, 0]
logit = [0.98, 1, 0.49, 0.51] # le threshold de binarisation est fixé par défaut à 0.5
```

```
m = tf.keras.metrics.BinaryAccuracy()  
m( y_true, logit )  
  
>> <tf.Tensor: shape=(), dtype=float32, numpy=0.75>
```

Mod 9. Rappels Python

Vous pouvez tester en ligne les exemples grâce aux sites suivant :

- <https://www.online-Python.com/>

- <https://replit.com/languages/Python3>
- https://www.onlinegdb.com/online_Python_compiler

Ce chapitre a été mis en place pour les personnes venant d'un autre langage. Il est aussi obligatoire pour les personnes ayant une faible pratique du langage Python. Y sont présentés les points critiques et les pièges propre à ce langage.

Pour ceux ayant une expérience avancée en Python, vous pouvez parcourir cette page pour détecter les sujets que vous connaissez moins bien et qui nécessitent un éclaircissement.

Dans tous les cas, les questions des QCM doivent être préparées.

Lancement du programme

▪ Compilation

Lorsqu'un programme Python (.py) est exécuté, le code source passe par une étape de compilation pour construire un bytecode Python (.pyc). Ce bytecode est ensuite chargé par une machine virtuelle Python qui va exécuter ligne à ligne le programme. On oublie souvent l'existence de cette phase de compilation en Python car peu d'erreurs y sont détectées. En effet, sont principalement détectées des erreurs basiques de structuration et de syntaxe :

<i>Exemple d'erreurs de compilation</i>	
<code>print("toto"</code>	oubli de la parenthèse fermante
<code>fr i in range(4):</code>	erreur de syntaxe sur le mot clef for
<code>for i in range(4)</code>	oubli des : en fin de ligne
<code>print("toto)</code>	oubli du " pour terminer la chaîne de caractères
<code>return = 4</code>	utilisation d'un mot clef comme variable
<code> a = 3</code>	mauvaise indentation

▪ Erreur d'exécution

Si la compilation réussit, les lignes de code sont interprétées une à une même si des erreurs grossières peuvent encore être présentes dans le code. En effet, l'absence de typage des paramètres des fonctions et des variables empêche le compilateur de vérifier la compatibilité des types lors d'un appel de fonction ou lors de l'application d'un opérateur. Ce sera uniquement au moment de l'exécution que cette ambiguïté pourra être levée. Voici quelques exemples :

```
print(a)    # erreur variable a inconnue, en effet, a est initialisée par la suite
a = 7
```

Avertissement :

Il se peut que la ligne `print(a)` ne déclenche pas d'erreur. En effet, si vous êtes en mode d'**exécution interactive**, les variables et les fonctions sont conservées d'un lancement à l'autre du code. Si dans un essai précédent, vous avez initialisé une variable `a`, cette valeur sera encore présente en mémoire. En mode d'exécution standard, les variables sont cette fois libérées en fin de programme.

▪ Définition

Le programme s'exécute ligne à ligne depuis la première ligne en haut du fichier source. Le code associé à une fonction est exécuté uniquement lors de l'appel de cette fonction.

Une variable/fonction ne peut être utilisée que si elle a été définie auparavant durant l'exécution.

```
a = 4
print(a)    # OK : la variable a été définie avant d'être utilisée
toto()      # ERREUR d'exécution : la fonction appelée est inconnue
def toto():
    print("toto")
```

▪ Stockage dynamique des variables/fonctions

Dans un code Python, les variables globales et les fonctions, ainsi que les espaces de noms, sont stockées dynamiquement durant l'exécution à l'intérieur d'un dictionnaire. On peut utiliser la fonction `vars()` pour afficher son contenu. Ainsi, l'utilisation d'une variable ou

d'une fonction en Python va d'abord déclencher la recherche de son nom dans le dictionnaire afin de pouvoir connaître la valeur ou le code associée à cette entrée :

```
a = 5
print(vars())      # { ..., 'a': 5}
def toto():
    print("toto")
print(vars())      # { ..., 'a': 5, 'toto': <function toto at 0x000002AB4A1CDEE8>}
```

▪ Import de librairies

Pour faire simple, l'import d'une librairie en Python grâce au mot clef **import** consiste à exécuter les fichiers de chargement de cette librairie. Un dictionnaire propre à cette librairie est rempli avec les noms des fonctions et des variables exportées par cette bibliothèque. Ensuite, un lien est créé depuis le dictionnaire du programme courant vers le dictionnaire associé à cette librairie. Ainsi, les fonctions de cette librairie deviennent accessibles depuis le programme principal en utilisant la syntaxe : *nom_librairie.fonction(...)*.

```
import numpy      # charge la librairie numpy

print(vars())     # le pointeur vers le dictionnaire de cette librairie apparaît dans les variables locales
>> ..., 'numpy': <module 'numpy' from ... >

print(vars(numpy)) # on peut regarder le contenu du dictionnaire associée à cette librairie
>> 'nansum': <function nansum at 0x000002043BF9F318>,
    'nanmax': <function nanmax at 0x000002043BF9AC18>,
    'nanmin': <function nanmin at 0x000002043BF9A9D8>,
    'nanargmax': <function nanargmax at 0x000002043BF9F0D8>,...

t = numpy.nansum(10) # appel de la fonction nansum de la librairie numpy
```

Avertissement :

Vous pourrez trouver des exemples de code utilisant cette syntaxe : *from numpy import **. Cette syntaxe permet de copier le contenu du dictionnaire de la librairie vers le dictionnaire du programme courant. Cette astuce permet d'éviter de nommer la librairie explicitement lors chaque l'appel : *nom_librairie.la_fonction(...)*. Ainsi, les fonctions sont appelées exactement comme si elles faisaient partie de votre code source : *la_fonction(...)*. Cette facilité est très dangereuse, car elle induit des conflits de nom entre les variables/fonctions de votre code et

celles de la librairie. De plus, cette astuce masque l'origine des fonctions ce qui représente un problème lorsque l'on utilise plusieurs librairies.

Préférez l'approche ci-dessous :

Afin d'éviter d'écrire le nom complet de la librairie, il est possible de donner un alias au nom de la librairie lors de la commande import :

```
import nom_librairie as alias
```

Voici un exemple :

```
import numpy as np
t = np.nansum(10) # np est suffisant pour appeler les fonctions de la librairie
```

▪ Indentation

La traditionnelle paire d'accolades a été remplacée en Python par l'indentation. On aime, on n'aime pas, ce n'est pas la question ! Cependant, les programmes écrits en Python ont une apparence plutôt structurée. Ainsi, chaque sous-bloc de code est introduit en laissant une tabulation ou plusieurs espaces. A l'intérieur du même bloc, l'indentation doit être identique. Voici le style généré :

```
a = 4           # déclaration d'une variable dans le bloc principal
def toto():
    print("toto") # cette ligne appartient au bloc de la fonction toto()
b = 5           # cette ligne appartient au bloc principal
def titi():
    for i in range(10): # cette ligne appartient au bloc de la fonction titi()
        print(i)      # indentation supplémentaire => print() est dans le corps de boucle
        print(i)      # on reste dans le même bloc, l'indentation est identique à la ligne précédente
    toto()           # appel de la fonction toto() depuis le bloc de la fonction titi()
titi()              # appel de la fonction titi() depuis le bloc principal
```

Structures de contrôle

▪ Les conditions

Caractéristiques principales :

- Il n'est pas nécessaire de parenthéser les conditions.

- Le signe `:` est utilisé pour marquer la fin du test.
- Les mots-clefs **and**, **or** et **not** désignent des opérateurs booléens.

Voici un exemple:

```
for i in range(100):  
    if i % 3 == 0 and i % 7 == 0 :  
        print(i)  
>> 0 21 42 63
```

→ A noter que les constantes booléennes sont : **True** et **False** avec des majuscules.

▪ Les boucles

Syntaxe pour aller de l'indice 0 à n-1 :

```
n = 5  
for i in range(n):  
    print(i)  
>> 0 1 2 3 4
```

Syntaxe pour aller de m à n-1 par pas de 2 :

```
m = 3  
n = 10  
for i in range(m,n,2):  
    print(i)  
>> 3 5 7 9
```

Les containers

▪ Les listes

Objet Python très commun, la liste permet de stocker et d'indexer plusieurs éléments de types hétérogènes. Les listes permettent d'ajouter ou de retirer des éléments dynamiquement. En

contrepartie de cette souplesse, elles sont plus lentes que des tableaux en langage C++ ou Java.

Exemple de méthodes disponibles pour une liste Python

<code>L = [1, 2, 5, 2]</code>	Création et initialisation
<code>L = [1, "toto", 3.1415, [1,2]]</code>	Création d'une liste contenant des types hétérogènes
<code>L.append(5)</code>	Ajout de l'élément 5 en fin de liste
<code>L.insert(2,9)</code>	Insère la valeur 9 à l'index 2
<code>L.remove(2)</code>	Retire la première valeur 2 trouvée dans la liste
<code>len(L)</code>	Nombre d'éléments dans la liste
<code>L.sort()</code>	Trie la liste inplace
<code>L[0] += 1</code>	Accès par index et incrémentation
<code>L.clear()</code>	Vide la liste
<code>[1, 2] + [4 , 5]</code>	L'opérateur + concatène deux listes
<code>print(L)</code>	Affiche le contenu de la liste

▪ Les tuples

Un tuple représente une liste de taille fixe non modifiable une fois créé (immutable). Un tuple peut stocker des éléments de types hétérogènes. On peut modifier le contenu des éléments mutables présents dans le tuple.

Exemple de méthodes disponibles pour un tuple

<code>L = (1, 2, [4,5])</code>	Création d'un tuple contenant des types hétérogènes
<code>len(L)</code>	Nombre d'éléments dans la liste

L[0]	Accès par indexation
L[2][0] +=1	Accès à la valeur 4 de la liste [4,5] => L: (1, 2, [5,5])
print(L)	Affiche le contenu du tuple

Pour faciliter l’empaquetage et le dépaquetage d’éléments, des syntaxes compactes sont proposées :

```
x,y,z = L      # dépaquetage
>> x: 1
>> y: 2
>> z: [5,5]

L = x,z        # empaquetage
>> (2,[5,5])
```

Les tuples sont censés être plus rapides en performance que les listes.

En utilisant la syntaxe compacte des tuples, on peut effectuer plusieurs affectations en 1 ligne :

```
x,y,z = 1,2,3
```

Les tuples représentent un moyen extrêmement pratique pour retourner plusieurs éléments comme résultat d’une fonction, éléments pouvant être de types différents.

```
def toto():
    return 1,"toto"
a,b = toto()
```

- **Les dictionnaires**

Les dictionnaires généralisent les listes en permettant une indexation non plus uniquement sur des entiers mais sur n'importe quel type d'éléments appelés **clefs**. Ainsi, un dictionnaire associe une valeur à une clef. Clefs comme valeurs peuvent être de types hétérogènes. Aucune clef ne peut exister en double dans le dictionnaire et si la valeur associée à une clef est modifiée, alors, la précédente valeur est écrasée. Voici les syntaxes des dictionnaires :

<i>Exemple de méthodes disponibles pour un dictionnaire :widths: 25 50</i>	
D = {}	Création d'un dictionnaire
D["PV"] = 4	Associe la valeur 4 à la clef PV
D["PV"] = 8	Modifie la valeur associée à la clef PV
D[9] = "ARC"	Associe la valeur "ARC" à la clef 9
print(D)	affiche le contenu du dictionnaire
if "PV" in D	Teste si la clef PV existe dans le dictionnaire
L = {"PV": 4, "OR": 2}	Initialisation d'un dictionnaire par passage de paires clefs/valeurs
len(L)	Indique le nombre d'entrées ou de clefs dans le dictionnaire

▪ Boucle sur les containers

On peut parcourir les éléments d'un container s'il est dit **iterable**. Les containers comme les listes, les tuples ou les dictionnaires appartiennent à cette catégorie. On peut ainsi écrire un algorithme à base de boucles for pouvant fonctionner avec différents types de container sans changer le code associé. Voici un exemple :

```
def Aff(data):
    for e in data : # syntaxe du foreach en Python
        print(e, " / " end="")

L = [4,5,6]      # container liste
Aff(L)
>> 4 / 5 / 6 /

L = (1,2)        # container tuple
Aff(L)
>> 1 / 2 /

L = {'PV': 4, 'OR': 2} # container dictionnaire
```

```
Aff(L)
>> PV / OR /
```

▪ Les pièges du langage

Nous vous présentons les pièges les plus courants faisant généralement perdre pas mal de temps aux développeurs.

▪ Conflit entre variables globales et locales

Dans le corps d'une fonction, si aucune variable locale ne porte le nom recherché, alors on recherche une variable globale portant ce nom. Voici un exemple :

```
a = 5
def fnt1():
    a = 6    # définition d'une variable locale valant 6 masquant la variable globale
    print(a) # >> 6
def fnt2():
    print(a) # absence de variable locale, on consulte la variable globale
```

La syntaxe : $a = \dots$ dans une fonction crée, par convention, une variable locale. Si vous cherchiez en fait à affecter une variable globale, le résultat est à l'opposé ce que vous attendiez car en plus cette nouvelle variable locale va masquer la variable globale que vous cherchiez à modifier. Pour modifier une variable globale depuis l'intérieur d'une fonction, il faut utiliser une syntaxe spéciale utilisant le mot clef *global* :

Pour pouvoir accéder à une variable globale en écriture à l'intérieur d'une fonction, il faut utiliser le mot clef **global** et déclarer la variable en début de fonction.

Voici un exemple :

```
a = 4
def fnt3():
    global a    # relie le nom "a" à la variable globale nommée "a"
    a = 6       # la variable globale vaut maintenant 6
fnt3()
print(a)       # >> 6
```

▪ Le tout objet

En Python tous les éléments sont des objets. Ainsi l'opération d'affectation : $a=b$ ne crée pas un nouvel objet mais une référence vers un objet existant.

Chaque objet reçoit un identificateur unique accessible à travers la fonction `id()` :

```
a = "TOTO"
print(id(a))    # 60400
b = a           # b se comporte comme une référence
print(id(b))    # 60400 => b pointe vers le même objet chaîne de caractères
a = a + "TITI"  # création d'un nouvel objet référencé par la variable "a"
print(id(a))    # 68304 => on a bien construit un nouvel objet
```

A titre de curiosité, nous pouvons remarquer que les types de base comme les entiers ou les flottants sont gérés comme des objets :

```
a = 4
print(id(a))    # 60400
b = a           # b désigne le même objet que a
print(id(b))    # 60400
a = a + 1       # création d'un nouvel objet entier de valeur 5 et association à la variable a
print(id(a))    # 68304 => on a bien construit un nouvel objet
print(id(b))    # 60400 => l'ancien objet existe toujours par l'intermédiaire de b
```

La ligne $a = a + 1$ déclenche la création d'un nouvel objet temporaire et la variable a pointe alors vers ce nouvel objet, ce qui explique que son identificateur change.

Si les entiers sont gérés comme des objets, alors, en écrivant $b = a$ puis $b++$, la variable a devrait aussi être modifiée ? Ceci créerait pas mal de confusion par rapport aux réflexes habituels. Mais ce scénario problématique ne peut arriver car les types de base ont été déclarés immutables. Ainsi, l'opérateur $++$ n'existe pas en Python rendant le scénario évoqué impossible.

▪ Passage d'arguments

Dans le langage Python, lors de l'appel d'une fonction, **les arguments sont passés par affectation**. Ainsi, pour chaque paramètre est créé une variable locale faisant référence à l'objet passé en argument. Ce mode de passage correspond au passage par adresse en C/C++ et au passage par référence en Java/C#. Il ne s'agit pas du passage par référence du C++ car l'argument et le paramètre sont deux variables distinctes.

Voici un exemple :

```

L = [ 1 , 2 , 3 ]

def fnt(T):
    T[0] = 7
    print(id(L))  # ID de L : 59912
    print(id(T))  # ID de T : 59912

    T = 4         # la variable s'utilise comme toute variable
    print(id(L))  # ID de L : 59912
    print(id(T))  # ID de T : 14392

fnt(L)
print(L)         # >> [7, 2, 3]

```

la variable L désigne un objet de type liste. L'appel à la fonction fnt initialise la variable locale T avec l'adresse de la liste L . Ainsi la modification $T[0]=7$ est directement effectuée sur l'objet liste initial. La variable L et la variable T désigne ainsi le même objet. La ligne $T = 4$, associe la variable T à un nouvel objet entier. Cette affectation n'a pas d'influence sur la variable L et la liste associée. Une fois la fonction terminée, la variable T est détruite, mais l'objet liste reste actif car il existe toujours la variable L permettant d'y accéder.

En conclusion:

Une affectation effectuée sur un paramètre de fonction ne modifie pas l'objet passé en argument.

▪ Modifier un entier passé en argument

Comme il n'est pas possible de modifier un entier passé en paramètre depuis l'intérieur d'une fonction, on peut utiliser le mécanisme de retour d'une fonction pour modifier son contenu :

```

x, y = 1, 2

def fnt1(a, b):
    a += 1      # a et b sont des variables locales
    b += 1
    print(x,y)  # 1 et 2, modifier a et b n'a pas changé x et y
    return a,b  # retour des résultats

x, y = fnt1(x, y)  # récupération des résultats et affectation de x et y dans la foulée
print(x,y)        # >> 2,3

```

L'autre astuce consiste à stocker les valeurs dans une liste qui permet la modification de ses valeurs à l'intérieur de la fonction :

X = [1,2]

def fnt1(L):

`L[0] += 1` # a et b sont des variables locales `L[1] += 1` `print(X)` # `>> 2, 3` la liste originale a été modifiée

`fn1(X)` # Le contenu de X sera modifié `print(X)` `>> 2, 3`

▪ Clonage

Comme les variables sont des adresses vers des objets Python en mémoire, le code suivant ne produit pas de duplication de l'objet liste :

```
L = [1, 2, 3]
T = L
T[0] += 1      # la modification sur T impacte la liste désignée par L
print(L)
>> [2, 2, 3]
```

Il faut passer par la fonction **copy()** pour effectuer une duplication :

```
L = [1, 2, 3]
T = L.copy()
T[0] += 1

print(T)
>> [2, 2, 3]
print(L)
>> [1, 2, 3]
```

Mais attention, il s'agit d'une copie superficielle effectuée au premier niveau : les éléments à l'intérieur de la liste ne sont pas dupliqués !

```
L = [ [1,2], [3,4] ]
T = L.copy()
T.append([5,6])

print(L)
>> [ [1,2], [3,4] ]
print(T)
>> [ [1,2], [3,4] , [5,6] ]  # T et L sont deux objets différents

L[0][0] = 9
L[1][1] = 8
print(L)
>> [ [9,2], [3,8] ]
print(T)
>> [ [9,2], [3,8] , [5,6] ]  # mais pas tant que cela, car T contient des adresses vers les éléments de L
```

La fonction `copy()` a créé un nouvel objet liste, mais elle l'a initialisé avec les adresses des listes contenues dans `L`. Ainsi on retrouve dans la liste `T`, les listes initiales `[1,2]` et `[3,4]`. Pour dupliquer un objet en construisant une copie profonde de manière récursive, il faut utiliser la fonction `copy.deepcopy()` :

```
import copy
L = [ [1,2], [3,4] ]
T = copy.deepcopy(L)
L[0][0] = 9
L[1][1] = 8

print(L)
>> [[9, 2], [3, 8]]
print(T)
>> [[1,2], [3,4]] - Les éléments des listes dans T sont indépendants de ceux présents dans la liste L
```

Introduction à l'apprentissage profond

Cette ressource est issue d'une publication du numéro 108 de La Revue 3EI d'avril 2022. Steeven Janny, ancien élève du DER Nikola Tesla de l'ENS Paris-Saclay, est doctorant au laboratoire LAGEPP sur l'établissement de modèles profonds pour la simulation et la prédiction de phénomènes physiques. Ludovic De Matteis, élève du DER Nikola Tesla, est auteur de l'article « Personalized Human-Swarm Interaction Through Hand Motion » [6] issu de son stage à l'EPFL sur l'apprentissage des mouvements intuitifs de l'utilisateur pour le contrôle d'une flotte de drone. Wenqi Shu- Quartier, ancienne élève du DER Nikola Tesla, est doctorante au laboratoire Neurospin du CEA sur l'application de l'apprentissage par ordinateur à la comparaison de la maladie d'Alzheimer avec son analogue chez le chimpanzé.

Cette ressource, la deuxième du « Dossier Intelligence Artificielle » [7], présente l'outil à la source des progrès déterminants de l'IA au 21^e siècle : les réseaux de neurones et leur entraînement dit apprentissage profond. Après une présentation détaillée des réseaux de neurones, la ressource développe les méthodes d'apprentissage en soulignant l'importance des données. Elle présente ensuite quelques évolutions des réseaux de neurones : convolutionnels (CNN), récurrents (RNN), auto-encodeurs (AE), adversariaux génératif (GAN) et transformers pour finir sur une implémentation pratique d'un entraînement de réseau de neurones pour la classification de chiffres manuscrits.

1 – Introduction

Depuis les bureaux spacieux des grands groupes industriels, jusqu'aux locaux des incubateurs à start-ups accompagnant de jeunes entrepreneurs ambitieux, tous annoncent la venue d'une quatrième révolution industrielle sonnant le début de l'ère de l'intelligence artificielle et du Big Data. Comme ses grandes sœurs, cette révolution s'appuie sur une nouvelle sémantique dont certains éléments de langage vous sont sûrement déjà familiers : Smart Tech, Cloud computing, IA, etc... La quatrième révolution industrielle serait donc celle d'une nouvelle génération de machines et d'une automatisation plus poussée et plus généralisée, une transition permettant d'améliorer l'efficacité et la productivité. Pour cela, le monde moderne dispose de deux nouveaux outils redoutables : l'expansion fulgurante d'Internet, et les récentes avancées en matière d'intelligence artificielle.

La dernière décennie donne en partie raison à ces nouveaux prophètes grâce à un alignement des astres technologiques particulièrement opportun : (1) une explosion de la puissance de calcul des ordinateurs, (2) une mise à disposition de quantités massives de données numériques (le fameux Big Data) et (3) la remise au goût du jour d'une technologie vieille de 60 ans dont le potentiel est enfin pleinement libéré grâce aux deux premiers points, nous voulons bien sûr parler des réseaux de neurones.

L'apprentissage profond (c'est le nom de la discipline) est un domaine aujourd'hui soutenu par une communauté très active d'industriels et de chercheurs, et compte plusieurs succès à son palmarès. Ses premières grandes avancées sont concentrées sur la vision par ordinateur : reconnaissance d'objets, localisation de défauts dans les chaînes de production, détection de maladies grâce à l'imagerie médicale, etc... Récemment, le champ d'application s'est élargi : traitement du langage naturel (GPT-3 développé par OpenAI, chatbots), robotique (apprentissage par renforcement), reconnaissance vocale (Alexa, Siri, Cortana...), et même physique, avec notamment la réussite spectaculaire d'un

modèle d'apprentissage profond pour la stabilisation du plasma dans le réacteur à fusion ITER, là où les techniques de contrôles traditionnelles avaient échoué.

Malheureusement, tout succès médiatique vient avec son lot de contre-vérités et d'exagérations. Nombre d'articles relatent chaque jour les dernières prouesses de l'IA. Souvent dithyrambiques, parfois même alarmistes, il est difficile de démêler la science du travail d'écriture. Après une première ressource [8] présentant différents domaines et méthodes de l'IA, cet article a pour ambition de poser les bases du fonctionnement et de la mise en œuvre des réseaux de neurones et de mettre en avant les principaux défis que ceux-ci doivent encore relever.

Figure 1 : Image générée par un réseau de neurones à partir de la phrase « Un astronaute jouant au basketball dans l'espace avec des chats, dessiné comme dans un livre pour enfants » (OpenAI Dall-E 2).

2 – Les réseaux de neurones comme modèles paramétriques.

2.1 - Les données, ou l'or numérique.

Votre âge, votre sexe, le contenu de votre dernier panier de courses, vos performances au dernier jeu mobile à la mode, toutes ces données ont un prix que bon nombre d'entreprises sont prêtes à acheter. Cette ruée vers l'or numérique est en partie motivée par le principe d'application du deep learning.

Mettons-nous quelques instants dans la peau d'un ingénieur en apprentissage profond. Nous nous intéressons donc à la recherche d'une fonction $y = f(x)$ dont nous ignorons complètement la formulation. Il peut s'agir d'une fonction arbitrairement complexe, comme la relation entre les images perçues par une caméra embarquée sur un véhicule et la trajectoire optimale que le véhicule devrait suivre pour atteindre un certain emplacement. À défaut de pouvoir écrire mathématiquement cette fonction, nous disposons de N mesures $(x_i, y_i)_{i=1 \dots N}$ de celle-ci, réunies dans un grand jeu de données.

Le problème est alors le suivant : comment construire une approximation de la fonction f à partir de ces seules mesures ? Avant même de s'intéresser aux méthodes envisageables, on peut intuitivement se convaincre qu'une fonction complexe nécessitera plus de points de mesure. Par exemple, deux mesures suffisent pour approcher une fonction linéaire, mais au moins trois mesures sont nécessaires pour approcher une fonction quadratique. Un des principaux enjeux du Big Data est ainsi de collecter une large quantité de données afin de pouvoir approcher des fonctions de plus en plus complexes.

2.2 - Un détour par la régression linéaire

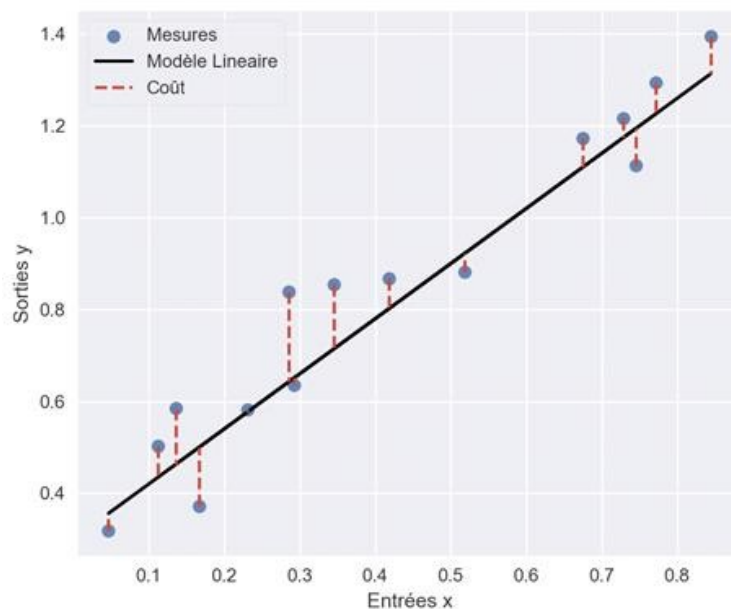
Pour approcher une fonction dont on ne connaît que quelques réalisations, la méthode classique consiste à choisir un modèle paramétrique, c'est-à-dire une fonction dont le comportement sera contrôlé par un certain nombre de paramètres qu'il nous faudra régler (on parle d'identification des paramètres).

La régression linéaire est l'exemple le plus trivial de cette technique : il s'agit de chercher les valeurs de a et b de sorte que l'équation $y = ax + b$ représente au mieux les points de mesure du jeu de données. L'identification de a et b se fait généralement en résolvant un problème de minimisation d'une mesure de l'erreur, comme la distance quadratique :

$$(a,b) = \underset{(a,b)}{\operatorname{argmin}} J = \sum_{i=0}^N (y_i - (ax_i + b))^2$$

Cette solution s'adapte sans problème aux plus grandes dimensions en ré-écrivant le modèle linéaire sous forme matricielle $y = Wx + b$. La régression linéaire est intéressante car sa forme, très simple, permet d'obtenir rapidement la solution optimale au problème, c'est-à-dire l'obtention des valeurs de W et b donnant la plus petite erreur J possible. L'inconvénient, c'est que bien peu de phénomènes sont linéaires : il y a peu de chance pour qu'un modèle linéaire parvienne à prédire l'évolution du prix des actions en bourse.

Figure 2 : La régression linéaire consiste à trouver la droite qui minimise la distance aux points de mesures, représentée en rouge sur la figure :



2.3 - Un modèle plus puissant

Tout comme la régression linéaire, le réseau de neurones est un autre modèle paramétrique qui, sous sa forme la plus simple, consiste en un enchaînement de modèles linéaires séparés par de petites fonctions non-linéaires :

$$y = W \sigma(W \dots \sigma(Wx + b) \dots + b) + b$$

Cela peut paraître surprenant, mais cette nouvelle structure, appelée perceptron multi-couche (multi-layer perceptron ou MLP) est en fait incroyablement plus puissante que la régression linéaire. Mathématiquement, il s'agit d'un approximateur universel, autrement dit, un MLP peut approcher n'importe quelle fonction avec une précision arbitraire, si tant est qu'il contienne suffisamment de couches linéaires enchaînées.

Il y a bien sûr une contrepartie : dans la quasi-totalité des cas, il est impossible de trouver les valeurs optimales de W , b et le mieux que l'on puisse garantir est de trouver un minimum local de J . Cet aspect a longtemps fait fuir les chercheurs en intelligence artificielle,

jusqu'à ce que l'on s'aperçoive que ce minimum local était déjà incroyablement performant pour de nombreuses tâches.

2.4 - Petit passage par la biologie

Si les réseaux de neurones s'appellent ainsi, c'est effectivement parce qu'ils sont en quelque sorte reliés à ce que l'on connaît du fonctionnement d'un neurone biologique. Le perceptron (Franck Rosenblatt, 1958) est inspiré du fonctionnement du cerveau : un neurone reçoit des impulsions électriques x depuis ses axones, qu'il combine entre elles. Si le signal résultant W est supérieur à x un certain seuil - b , le neurone est activé (et émet à son tour une nouvelle impulsion électrique). Mathématiquement, cela se met sous la forme suivante :

$$y = \sigma(W + b) x$$

où σ est la fonction signe, renvoyant 0 si l'entrée est négative et 1 sinon. En 1969, Minsky et Papert assènent un coup terrible au perceptron en mettant en évidence les faiblesses théoriques qui rendent les perceptrons très peu efficaces (impossibilité de représenter une fonction OU EXCLUSIF). Cette découverte reléguera au placard le perceptron pour une petite décennie, jusqu'aux travaux de Rumelhart et Hinton, qui en 1986 proposent de combiner plusieurs perceptrons organisés en réseau pour former le MLP.

La véritable révolution surviendra au début des années 2010, avec des succès retentissants comme la reconnaissance d'images avec des modèles remplaçant les couches linéaires par des convolutions sur des images (comme AlexNet en 2012), ou la résolution du jeu de Go (avec AlphaGo en 2015). Aujourd'hui, les modèles véritablement utilisés en apprentissage profond n'ont plus grand-chose à voir avec le neurone biologique.

3 – Architecture d'un réseau de neurones

3.1 - Définition et structure générale

Comme expliqué précédemment, les réseaux de neurones permettent de modéliser des

phénomènes complexes. Ils sont basés sur l'enchaînement de modèles linéaires et de fonctions non linéaires. Les transformations $h = \sigma(W h + b)$ sont appelées couches. On distingue la couche d'entrée, qui reçoit le vecteur d'entrée x , la couche de sortie, qui renvoie un vecteur de sortie y (sans lui appliquer de fonction non linéaire σ), et les couches cachées, qui reçoivent et renvoient des vecteurs intermédiaires h_i . Ces vecteurs intermédiaires sont appelés représentations cachées (ou représentations latentes) et sont des variables internes au réseau de neurones. Les non-linéarités σ , quant à elles, sont appelées fonctions d'activation et nous les présenterons plus en détail dans la suite de cette partie. Il est intéressant de noter que les fonctions d'activation peuvent en théorie être différentes pour chaque couche (même si ce n'est généralement pas le cas).

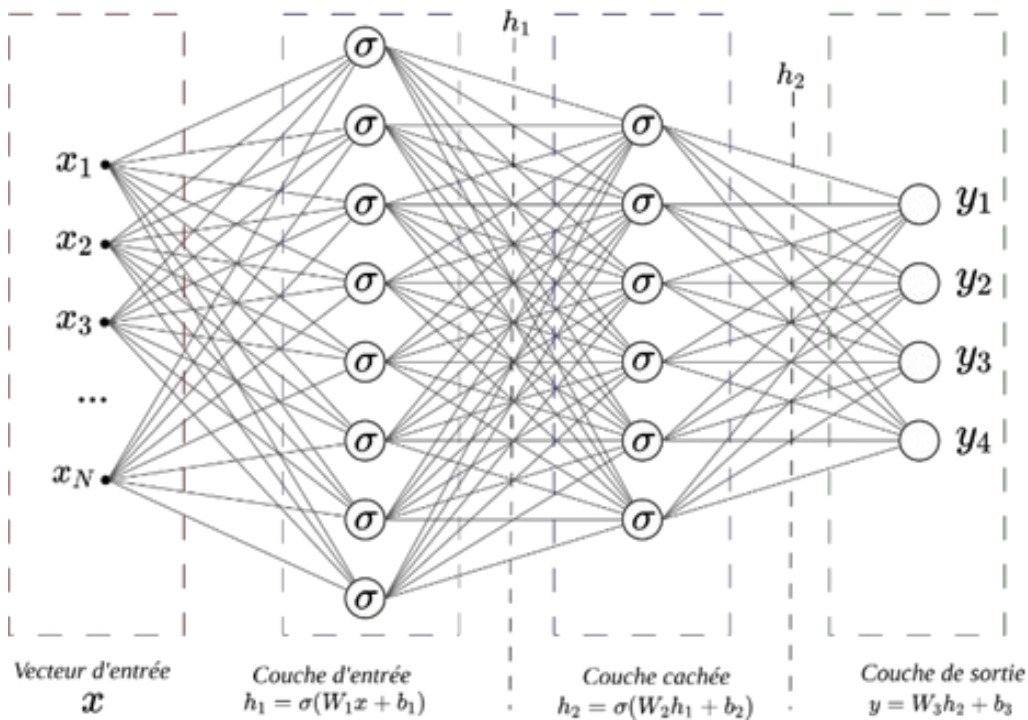
3.2 - Paramètres et hyperparamètres

Le réseau de neurones dispose d'un certain nombre de paramètres et hyperparamètres. Il est important de bien distinguer ces deux notions :

- Les paramètres sont des éléments variables du modèle qui définissent le comportement du modèle. Les paramètres changent au cours de l'apprentissage de manière à obtenir un modèle répondant à la tâche demandée. Dans le cas du MLP, les paramètres sont l'ensemble des poids W et des biais b .
- Les hyperparamètres sont des réglages du modèle et de l'algorithme d'apprentissage, choisis par le concepteur et qui ne sont pas modifiés par l'optimisation. Ils conditionnent eux aussi le comportement du modèle et doivent donc être choisis judicieusement. Dans le cas d'un MLP, les hyperparamètres sont (entre autres) le nombre de couches cachées, les dimensions des représentations cachées, les fonctions d'activations utilisées et le taux d'apprentissage.
- Il est important de bien choisir les hyperparamètres de notre modèle car ces derniers influent sur la phase d'apprentissage et sur les performances finales du modèle. Il s'agit là d'une des principales difficultés de l'utilisation de réseaux de neurones. Le concepteur doit avoir une idée au moins approximative de l'influence de chacun des hyperparamètres pour pouvoir les choisir correctement.

3.3 - Les fonctions d'activation

La fonction d'activation σ utilisée sur chacune des couches est une fonction non-linéaire qui s'applique sur chaque élément des vecteurs h_i . Il existe différentes fonctions d'activations utilisées classiquement, ayant chacune leurs avantages et leurs inconvénients. Nous n'allons en présenter ici que quelques-unes parmi les plus courantes.



- **Fonction ReLU (Rectified Linear Unit)** : Cette fonction est définie comme égale à 0 sur $\mathbb{R}^-$ et égale à l'identité sur $\mathbb{R}^+$. Il s'agit de la fonction d'activation la plus couramment utilisée dans le cas général. Il existe de nombreuses variantes de la fonction ReLU réalisant différentes approximations en 0 dans le but de la rendre dérivable. Cependant, la fonction ReLU présente l'avantage d'être linéaire par morceaux, donc rapide à calculer et présentant une dérivée simple.

- **Fonction Sigmoidale** : Cette fonction est définie par :

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Les valeurs renvoyées par cette fonction sont comprises entre 0 et 1. Cette fonction est souvent utilisée sur la dernière couche du réseau de neurones pour des problèmes de classification.

- **Fonction tangente hyperbolique** : Cette fonction est définie par :

$$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Elle renvoie des valeurs comprises entre -1 et 1. Elle présente principalement l'avantage de renvoyer des valeurs bornées et donc d'éviter les divergences lors des prédictions.

Une illustration de ces différentes fonctions est présentée figure 4.

Figure 4 : Évolution des différentes fonctions d'activations présentée. Les tracés sont faits pour des valeurs des neurones comprises entre -5 et 5 .

3.4 - Exemple applicatif : classification de chiffres manuscrits

Pour illustrer le fonctionnement d'un réseau de neurones, nous nous intéressons à la classification de chiffres manuscrits. 60 000 exemples peuvent être obtenus grâce à la base de données MNIST. Il s'agit ici d'apprentissage supervisé, on dispose d'exemples d'entrées x_i étiquetées avec les sorties correspondantes y_i souhaitées.

Il est possible de générer un vecteur d'entrée en prenant les pixels un à un. On obtient donc un vecteur d'entrée x de taille $28 \times 28 = 784$. La sortie y est constituée de 10 valeurs, chacune associée

à une classe possible comprise entre 0 et 9 (estimée par le réseau de neurones). La classe estimée correspond alors à celle pour laquelle la valeur est la plus élevée. On peut en quelque sorte voir cette valeur comme un indice de confiance qu'a le modèle en une classe : plus cette valeur est élevée (en comparaison aux autres), plus le MLP estime que cette classe est la bonne.

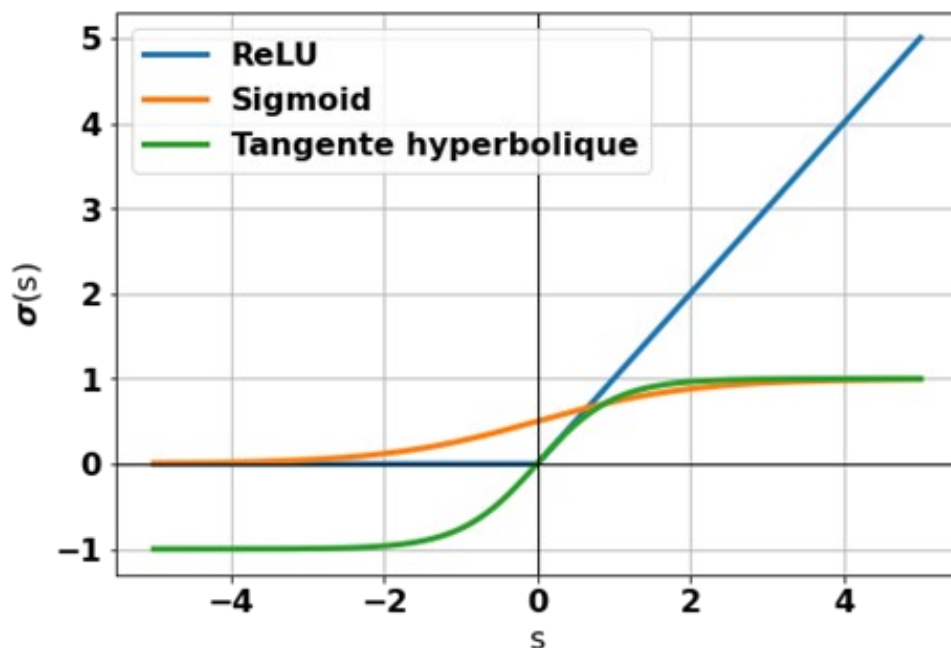


Figure 5 : Exemples d'éléments de la base de données
Les entrées sont des images de 28x28 pixels et les sorties sont des classes entre 0 et 9, correspondant au chiffre identifié

Avant de pouvoir entraîner le modèle (c'est-à-dire trouver les valeurs des paramètres), il nous faut choisir un certain nombre d'hyperparamètres comme le nombre de couches cachées, la taille de ces couches et la fonction d'activation à utiliser. Ce problème étant assez simple, il fonctionne assez bien avec une unique couche cachée de 200 neurones. On observe expérimentalement que la fonction d'activation ReLU permet dans ce cas d'avoir de bons résultats en limitant le nombre de calculs nécessaires et donc le temps d'apprentissage.

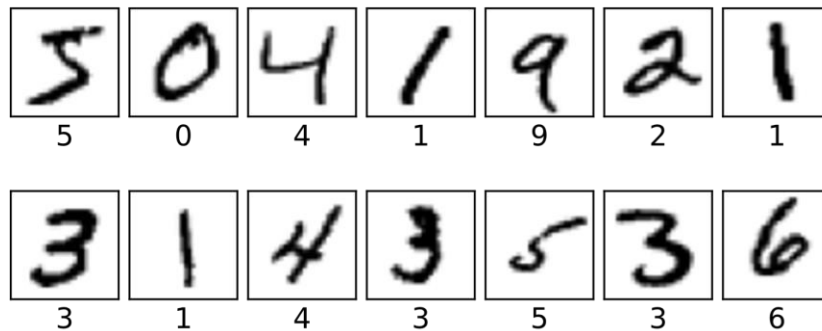
Cependant, nous avons expliqué précédemment que la couche de sortie est constituée de 10 valeurs qui sont comparées les unes aux autres afin de choisir une classe à prédire. Pourtant, les valeurs de la couche de sortie (dans le cas de la fonction d'activation ReLU) peuvent théoriquement prendre des valeurs infiniment grandes. Il devient alors difficile de comparer le vecteur de sortie du MLP avec un vecteur cible comme on le voudrait. On choisit alors de normaliser les valeurs de la couche de sortie entre 0 et 1 à l'aide d'une couche appelée Softmax.

La couche Softmax est une couche particulière qui ne traite pas les éléments du vecteur indépendamment. Le nouveau vecteur de sortie $Z = f(y)$ est défini par :

$$Z_i = \frac{\exp(y_i)}{\sum_{j=1}^N \exp(y_j)}$$

Cette fonction a pour objectif de ramener les valeurs de la couche de sortie entre 0 et 1 afin de les interpréter comme des probabilités (même si ce ne sont pas vraiment des probabilités, plutôt des taux de confiance du modèle).

On dispose donc d'un modèle prenant en entrée une image x de chiffres manuscrits et renvoyant un vecteur y de 10 valeurs permettant de prédire l'appartenance à une classe. On a également un grand nombre de données d'entraînement étiquetées qui serviront d'exemples pour entraîner le modèle (la classe d'un élément de la base d'apprentissage est convertie en un vecteur de taille 10 avec un 1 sur la classe réelle et neuf 0 partout ailleurs). À présent, il ne nous manque plus qu'à entraîner correctement les paramètres du réseau de neurones afin que ce dernier ait le comportement voulu et classe correctement nos images.



4 – Comment entraîner un réseau de neurones ?

4.1 - Minimiser un coût pour résoudre un problème

Dans l'exemple précédent, le réseau de neurones a pour but de classifier l'image qu'on lui fournit en entrée : autrement dit, la sortie souhaitée ne peut prendre que des valeurs discrètes. On parle alors de problème de classification. (Remarquez cependant que la sortie prédite se trouve sous la forme d'un vecteur y listant les « probabilités » pour chaque classe possible.) Lorsque la sortie du modèle prend des valeurs continues, comme dans le cas d'un réseau de neurones qui doit prédire le niveau de l'eau dans 5 ans d'une zone à risque, on parle de problème de régression.

Ces deux types de problèmes sont assez similaires dans leur résolution. Seule la fonction de coût change : cette fonction évalue quantitativement la distance entre la prédiction du modèle et la réponse attendue au cours de l'entraînement, et sa minimisation à l'aide d'algorithmes itératifs permet de guider la mise à jour des valeurs des paramètres, i.e. les poids W et les biais pour un i

MLP. En voici une liste non exhaustive :

- Pour la régression : on utilise souvent une fonction de coût quadratique, qui est l'erreur quadratique moyenne entre les sorties prédites $\hat{y}_i$ par le modèle pour les N données d'entraînement et les sorties réelles y_i correspondantes :

$$J = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

On peut aussi utiliser l'erreur absolue moyenne, ou encore la fonction de coût de Huber qui permet d'être moins sensible aux outliers (valeurs plus éloignées du modèle que les autres). Pour la classification : on utilise presque toujours la fonction d'entropie croisée. Dans le cas d'une classification binaire (i.e. lorsqu'il n'y a que deux classes possibles, 0 ou 1), la fonction de coût est calculée comme suit, où $\hat{y}_i$ représente donc la « probabilité » (estimée par le modèle) qu'a le i -ième échantillon d'être dans la classe 1:

$$J = - \frac{1}{N} \sum_{i=1}^N (y \ln(\hat{y}) + (1-y) \ln(1-\hat{y})) \quad \text{iii}$$

Si nous avons affaire à un problème de classification multi-classes, il suffit de calculer séparément l'entropie croisée pour chacune des classes à chaque donnée d'entraînement et d'en faire la somme. Mais parfois, on préfère convertir le problème en une succession de problèmes de classification binaire avant de fusionner leurs réponses. Deux stratégies existent : la première, celle du « un-contre-le-reste », consiste à apprendre autant de classifieurs qu'il y a de classes, et chaque modèle doit estimer la probabilité d'une classe contre celles de toutes les autres. La deuxième, qui est paradoxalement préférable lorsque le nombre de classes est grand, s'appuie sur du « un-contre-un » : un classifieur est appris pour chaque paire de classes, et la classe prédite est celle qui a remporté le plus de matchs en un contre un.

La phase d'entraînement d'un réseau de neurones est donc entièrement consacrée à la minimisation d'une fonction de coût, qui représente l'écart entre les prédictions du modèle pour les données d'entraînement et les sorties réelles.

4.2 - L'algorithme de descente du gradient

La fonction de coût ne sert pas seulement à dire au modèle s'il a bien fait son travail de prédiction. Elle permet surtout de guider l'algorithme de minimisation sur le terrain, en lui montrant comment ajuster les valeurs des paramètres à chaque itération pour tendre vers la minimisation de la fonction coût. En effet, lorsqu'un problème d'optimisation n'est pas soluble de manière déterministe, comme pour les réseaux de neurones qui représentent un problème trop complexe pour qu'on puisse calculer une solution analytique, il existe des algorithmes itératifs qui permettent de trouver une solution approchée, à condition toutefois que la fonction à minimiser soit dérivable, ce qui est le cas ici. Classiquement, on utilise l'algorithme de descente du gradient : voyons ce qu'il a sous le capot.

Visualisez-vous les yeux bandés sur le flanc d'une montagne : pour arriver en bas de la montagne, c'est-à-dire au point d'altitude minimale, vous allez évaluer du pied la pente à l'endroit où vous vous trouvez, puis vous ferez un pas dans la direction descendante, et vous recommencerez jusqu'à atteindre un creux, n'est-ce pas ? Mathématiquement, la montagne peut être vue comme la représentation graphique de la fonction coût, et le gradient d'une fonction en un point correspond grossièrement à la pente en ce point. Si le gradient est positif et élevé, cela signifie que la pente est grande et monte vers la droite, donc on peut faire un grand pas vers la gauche ; si le gradient est négatif et petit, la pente est faible et descend vers la droite, donc on fait un petit pas vers la droite (de peur de dépasser le minimum et de devoir revenir sur nos pas). Autrement dit, à chaque itération de l'algorithme, on met à jour chaque paramètre en lui retirant une valeur proportionnelle au gradient du coût en ce point (à une constante multiplicative près, appelée taux d'apprentissage η , fixée arbitrairement) :

$$W \leftarrow W - \eta \times \frac{\partial J}{\partial W_{jj}}$$

manière à mettre du gradient, on a donc :

en évidence l'approche itérative du calcul

$$\hat{y} = h = \sigma(W \cdot h_{M-1} + b_M)$$

$\frac{\partial J}{\partial W_{jj}}$

$\frac{\partial J}{\partial W_{jj}}$

$$b \leftarrow b - \eta \times \frac{\partial J}{\partial b_{jj}}$$

$\frac{\partial J}{\partial b_{jj}}$

$\frac{\partial J}{\partial b_{jj}}$

Il faut donc pouvoir calculer la valeur du gradient de la fonction coût par rapport à chaque paramètre. Pour cela, on réalise deux passages dans le réseau de neurones à chaque itération (ou epoch) :

- Une propagation de l'information vers l'avant (forward propagation) : on fournit les données d'entraînement au réseau en entrée pour obtenir les valeurs des prédictions $\hat{y}$ en sortie, en appliquant à chaque couche de neurones la formule suivante :
-

$$h_j = \sigma(W_{jj} h_{j-1} + b_j)$$

(h_0 étant donc ici la donnée d'entraînement x qu'on met en entrée du modèle, et le signal h_M final représente la sortie prédite $\hat{y}$).

- Une propagation de l'information vers l'arrière (backward propagation) : on calcule, à partir des prédictions $\hat{y}$, la valeur des gradients du coût par rapport aux poids W et biais b du réseau, dont les expressions sont obtenues par la règle de la chaîne en « remontant » les couches de neurones. Prenons l'exemple du coût quadratique :

$$J = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

où $\hat{y}_i = \sigma(W_M \cdot \sigma(W_{M-1} \dots + b_{M-1}) + b_M)$ est la sortie prédite pour la i-ième donnée d'entraînement et s'exprime en fonction des paramètres W et b . Reformulons $\hat{y}_i$ de

pour un MLP à M couches. Puisqu'on dérive des fonctions composées, les gradients du coût par rapport aux paramètres de la dernière couche s'expriment par :

$$\frac{\partial J}{\partial W_M} = \frac{\partial J}{\partial h_M} \frac{\partial h_M}{\partial W_M}$$

Et Il suffit à présent d'exprimer séparément chacune des dérivées :

$$\begin{aligned} \frac{\partial J}{\partial h_M} &= \sum_{i=1}^N (\hat{y}_i - y_i) \\ \frac{\partial h_M}{\partial W_M} &= \sigma'(W_{M-1} \cdot h_{M-1} + b_{M-1}) \\ \frac{\partial J}{\partial W_M} &= \frac{\partial J}{\partial h_M} \frac{\partial h_M}{\partial W_M} \\ &= \sum_{i=1}^N (\hat{y}_i - y_i) \sigma'(W_{M-1} \cdot h_{M-1} + b_{M-1}) \end{aligned}$$

Ainsi, la rétropropagation consiste surtout à calculer les $\frac{\partial J}{\partial h_j}$ successifs, puisque les autres $\frac{\partial J}{\partial h_j}$ gradients sont faciles à déduire. La formule de récurrence ci-dessous nous permet de le faire, en remontant les couches jusqu'à arriver au bout du réseau de neurones :

$$\frac{\partial J}{\partial h_{j-1}} = W \cdot \sigma'(W \cdot h + b) \frac{\partial J}{\partial h_j} \text{ avec } j \text{ allant de } 1 \text{ à } n$$

Une fois tous les gradients calculés, on peut mettre à jour les valeurs des paramètres. L'initialisation des paramètres se fait généralement en prenant une valeur choisie au hasard dans $[-1, 1]$, suivant une loi uniforme ou normale, même s'il existe d'autres stratégies plus sophistiquées (initialisation de Xavier, He...). Le critère d'arrêt se présente souvent sous la forme d'un nombre limité d'itérations afin de limiter la durée de la phase d'entraînement. Enfin, le choix du taux d'apprentissage peut se révéler délicat : un pas trop grand pourrait vous faire louper le minimum et empêcher l'algorithme de converger, mais un pas trop petit peut aussi être pénalisant en vous faisant avancer trop lentement... À vous de tester plusieurs valeurs pour que la fonction de coût converge proprement. Souvent, il est utile de commencer l'entraînement avec un taux suffisamment élevé pour atteindre rapidement des résultats satisfaisants, puis de baisser progressivement le pas afin d'affiner le modèle.

4.3 - En pratique, ça donne quoi ?

L'un des problèmes que présente l'algorithme de descente du gradient réside dans la possibilité qu'on reste bloqué dans un minimum local. Autrement dit, au lieu de parvenir en bas de la montagne, vous êtes malheureusement descendu dans une petite grotte et l'algorithme ne vous permet pas d'en sortir. Compte tenu des courbes d'erreurs très accidentées dessinées par les réseaux de neurones, il existe une multitude de minima locaux. De ce fait, l'apprentissage global converge rarement vers le minimum global de la fonction d'erreur lorsqu'on applique les algorithmes basés sur le gradient global. L'apprentissage avec l'algorithme de descente du gradient stochastique est une solution permettant de mieux explorer ces courbes d'erreurs : au lieu d'utiliser toutes les N données d'entraînement à chaque itération pour calculer les gradients, on ne va en prendre que quelques-uns, choisis au hasard, en mini-lots (ou mini-batches). Cela permet de plus d'éviter le stockage d'un trop grand nombre de données pour des fonctions ayant parfois quelques milliers de paramètres. Pour des réseaux de neurones plus petits, on peut se permettre de choisir des algorithmes d'optimisation plus coûteux en calculs, qui font intervenir des moments d'inertie (imaginez-vous à nouveau sur la montagne, le fait d'avoir un peu d'élan vous permettrait de sortir d'un creux ponctuel sur votre descente), ou des mises à jour automatiques du pas (si la pente est la même depuis un certain temps, vous pouvez vous permettre de faire des pas plus grands par exemple). Citons-en quelques-uns : Adagrad, Adam, RMSProp, Nesterov...

Certains problèmes récurrents peuvent empêcher le bon déroulement de la phase d'entraînement, notamment la disparition du gradient (vanishing gradient), ou son explosion (exploding gradient). Dans la première situation, les gradients deviennent de plus en plus petits lorsque l'algorithme progresse vers les couches inférieures du réseau, ce qui amenuise considérablement l'effet de la mise à jour des poids par descente de

gradient et empêche une convergence rapide. Fixer un taux d'apprentissage plus petit, ou choisir des fonctions d'activation σ qui ne sont pas nulles pour les valeurs négatives (comme la fonction Leaky ReLU, dont la dérivée est égale à 1 lorsque l'entrée est positive et à une petite constante lorsque l'entrée est négative). Dans la deuxième situation, les gradients deviennent de plus en plus grands, ce qui fait diverger l'algorithme : en effet, lorsqu'on a un réseau très profond, comme les gradients de chaque couche sont multipliés entre eux au cours de la backpropagation, on peut très rapidement avoir un gradient qui explose de manière exponentielle. Le problème peut être partiellement résolu en utilisant un taux d'apprentissage suffisamment faible, ainsi qu'en choisissant judicieusement les fonctions d'activation et la fonction de coût, ou en normalisant les données d'entraînement (i.e. en leur soustrayant leur moyenne et en les divisant par leur écart-type, et en gardant ces valeurs caractéristiques pour traiter les données de test de la même manière).

Enfin, notons l'existence du package Autograd dans Pytorch qui effectue automatiquement les calculs de gradients à partir de la fonction coût en collectant toutes les opérations de manière dynamique dans un graphe.

5 – L'apprentissage profond n'est pas l'Eldorado numérique

5.1 - De l'importance des données

Pour un industriel, les possibilités qu'offre l'apprentissage profond sont immenses : la clé de voûte, c'est une grande quantité de données sur le problème que l'on souhaite résoudre. Vous souhaitez détecter automatiquement la présence de tumeurs sur des images de scanner ? Les photos et les dossiers médicaux associés sont déjà là, dans la base de données de la clinique. Un algorithme pour savoir s'il faut accorder ou non un crédit à un client ? Les livres de compte de votre établissement bancaire feront l'affaire. Optimiser l'organisation de votre magasin pour vendre plus ? C'est facile, il suffit de jeter un œil aux tickets de caisse, facilement identifiés au profil du client grâce à sa carte de fidélité. Laissons de côté les fondements éthiques de telles pratiques (cela mériterait un article entier) et tempérons notre excitation : le deep learning n'a besoin que de données pour apprendre, il est vrai, mais encore faut-il que ces données soient propres et suffisamment nombreuses.

Le nettoyage des bases de données est un travail ennuyeux, ingrat, et pourtant absolument indispensable et terriblement chronophage. Il s'agit principalement de corriger toutes les petites anomalies qui pourraient gêner l'entraînement d'un réseau de neurones. Pour un ordinateur, «Dupon», «dupont» et «DUPONT» sont trois personnes différentes. Il s'agit également d'encoder les données en un format compréhensible pour le réseau, c'est-à-dire sous forme d'un vecteur mathématique. La question est vite réglée pour une image, qui n'est qu'une matrice de nombres, mais l'encodage du sexe d'une personne, par exemple, est déjà plus complexe. Nous avons vu dans l'exemple précédent que la solution classique consiste à passer par un vecteur one-hot rempli de zéro à l'exception de la ligne correspond à la classe. La tâche se complique très largement lorsque l'on souhaite travailler avec des données textuelles (avis sur un site de e-commerce, description des produits, etc.). Il y a trop de mots dans le dictionnaire pour imaginer passer par un vecteur one-hot. On utilise plutôt des algorithmes appelés word2vec qui associe un vecteur à valeurs réelles et de grande dimension à chaque mot (de sorte que, la somme du vecteur « roi » et « femme » est très proche du vecteur du mot « reine »).

Mais supposons que nos données soient déjà parfaitement propres : nos employés ont respecté à la lettre les règles de formatage, et celles-ci sont déjà stockées au format vectoriel. Reste encore un problème, et celui-ci est de taille : avons-nous assez de données pour entraîner un réseau de neurones ? Le jeu de données doit représenter avec suffisamment de finesse les variations de la fonction que l'on cherche à apprendre. C'est comme essayer de reproduire le plan d'une ville avec quelques photos aériennes : si vous n'avez pas assez de photos, la tâche est impossible. Il existe bien quelques techniques d'augmentation de données (symétrie des images, rotations, ...), mais en définitive, on ne peut pas faire de miracle.

On peut imaginer des variantes du problème de manque de données, dans lesquelles les données seraient riches sur un certain domaine, mais très pauvres ailleurs. L'entreprise Apple en a fait les frais en 2017. Plusieurs plaintes ont mis en évidence le fait que sa nouvelle application de déverrouillage par reconnaissance de visage était moins fiable sur les visages asiatiques que sur les visages européens. Évidemment, les développeurs de la multinationale n'ont pas volontairement codés un comportement discriminatoire dans leur application. Le problème venait en réalité des données d'entraînements, qui contiennent majoritairement des visages de personnes blanches de peau, originaires d'Amérique ou d'Europe. Le réseau de neurones n'ayant vu que très peu de visages asiatiques, celui-ci n'a pas été capable de généraliser ce qu'il avait appris sur les visages européens.

Plus facilement identifiables, les événements rares nécessitent une adaptation dans la manière de construire la fonction de coût pendant l'entraînement. Par exemple, lorsqu'une banque cherche à détecter les fraudes à la carte bancaire, celle-ci doit exploiter des bases de données contenant des millions de transactions licites, mais un nombre dérisoire de transactions officiellement détectées comme illégales. Ce déséquilibre peut par exemple être pris en compte en pondérant plus fortement les erreurs sur les événements rares dans la fonction de coût.

En définitive, tout est une question de généralisation : dans quelle mesure un réseau de neurones pourra prédire correctement sur des données différentes de celles vues lors de l'entraînement ? Cette question est centrale en apprentissage automatique.

5.2 - Sur et sous- apprentissage

La mise en place d'un réseau de neurones demande d'effectuer de nombreux choix arbitraires pour choisir les hyperparamètres (nombre de couches, forme du réseau, fonctions d'activation, etc.). Ceux-ci peuvent également impacter la capacité de généralisation du réseau. Le sous- apprentissage, par exemple, apparaît lorsque le réseau de neurones ne contient pas assez de paramètres pour apprendre la fonction (voir Fig.3d). On corrige cela en augmentant le nombre de couches et la taille des représentations cachées.

Mais même en apprentissage profond, le mieux est l'ennemi du bien. En augmentant le nombre de paramètres, on peut se retrouver dans la situation contraire du sur-apprentissage (voir Fig.6c). Le réseau de neurones exploite sa grande capacité pour apprendre par cœur les données d'entraînement, et devient incapable de généraliser ce qu'il a appris aux nouvelles mesures. Le sur-apprentissage (ou overfitting) est une épée de Damoclès permanente qui menace (et terrifie) tous ceux travaillant en deep learning. Un algorithme peut fonctionner parfaitement en laboratoire, sur les données d'entraînement, mais être parfaitement inutilisable en situation réelle ! Heureusement, il est possible de limiter les risques en respectant quelques règles simples. La première, et la plus importante de ces règles, nécessite de découper le dataset en trois ensembles de mesures totalement disjoints.

L'entraînement du réseau de neurones (la descente de gradient) à proprement parler s'effectue sur la totalité des données de l'ensemble d'entraînement. Il représente entre 60 et 80% des données. L'évaluation des performances du modèle entraîné est réalisée sur un deuxième ensemble, appelé ensemble de validation. C'est notamment ce dernier qui servira à régler les hyperparamètres du modèle. Insistons encore un peu sur le mot « disjoint » : comme les exemples de l'ensemble de validation n'ont jamais été vus par le réseau de neurones pendant l'entraînement, on pourra détecter le sur-apprentissage lorsque l'écart entre l'erreur d'entraînement et l'erreur de validation dépassera un seuil raisonnable. Cette séparation a valeur de loi en apprentissage profond : évaluer son modèle sur des données vues en entraînement est considéré comme de la triche et de la malhonnêteté !

Dans le cas idéal, on ajoute un troisième ensemble de test permettant d'évaluer la version finale du modèle, c'est-à-dire après avoir fixé tous les hyper-paramètres. En effet, on peut également sur-apprendre les hyper-paramètres du modèle, d'où cette nouvelle étape de sécurité.

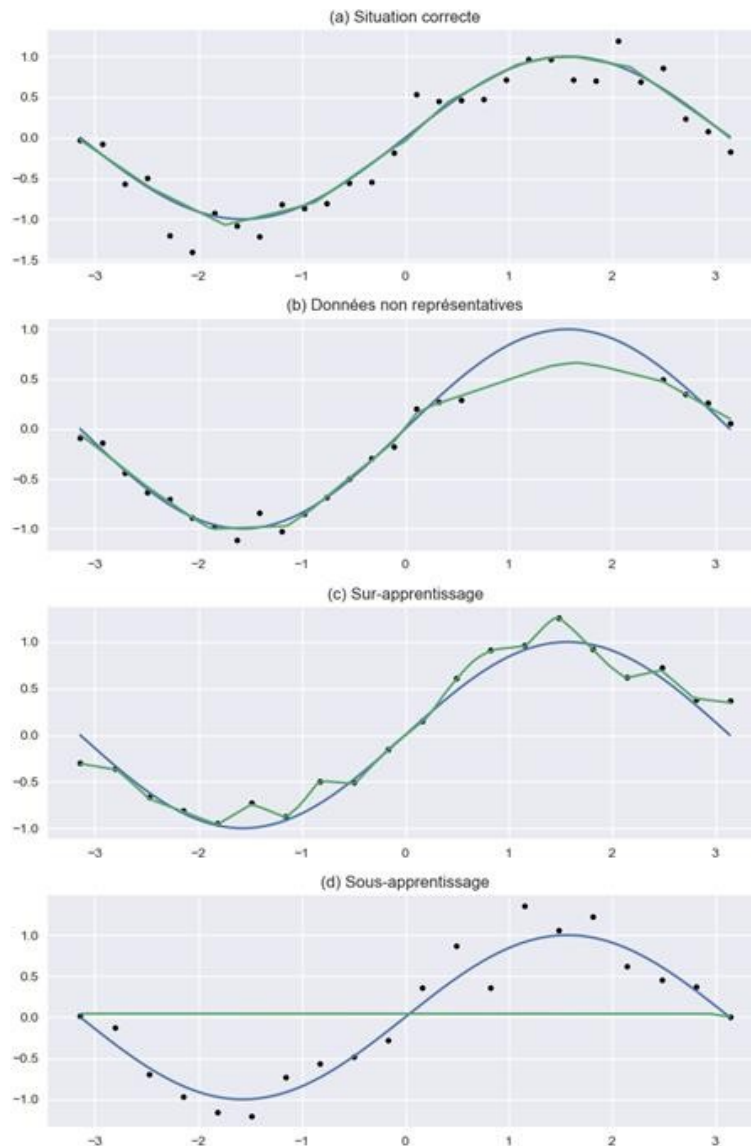


Figure 6 : Résultat d'entraînement d'un MLP à une couche cachée à partir des mesures (points noirs) issues d'une fonction sinus. (a) Les données représentent bien la fonction,

et le MLP est bien choisi. (b) Certaines données ont été supprimées, le réseau y est moins performant. (c) Le MLP est sur-dimensionné, celui-ci ne généralise pas. (d) Cette fois, le réseau est sous-dimensionné et ne parvient pas à apprendre quoi que ce soit.

5.3 - Mesures de performances

Félicitations ! Vous avez surmonté toutes les difficultés liées à vos données après plusieurs heures à les nettoyer. Vous avez soigneusement fabriqué vos trois ensembles train/validation/test et l'algorithme de descente de gradient semble finalement avoir convergé après plusieurs heures à monopoliser votre GPU. Voici venu le moment crucial : l'évaluation des performances du modèle. Cette étape vient (bien sûr) s'effectuer sur les données de test, mais la métrique utilisée va varier selon le type de problème. Certaines tâches ont d'ailleurs des métriques qui leur sont propres, comme la détection d'objets dans une image, les problèmes de suivi de mouvement, où encore les tâches de robotique. Loin de nous l'idée d'en dresser une liste exhaustive, étudions plutôt celles que vous serez le plus probablement amené à utiliser.

Pour les problèmes de régression, il n'y a pas grand-chose de nouveau : la solution simple utilise la distance euclidienne moyenne entre votre prédiction $\hat{y}$ et valeur attendue y . On la retrouve généralement sous l'appellation RMSE (pour root mean squared error) :

$$RMSE = \sqrt{\frac{1}{N} \sum (y - \hat{y})^2}$$

Pour les problèmes de classification, les choses se corsent. Pour l'instant, concentrons-nous sur un problème à deux classes (ce qui correspond à une classification binaire, comme un algorithme qui détecte la présence de défauts sur une chaîne de fabrication à partir des photos d'un composant : y a-t-il des défauts, oui ou non ?). La première idée, qui vient d'ailleurs assez naturellement, serait de compter le nombre de bonnes prédictions par rapport au nombre de prédictions total, autrement dit, la précision du modèle en langage courant. C'est effectivement une bonne mesure des performances, que l'on retrouve d'ailleurs quasi systématiquement. Mais cela ne suffit pas !

A priori, la plupart des composants de la chaîne de production sont de bonne facture (disons, 90%). Les données de test traduisent également de cette répartition et contiennent 90% de photos de « bon composant ». Comment dans ce cas interpréter les performances d'un réseau de neurones qui aurait appris à prédire « Bon » pour absolument tous les composants ? Sa précision serait de 90%, ce qui semble plutôt bon. Pourtant, il est évident que celui-ci est parfaitement inutile ! Il faut donc une autre métrique, et pour cela, il faut différencier les vrais positifs (les composants correctement identifiés comme défectueux) des faux positifs (les composants défectueux identifiés comme bons). On fait de même avec les vrais négatifs et les faux négatifs. Pour mesurer précisément les performances d'un algorithme de classification binaire, on introduit une matrice de confusion dans laquelle chaque ligne correspond à une classe réelle et chaque colonne, à une classe prédite. Ainsi, habituellement, la première ligne de la première colonne correspond au nombre de composants défectueux (positifs) identifiés comme tels, c'est-à-dire le nombre de vrais positifs.

Cette matrice peut se lire de plusieurs façons (cf. figure 7). La diagonale par exemple correspond à la précision dont nous discutons dans le paragraphe précédent. En lisant la

dernière ligne, on évalue la capacité du modèle à détecter les pièces valides. La première colonne nous donne une indication sur la fiabilité du réseau lorsque celui-ci prédit qu'un composant est défectueux. La matrice de confusion permet de repérer immédiatement le problème décrit précédemment : si le réseau donne toujours la même réponse, alors l'une des colonnes de la matrice sera entièrement nulle.

<i>ii i=0</i>

Le terme précision en français prête à confusion dans son emploi dans un problème de classification. On y préfère d'ailleurs le mot anglais accuracy ou encore exactitude en français, car la précision définit une autre métrique qui n'a pas tout à fait la même signification. La précision est en réalité le rapport entre le nombre de vrais positifs, divisé par le nombre d'éléments détectés comme positifs. Par exemple, si la précision de votre détecteur de défaut est de 70%, alors sur 100 pièces détectées comme à jeter, 30 d'entre elles étaient parfaitement fonctionnelles.

Une troisième métrique s'intéresse aux pièces défectueuses qui ne sont pas détectées par le réseau. Il s'agit du rappel. Ainsi, un rappel de 80% indique que sur 100 composants défectueux, seulement 80 ont été correctement identifiés. Le jonglage entre précision et rappel est une gymnastique qui demande un peu de pratique pour être parfaitement à l'aise. En résumé, la précision juge le nombre de composants jetés pour rien, et le rappel le nombre de mauvais composants qui ont été ignorés. Ces métriques s'étendent aux problèmes de classification dans plusieurs catégories. On calcule dans ce cas la précision, l'accuracy et le rappel pour chaque classe.

6 – Le Deep Learning en pratique

6.1 - Structures des réseaux

Théoriquement, le MLP est à lui seul suffisamment puissant pour approcher n'importe quelle fonction que l'on souhaite apprendre. En pratique, il y a plusieurs prérogatives à cela : nous avons déjà évoqué l'importance des données, et l'influence des hyperparamètres. Quand bien même toutes ces conditions seraient réunies, il n'y a, en fait, aucune garantie pour que la descente de gradient parvienne à trouver les meilleures valeurs des poids possibles, et si possible dans un délai raisonnable. On doit donc se résoudre à inventer des structures de réseau de neurones qui facilitent l'apprentissage sur certaines tâches. C'est ce qu'on appelle la *bitter lesson* en apprentissage profond : tous les efforts que nous mettons en œuvre aujourd'hui pour inventer de nouvelles structures de modèles profond est vain, car l'évolution de domaine et l'augmentation des puissances de calculs des ordinateurs auront permis l'émergence de nouvelles méthodes plus simples qui résoudront le problème avec au moins autant de succès.

		Classe prédite	
		Défaut	OK
Classe réelle	Défaut	Vrais positifs	Faux négatif
	OK	Faux positifs	Vrais négatifs

$$\text{Accuracy} : \frac{\text{Vrais positifs} + \text{Vrais négatifs}}{\text{Vrais positifs} + \text{Faux négatifs} + \text{Faux positifs} + \text{Vrais négatifs}}$$

$$\text{Précision} : \frac{\text{Vrais positifs}}{\text{Vrais positifs} + \text{Faux positifs}}$$

$$\text{Rappel} : \frac{\text{Vrais positifs}}{\text{Vrais positifs} + \text{Faux négatifs}}$$

Mais aujourd'hui, le MLP ne suffit pas ! Il est cependant toujours utilisé comme brique élémentaire de construction pour des modèles plus complexes. Éloignons-nous donc maintenant des principes introductifs pour nous intéresser aux pratiques plus modernes du deep learning. Voici un petit bestiaire des modèles classiques :

Les **réseaux convolutionnels** (CNN) (ou réseau de neurones convolutifs) sont bien plus adaptés au traitement de l'image que ne l'est le MLP. Pour cela, les transformations linéaires du MLP sont remplacées par des convolutions 2D, qui ont l'avantage d'utiliser moins de paramètres, et surtout d'être invariants aux translations (déplacer un objet dans l'image ne change pas drastiquement les calculs du réseau de neurone).

Figure 8 : Exemple d'une convolution 2D entre une matrice d'entrée x et un noyau de convolution k . Les seuls paramètres à calculer sont les poids du noyau de convolution

Les réseaux convolutionnels sont à l'origine du regain d'intérêt pour les réseaux de neurones au début des années 2010 en surpassant de très loin les performances des méthodes de vision par ordinateur plus classiques. La pratique courante consiste à appliquer une série de couches de convolution (avec leurs fonctions d'activation) sur une image, puis d'aplatir le résultat final en un long vecteur qui passe ensuite dans un MLP qui prédit alors la grandeur d'intérêt. Il existe également quelques utilisations de réseaux convolutionnels en 1D et 3D (pour l'audio et la vidéo par exemple).

Les réseaux récurrents (RNN) sont adaptés à l'analyse de séries temporelles, en ajoutant une mémoire au réseau. Chaque donnée x_t de la séquence est utilisée pour mettre à jour un vecteur caché h_t .

$$h = \sigma(Wh + Wx + b)_{t+1} h_t x_t$$

Ce vecteur caché peut être utilisé de plusieurs façons : pour une tâche de classification (par exemple, prédire si une action va prendre de la valeur ou non), on peut faire passer la dernière valeur de h_t dans un MLP. Ce principe est particulièrement utilisé par le traitement de flux vidéo et le traitement de l'écriture manuscrite. L'équation précédente est un peu passée de mode, et présente un mauvais comportement pendant la descente de gradient (le gradient a tendance à s'évanouir, c'est-à-dire valoir zéro, lorsque la séquence est longue). Il existe des structures plus modernes comme les Long Short-Term Memory (LSTM) ou plus récemment les réseaux Gated Recurrent Unit (GRU).

Les **auto-encodeurs** (AE) sont très pratiques dans beaucoup d'applications. Une donnée de grande dimension (généralement une image) est encodée par un réseau de neurones dans un vecteur contenant largement moins de valeurs, qui est ensuite décodé pour reconstruire la donnée d'origine. Il s'agit d'un algorithme de compression très efficace, qui peut être détourné pour, par exemple, encoder une image et décoder l'image en profondeur correspondante. Cette structure en entonnoir est présente dans beaucoup de modèles profonds.

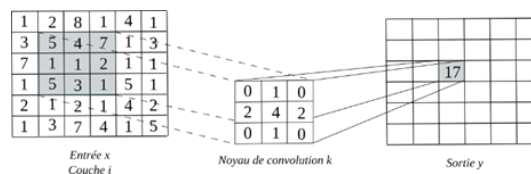


Figure 9 : Aucun de ces visages n'existent dans la vraie vie, ils ont été générés par un GAN, un modèle à deux réseaux : un générateur d'images, et un discriminateur chargé de distinguer les vrais visages des faux

Les **réseaux adversariaux génératif** (GAN) sont une idée astucieuse lorsque la tâche consiste à générer des données, comme des visages humains ou de la musique. Le principe consiste à entraîner deux réseaux de neurones antagonistes : le générateur fabrique des images à partir d'un vecteur d'entrée aléatoire et le discriminateur est entraîné à différencier une vraie donnée d'une donnée fabriquée par le générateur. Au début, la tâche est facile pour le discriminateur, mais se complique lorsque le générateur s'améliore. Les deux réseaux sont entraînés en simultané. Certains auto-encodeurs peuvent d'ailleurs être entraînés comme des GAN.

Tout récemment, une nouvelle structure a fait son apparition en révolutionnant les méthodes de traitement du langage : il s'agit des **transformers** (nous vous éviterons le jeu de mot avec les films de Michael Bay). Sans rentrer dans les détails, ces modèles reposent sur un mécanisme d'attention entre les mots d'une phrase. Ils sont à l'origine des immenses progrès des services de traduction en ligne, mais aussi de certains assistants virtuels.

La liste pourrait s'allonger indéfiniment, car c'est là l'une des libertés de l'apprentissage profond : toutes les structures sont bonnes à être testées. Le gros de la recherche se

concentre donc sur la fabrication de modèles spécifiquement dédiés à une tâche. Nous pourrions mentionner par exemple les réseaux en graphe (GNN) qui excellent dans tous les problèmes pouvant se mettre sous cette forme particulière.

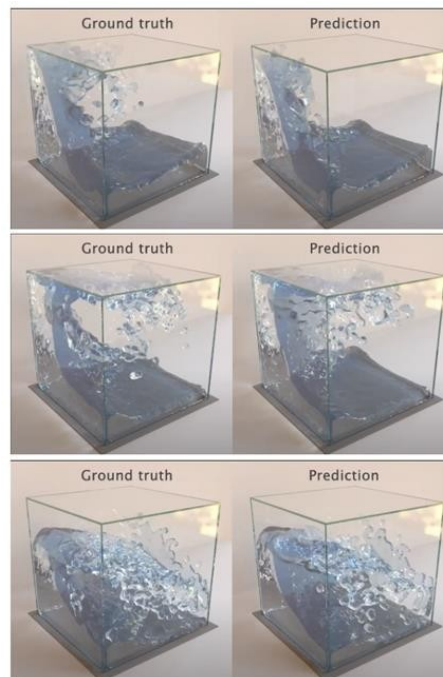


Figure 10 : Les mouvements du fluide prédit par un réseau de neurones en graphe, qui modélise le fluide par un ensemble de particules reliés entre eux, source [1].

6.2 - Implémentation pratique

Il faut différencier la phase d'apprentissage d'un réseau de neurones de la phase d'inférence, c'est-à-dire le moment où celui-ci est réellement utilisé. L'entraînement est une étape très longue, en grande partie à cause de la descente de gradient, qui nécessite de calculer la dérivée du critère d'optimisation par rapport à un très grand nombre de paramètres. L'avantage, c'est qu'il s'agit d'une étape qui se parallélise très bien, d'où l'utilisation massive des cartes graphiques. Architecturalement, une carte graphique est composée de milliers de cœurs très simples, là où un processeur n'est composé que d'une dizaine de cœurs en moyenne, mais bien plus puissants. Le GPU permet donc d'effectuer beaucoup plus d'opérations en parallèle qu'un CPU. L'inférence, par contre, est très rapide, puisqu'il s'agit majoritairement de multiplications matricielles, ce qui rend possible l'utilisation de réseaux de neurones en temps réel, dans des véhicules autonomes par exemple.

Dans cette partie, nous allons voir comment implémenter un MLP. Nous nous limiterons ici au langage Python, qui dispose de nombreuses bibliothèques facilitant la mise en œuvre de réseaux de neurones. À vrai dire, la quasi-totalité des implémentations de réseaux de neurones se fait avec ce langage. Les bibliothèques les plus couramment utilisées pour l'implémentation de réseaux de neurones sont PyTorch (maintenu par Meta), Tensorflow (maintenu par Google). Ces deux bibliothèques diffèrent quelque peu dans leur manière de définir un réseau de neurones, mais s'appuient sur la même fonction de base : autograd. Il s'agit d'une bibliothèque permettant de dériver n'importe quelle fonction codée en Python (pour peu que l'on respecte certaines règles). Sans autograd, le deep learning ne connaîtrait pas le succès qu'on lui connaît. Ces deux bibliothèques sont gratuites et libres, et extrêmement bien documentées en ligne.

Pour cet exemple, nous allons plutôt utiliser Scikit-learn, une bibliothèque qui regroupe des centaines de fonctions liées au machine learning. On y retrouve bien sûr le MLP, que nous allons utiliser, mais aussi l'algorithme des k-plus-proches voisins, random forest, etc... Son utilisation est extrêmement simplifiée et elle est donc idéale pour comprendre le fonctionnement des algorithmes avec de petits modèles. Cependant, pour des problèmes plus complexes, l'utilisation de bibliothèques spécialisées et optimisées comme PyTorch ou Tensorflow est obligatoire.

Dans un premier temps, on cherche à importer les données qui serviront à entraîner le réseau de neurones. Nous allons garder les chiffres manuscrits de la base de données MNIST mentionnées précédemment.

Ces quelques lignes téléchargent automatiquement les données de MNIST et stockent les entrées X et les sorties y dans les variables correspondantes. On peut ensuite visualiser une image. Notons que les images des chiffres manuscrits sont redimensionnées en 28×28 pixels :

*Figure 11 : Image extraite de la base de données MNIST. Il s'agit d'une image 28*28 pixels, pouvant également être représentée comme un vecteur "plat" de 784 éléments*

On doit maintenant séparer les données de notre dataset en deux sous-ensembles : test et train, pour éviter tout risque de sur-apprentissage. Scikit-learn propose une fonction pour automatiser ce découpage. Notez que nous n'utilisons pas d'ensemble de validation dans cet exemple, ce n'est pas nécessaire dans la mesure où le problème est très simple. On sélectionne ici 20% des données pour constituer l'ensemble de test.

L'argument « shuffle » permet de mélanger les données afin de bien répartir équitablement chaque classe dans les deux ensembles. Cela permet de limiter le biais lié à la structure de la base de données (Par exemple, tous les 9 pourraient être à la suite de tous les 8, eux-mêmes à la suite de tous les 7 etc.).

```
from sklearn.datasets import fetch_openml

import numpy as np

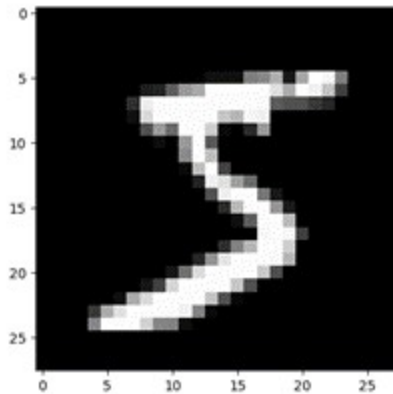
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)

import matplotlib.pyplot as plt

plt.gray()
```

```
plt.imshow(np.reshape(np.array(X)[0,:],(28,28)))
```

```
plt.show()
```



Une bonne pratique consiste à normaliser les entrées du réseau de neurones, cela a tendance à accélérer un peu l'apprentissage. On divise donc les images par 255. Les données sont maintenant prêtes, on peut créer le MLP et l'entraîner en lui fournissant les données connues. Plusieurs hyperparamètres sont précisés ici :

- Il y a 1 couche cachée de 50 neurones. On observe expérimentalement que cela suffit pour la reconnaissance de chiffres manuscrits ;
- On réalise au maximum 10 passages sur l'ensemble de la base de données (epoch) d'entraînement. Cela est suffisant pour obtenir de bons résultats dans notre cas mais peut être bien plus élevé pour des applications plus complexes ou si plus de paramètres sont à déterminer ;
- L'algorithme d'apprentissage utilisé est une descente de gradient stochastique.

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
X = X / 255.
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True,
```

```
test_size=0.2)
```

```
# Définition du modèle

mlp = MLPClassifier(hidden_layer_sizes=(50, ), max_iter=10, solver='sgd',
verbose=10, learning_rate_init=.1)

# Entraînement

model_final = mlp.fit(X_train, y_train)

# Calcul des précisions

print(f"Précision sur les données d'entraînement : {mlp.score(X_train,
y_train):.3f}")

print(f"Précision sur les données de test : {mlp.score(X_test,
y_test):.3f}")
```

Enfin, on peut visualiser une des prédictions :

Le code de cet exemple de classification des chiffres manuscrits avec la bibliothèque Scikit-learn, ainsi que le code complet de l'apprentissage sans bibliothèque, sont disponibles sur le dépôt : https://github.com/LudovicDeMatteis/Revue3EI_IA .

7 – Conclusion

Le deep learning est un domaine en pleine expansion qui permet d'obtenir des performances très élevées dans de nombreuses tâches, notamment la reconnaissance d'images et de parole. Il repose sur des architectures de réseaux de neurones particulièrement complexes, appelés réseaux profonds. Bien qu'il soit extrêmement performant, le deep learning n'est pas exempt de difficultés et il est important de bien comprendre les principes de base avant d'en faire un usage aveugle. En particulier, il est important de disposer d'un jeu de données suffisamment important et représentatif pour pouvoir entraîner un réseau de neurones de manière efficace. Son apparition dans les programmes de l'éducation nationale permettra sans doute aux élèves et aux étudiants de mieux comprendre les principes fondamentaux du deep learning et plus généralement du machine learning, un sujet porteur qui interviendra nécessairement dans la carrière des futurs ingénieurs et techniciens. Il s'agit néanmoins d'une discipline jeune et en plein

développement, il est important de suivre les évolutions en cours pour pouvoir tirer le meilleur parti des outils et des méthodes qui sont mis à disposition.

```
exemple_test_x = np.array(X_test[:1])
exemple_test_y = np.array(y_test[:1])

plt.gray()

plt.imshow(exemple_test_x.reshape((28,28)))

plt.show()

print(f"Prediction : {mlp.predict(exemple_test_x)}")

print(f"Sortie : {exemple_test_y}")
```

Cette ressource est une brève introduction aux réseaux de neurones, mais leur champ d'application grandit un peu plus chaque jour, offrant son lot de nouvelles applications et de nouveaux challenges à relever ! Pour être honnête, ces derniers mots ne sont pas les nôtres, mais en réalité ceux d'une IA, qui a proposé ce paragraphe à partir de quelques extraits de l'article que vous venez de lire. Il s'agit de GPT3, un modèle basé sur les transformers pour le traitement du langage naturel.

Références :

[1]: Figure 10 issue de “Learning to simulate complex physics with graph networks”, de Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec et Peter Battaglia.

[2]: “L'apprentissage profond”, Cours de Yann Le Cun au Collège de France,
<https://www.college-de-france.fr/site/yann-lecun/course-2015-2016.htm>

[3]: “The bitter lesson”, de Richard Sutton,

<http://www.incompleteideas.net/IncIdeas/BitterLesson.html>

[4]: “Deep Learning” de Ian Goodfellow, Yoshua Bengio et Aaron Courville, éditions MIT press [5]: TensorFlow Playground : <https://playground.tensorflow.org/>

[6]: Personalized Human-Swarm Interaction Through Hand Motion, M. Macchini, L. de Matteis, F. Schiano, D. Floreano, mars 2021, <https://arxiv.org/abs/2103.07731>

[7]: Dossier Intelligence Artificielle, juin 2022, https://eduscol.education.fr/sti/si-ens-paris-saclay/ressources_pedagogiques/dossier-intelligence-artificielle

[8]: Introduction à l'apprentissage automatique, L. de Mateïs, S. Nathan, juin 2022,

https://eduscol.education.fr/sti/si-ens-paris-saclay/ressources_pedagogiques/introduction-lapprentissage-automatique

Table des matières

Généralités et présentation du loto français.....	1
Collecte des données et calcul de features.....	2
Modèle.....	3
Formation du réseau.....	4
Optimiseur.....	4
Entraînement.....	5
Résultats d'apprentissage sur le tirage du 21 octobre 2020.....	6
Résultats de prédiction pour le tirage du 21 octobre 2020.....	7
Résultats de prédiction pour le tirage du 24 octobre 2020.....	8
Conclusion.....	8
La cellule LSTM: Long Short Term Memory.....	9
Couches de LSTM.....	11
Aller plus loin.....	13
Généralisation de la rétropropagation du gradient.....	14
Mod 2. La méthode de descente du gradient.....	31

Mod 3. La rétropropagation.....	35
▪ Démarrage.....	36
▪ Cascadage.....	37
▪ Traiter la couche courante.....	37
▪ Exemple.....	38
Mod 4. L'algorithme d'apprentissage.....	40
Mod 5. Réseaux de neurones – Introduction.....	42
Mod 5. Les réseaux convolutionnels.....	47
▪ Principe.....	47
▪ Les noyaux de taille complexe.....	49
▪ Terminologie.....	51
▪ Interprétation des features.....	51
▪ Objectif.....	52
Mod 6. Les optimiseurs.....	56
▪ Gradient Descent.....	56
▪ Stochastic Gradient Descent (SGD).....	57
▪ Mini-Batch Gradient Descent.....	57
▪ Momentum.....	58
▪ RMSProp.....	59
▪ Adadelta.....	59
▪ Adam.....	60
▪ Définition.....	61
Propriétés.....	62
▪ Remarque.....	63
Mod 8. Les fonctions d'erreur.....	66
▪ MeanSquaredError.....	66
▪ Keras.....	67
▪ Présentation.....	67
▪ CrossEntropyLoss.....	69
▪ Comportement.....	70
▪ Keras.....	70
• Les métriques.....	71
▪ Logit.....	71
▪ Keras.....	72
▪ Compilation.....	74
▪ Erreur d'exécution.....	75
▪ Définition.....	75
▪ Stockage dynamique des variables/fonctions.....	76
▪ Import de librairies.....	76
▪ Les conditions.....	78
▪ Les boucles.....	78
▪ Les listes.....	79
▪ Les tuples.....	79
▪ Les dictionnaires.....	81
▪ Boucle sur les containers.....	81
▪ Conflit entre variables globales et locales.....	82
▪ Le tout objet.....	83
▪ Passage d'arguments.....	83
▪ Modifier un entier passé en argument.....	84
▪ Clonage.....	85

Merci pour votre attention et bonne année scolaire

INTRODUCTION

Dans ce chapitre d'introduction, nous allons présenter un premier réseau neuronal appelé le Perceptron. Ce modèle est un réseau neuronal constitué d'un seul neurone, et nous l'utiliserons ici pour introduire des concepts-clés que nous détaillerons plus tard dans le cours.

1.1 Un premier modèle : le perceptron

Dans la terminologie des réseaux de neurones, un neurone est une fonction paramétrée qui prend un vecteur x en entrée et sort une valeur unique a comme suit :

$$a = \varphi(\underbrace{w}_w x + b), o$$

où les paramètres du neurone sont ses poids stockés dans w , et un terme de biais b , et φ est une fonction d'activation qui est choisie a priori (nous y reviendrons plus en détail plus tard dans le cours) :

$$x_0 x_1 x_2 x_3 \dots x_n \quad w_0 w_1 w_2 \dots w_n \quad \varphi \quad a_2 \quad x_3 + 1$$

Un modèle constitué d'un seul neurone est appelé perceptron.

$$w_0 w_1 w_2 \dots w_n - b$$

1.2 Optimisation

Les modèles présentés dans ce document ont pour but de résoudre des problèmes de prédiction dans lesquels l'objectif est de trouver des valeurs de paramètres « suffisamment bonnes » pour le modèle en jeu compte tenu de données observées.

Le problème de la recherche de telles valeurs de paramètres est appelé optimisation. L'apprentissage profond (ou deep learning) fait un usage intensif d'une famille spécifique de stratégies d'optimisation appelée descente gradiente.

1.2.1 Descente de gradient

Pour se faire une idée de la descente de gradient, supposons que l'on nous donne le jeu de données suivant sur les prix de l'immobilier :

```
import pandas as pd
boston = pd.read_csv("../data/boston.csv")[["RM", "PRICE"]]
boston
```

RM PRICE

1. 0 6.575 24.0

2. 1 6.421 21.6

3. 2 7.185 34.7

4. 3 6.998 33.4

5. 4 7.147 36.2

..

501. 501 6.593 22.4

```
502.  
503. 502  6.120  20.6  
504.  
505. 503  6.976  23.9  
506.  
507. 504  6.794  22.0  
508.  
509. 505  6.030  11.9  
510.
```

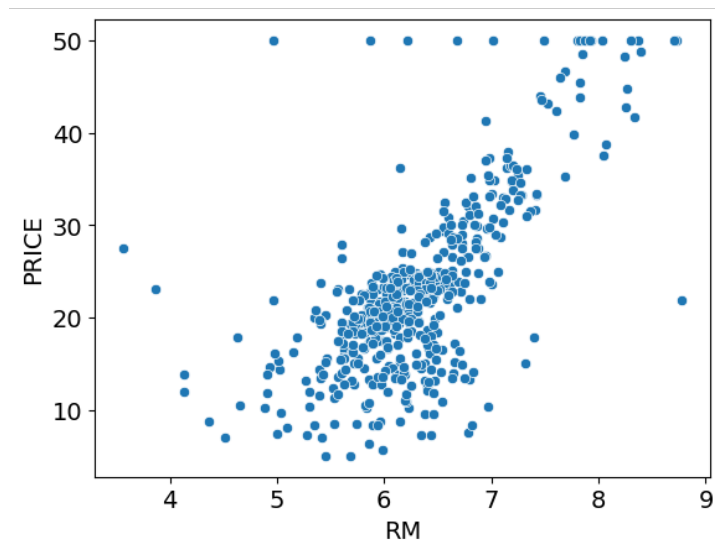
```
[506 rows x 2 columns]
```

Dans notre cas, nous essaierons (pour commencer) de prédire la valeur cible "PRICE" de ce jeu de données, qui est la valeur médiane des maisons occupées par leur propriétaire en milliers de dollars en fonction du nombre moyen de pièces par logement "RM" :

```
sns.scatterplot(data=boston, x="RM", y="PRICE");
```

Chapitre 1.

Introduction :



Une courte note sur ce modèle .

Dans la terminologie du Perceptron, ce modèle :

- n'a pas de fonction d'activation (i.e. φ est la fonction d'identité)
- n'a pas de biais (i.e. b est fixé à 0, il n'est pas appris)

Supposons que nous ayons une approche naïve dans laquelle notre modèle de prédiction est linéaire sans biais, c'est-à-dire que pour une entrée donnée x_i la sortie prédite est calculée comme suit :

$$\hat{y} = wx_{ii}$$

où w est le seul paramètre de notre modèle.

Supposons en outre que la quantité que nous cherchons à minimiser (notre objectif, également appelé fonction de perte) est :

$$L(w) = \sum_i (\hat{y} - y)^2_{ii}$$

où y_i est la valeur cible associée au i -ème échantillon de jeu de données.

Examinons cette quantité en fonction de w :

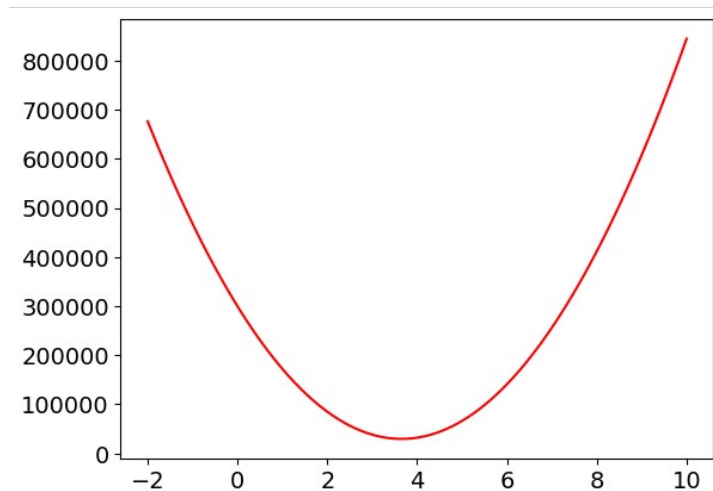
```
import numpy as np

def loss(w, x, y): w = np.array(w)

return np.sum(
(w[:, None] * x.to_numpy()[None, :] - y.to_numpy()[None, :]) ** 2, axis=1
)

w = np.linspace(-2, 10, num=100)
x = boston["RM"]
y = boston["PRICE"]
```

```
plt.plot(w, loss(w, x, y), "r-");
```



Ici, il semble qu'une valeur de w autour de 4 devrait être un bon choix. Cette méthode (générer de nombreuses valeurs pour le paramètre et calculer la perte pour chaque valeur) ne peut pas s'adapter aux modèles qui ont beaucoup de paramètres, donc nous allons donc essayer autre chose.

Supposons que nous ayons accès, à chaque fois que nous choisissons une valeur candidate pour w , à la fois à la perte L et aux informations sur la façon dont L varie, localement. Nous pourrions, dans ce cas, calculer une nouvelle valeur candidate pour w en nous déplaçant à partir de la valeur candidate précédente dans la direction de la descente la plus raide.

C'est l'idée de base de l'algorithme de descente du gradient qui, à partir d'un candidat initial w_0 , calcule itérativement de nouveaux candidats comme :

$$w_{t+1} = w_t - \rho \left. \frac{\partial L}{\partial w} \right|_{w=w_t}$$

où ρ est un hyper-paramètre (appelé taux d'apprentissage) qui contrôle la taille des pas à effectuer, et $\left. \frac{\partial L}{\partial w} \right|_{w=w_t}$ est le

gradient de L par rapport à w , évalué en $w = w_t$. Comme vous pouvez le voir, la direction de la descente la plus raide est l'opposé de la direction indiquée par le gradient (et cela vaut aussi pour les paramètres vectoriels).

Ce processus est répété jusqu'à la convergence, comme l'illustre la figure suivante :

```
rho = 1e-5

def grad_loss(w_t, x, y): return np.sum(
    2 * (w_t * x - y) * x
)
ww = np.linspace(-2, 10, num=100)
plt.plot(ww, loss(ww, x, y), "r-", alpha=.5);

w = [0.]
for t in range(10):
```

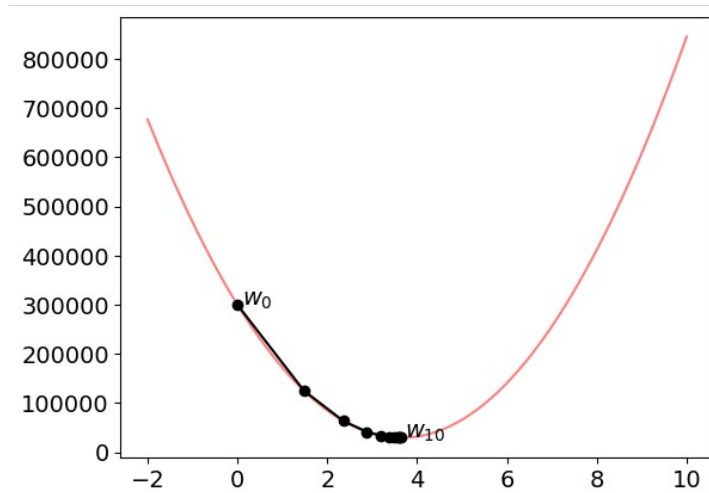


```

w_update = w[t] - rho * grad_loss(w[t], x, y)
w.append(w_update)

plt.plot(w, loss(w, x, y), "ko-")
plt.text(x=w[0]+.1, y=loss([w[0]], x, y), s="$w_{0}$") plt.text(x=w[10]+.1, y=loss([w[10]], x, y), s="$w_{10}$");

```



Qu'obtiendrions-nous si nous utilisons un taux d'apprentissage plus faible ?

```

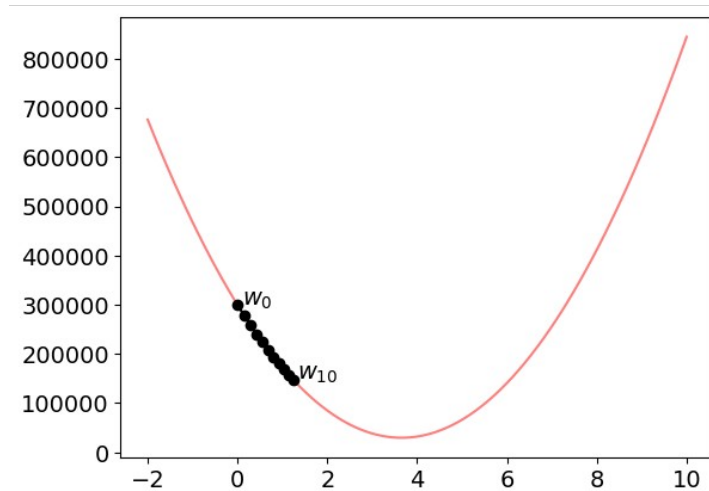
rho = 1e-6
ww = np.linspace(-2, 10, num=100)
plt.plot(ww, loss(ww, x, y), "r-", alpha=.5);

w = [0.]
for t in range(10):

    w_update = w[t] - rho * grad_loss(w[t], x, y)
    w.append(w_update)

plt.plot(w, loss(w, x, y), "ko-")
plt.text(x=w[0]+.1, y=loss([w[0]], x, y), s="$w_{0}$") plt.text(x=w[10]+.1, y=loss([w[10]], x, y), s="$w_{10}$");

```



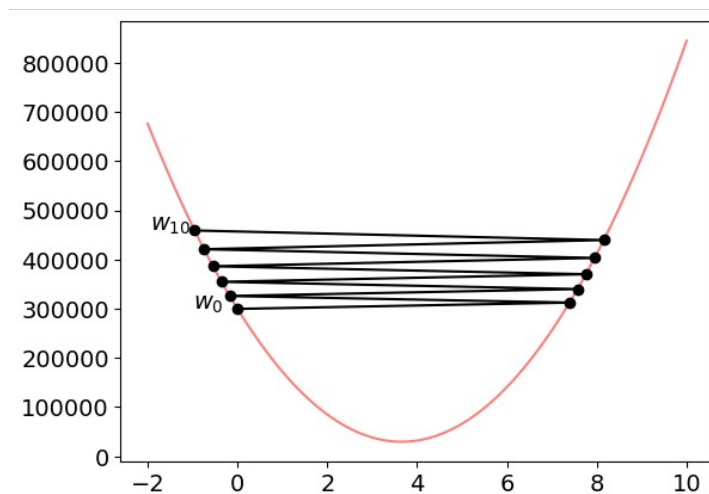
Cela prendrait certainement plus de temps pour converger. Mais attention, un taux d'apprentissage plus élevé n'est pas toujours une bonne idée :

```
rho = 5e-5
ww = np.linspace(-2, 10, num=100)
plt.plot(ww, loss(ww, x, y), "r-", alpha=.5);

w = [0.]
for t in range(10):

    w_update = w[t] - rho * grad_loss(w[t], x, y)
    w.append(w_update)

plt.plot(w, loss(w, x, y), "ko-")
plt.text(x=w[0]-1., y=loss([w[0]], x, y), s="$w_{0}$") plt.text(x=w[10]-1., y=loss([w[10]], x, y), s="$w_{10}$");
```



Vous voyez comment nous divergeons lentement parce que nos pas sont trop grands ?

1.3 Récapitulatif

Dans cette section, nous avons introduit :

un modèle très simple, appelé le Perceptron : ce sera une brique de base pour les modèles plus avancés que nous détaillerons plus tard dans le cours, tels que :

- le [Perceptron multi-couches](#)
- les [architectures convolutionnelles](#)
- les [architectures récurrentes](#)

le fait qu'une tâche s'accompagne d'une fonction de perte à minimiser (ici, nous avons utilisé l'erreur quadratique moyenne pour notre tâche de régression), qui sera discutée dans [un chapitre dédié](#);

le concept de descente de gradient pour optimiser la perte choisie sur le paramètre unique d'un modèle, et ceci sera étendu dans [notre chapitre sur l'optimisation](#).

CHAPITRE 2

PERCEPTRONS MULTICOUCHES

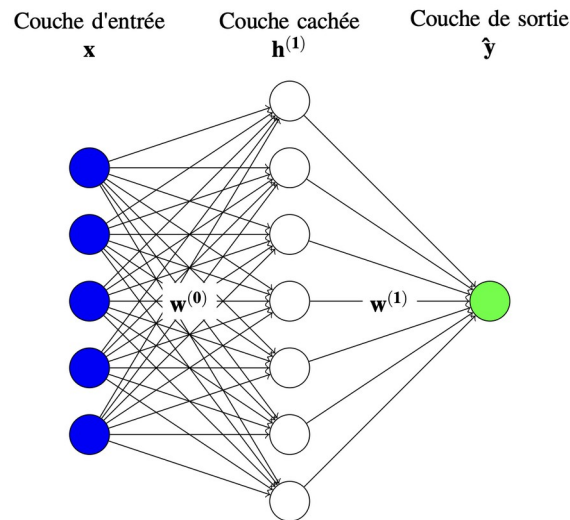
Dans le chapitre précédent, nous avons vu un modèle très simple appelé le perceptron. Dans ce modèle, la sortie prédite $\hat{y}$ est calculée comme une combinaison linéaire des caractéristiques d'entrée plus un biais :

$$\hat{y} = \sum_{j=1}^d x_j w_j + b$$

En d'autres termes, nous optimisons parmi la famille des modèles linéaires, qui est une famille assez restreinte.

2.1 Empiler des couches pour une meilleure expressivité

Afin de couvrir un plus large éventail de modèles, on peut empiler des neurones organisés en couches pour former un modèle plus complexe, comme le modèle ci-dessous, qui est appelé modèle à une couche cachée, car une couche supplémentaire de neurones est introduite entre les entrées et la sortie :



La question que l'on peut se poser maintenant est de savoir si cette couche cachée supplémentaire permet effectivement de couvrir une plus grande famille de modèles. C'est à cela que sert le théorème d'approximation universelle ci-dessous.

Théorème d'approximation universelle :

$$\hat{y} = \varphi_{\text{out}} \left(\sum_i w_i^{(2)} h_i^{(2)} + b^{(2)} \right) \quad (2.1)$$

$$\forall i, h_i^{(2)} = \varphi \left(\sum_j w_{ij}^{(1)} h_j^{(1)} + b_i^{(1)} \right) \quad (2.2)$$

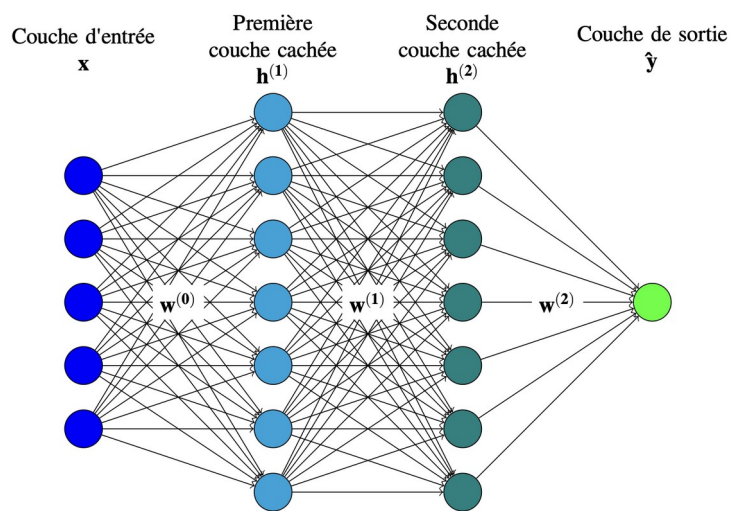
$$\forall i, h_i^{(1)} = \varphi \left(\sum_j w_{ij}^{(0)} x_j + b_i^{(0)} \right) \quad (2.3)$$

$$\iint 2e^{-i\omega t} \frac{dy e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, -\infty < x < \infty \max_{0 \leq x \leq 1} x e^{-x^2}}{dx}$$

Le théorème d'approximation universelle stipule que toute fonction continue définie sur un ensemble compact peut être approchée d'aussi près que l'on veut par un réseau neuronal à une couche cachée avec activation sigmoïde.

En d'autres termes, en utilisant une couche cachée pour mettre en correspondance les entrées et les sorties, on peut maintenant approximer n'importe quelle fonction continue, ce qui est une propriété très intéressante. Notez cependant que le nombre de neurones cachés nécessaire pour obtenir une qualité d'approximation donnée n'est pas discuté ici. De plus, il n'est pas suffisant qu'une telle bonne approximation existe, une autre question importante est de savoir si les algorithmes d'optimisation que nous utiliserons convergeront in fine vers cette solution ou non, ce qui n'est pas garanti, comme discuté plus en détail dans [le chapitre dédié](#).

En pratique, nous observons empiriquement que pour atteindre une qualité d'approximation donnée, il est plus efficace (en termes de nombre de paramètres requis) d'empiler plusieurs couches cachées plutôt que de s'appuyer sur une seule :



La représentation graphique ci-dessus correspond au modèle suivant :

$$\hat{y} = \varphi \left(\sum_i w^{(2)}_i h^{(2)}_i + b^{(2)} \right) \quad (2.1)$$

$$h^{(1)}_i = \varphi \left(\sum_j w^{(1)}_{ij} h^{(0)}_j + b^{(1)}_i \right) \quad (2.2)$$

$$h^{(0)}_i = \varphi \left(\sum_j w^{(0)}_{ij} x_j + b^{(0)}_i \right) \quad (2.3)$$

où

$$\varphi(x) = \frac{1}{1 + e^{-x}}$$

Pour être précis, les termes de biais $b^{(l)}$ ne sont pas représentés dans la représentation graphique ci-dessus.

De tels modèles avec une ou plusieurs couches cachées sont appelés Perceptrons multicouches (ou Multi-Layer Perceptrons, MLP).

2.2 Décider de l'architecture d'un MLP

Lors de la conception d'un modèle de perceptron multicouche destiné à être utilisé pour un problème spécifique, certaines quantités sont fixées par le problème en question et d'autres sont des hyper-paramètres du modèle.

Prenons l'exemple du célèbre jeu de données de classification d'iris :

```
import pandas as pd
iris = pd.read_csv("../data/iris.csv", index_col=0)
iris
sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) \
0
1
2
3
4
..
145
146
147
148
149

target 00 10 20 30 40 .. ...
145.      145  2
146.      146  2
147.      147  2
148.      148  2
149.      149  2

5.1      3.5
4.9      3.0
4.7      3.2
4.6      3.1
5.0      3.6
...      ...
6.7      3.0
6.3      2.5
6.5      3.0
6.2      3.4
5.9      3.0
1.4      0.2
1.4      0.2
1.3      0.2
1.5      0.2
1.4      0.2
...      ...
5.2      2.3
5.0      1.9
5.2      2.0
5.4      2.3
5.1      1.8
[150 rows x 5 columns]
```

L'objectif ici est d'apprendre à déduire l'attribut « cible » (3 classes différentes possibles) à partir des informations contenues dans les 4 autres attributs.

La structure de ce jeu de données dicte :

- le nombre de neurones dans la couche d'entrée, qui est égal au nombre d'attributs descriptifs dans notre jeu de

données (ici, 4), et

- le nombre de neurones dans la couche de sortie, qui est ici égal à 3, puisque le modèle est censé produire une probabilité par classe cible.

De manière plus générale, pour la couche de sortie, on peut être confronté à plusieurs situations :

- lorsqu'il s'agit de régression, le nombre de neurones de la couche de sortie est égal au nombre de caractéristiques à prédire par le modèle,

- quand il s'agit de classification

- Dans le cas d'une classification binaire, le modèle aura un seul neurone de sortie qui indiquera la probabilité de la classe positive,

- dans le cas d'une classification multi-classes, le modèle aura autant de neurones de sortie que le nombre de classes du problème.

Une fois que ces nombres de neurones d'entrée / sortie sont fixés, le nombre de neurones cachés ainsi que le nombre de neurones par couche cachée restent des hyper-paramètres du modèle.

2.3 Fonctions d'activation

Un autre hyper-paramètre important des réseaux neuronaux est le choix de la fonction d'activation φ .

Il est important de noter que si nous utilisons la fonction identité comme fonction d'activation, quelle que soit la profondeur de notre MLP, nous ne couvrirons plus que la famille des modèles linéaires. En pratique, nous utiliserons donc des fonctions d'activation qui ont un certain régime linéaire mais qui ne se comportent pas comme une fonction linéaire sur toute la gamme des valeurs d'entrée.

Historiquement, les fonctions d'activation suivantes ont été proposées : $\tanh(x) = \frac{2}{1+e^{-2x}} - 1$

$$\frac{2}{1+e^{-2x}} - 1$$

$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

$$\text{ReLU}(x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{sinon} \end{cases}$$

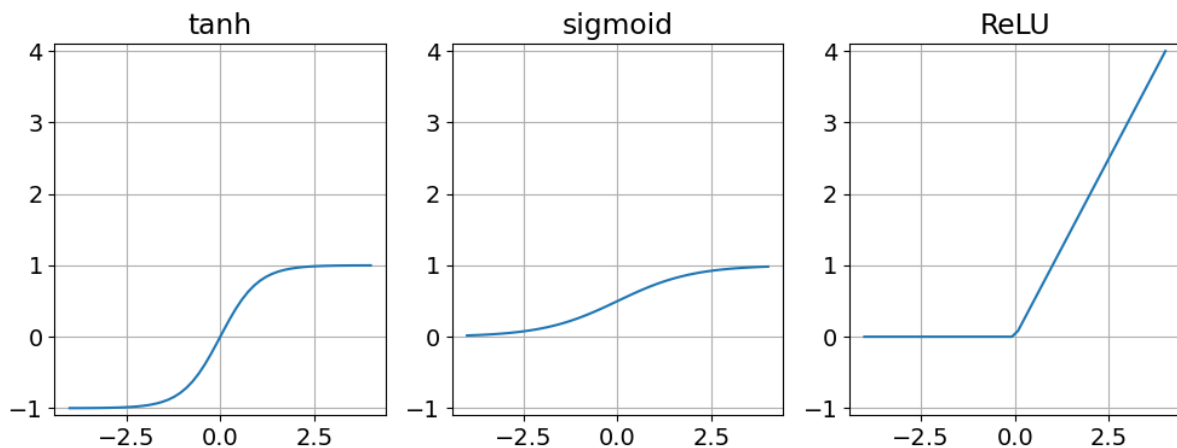
En pratique, la fonction ReLU (et certaines de ses variantes) est la plus utilisée de nos jours, pour des raisons qui seront discutées plus en détail dans [notre chapitre consacré à l'optimisation](#).

2.3.1 Le cas particulier de la couche de sortie

Vous avez peut-être remarqué que dans la formulation du MLP fournie par l'équation (1), la couche de sortie possède sa propre fonction d'activation, notée φ_{out} . Cela s'explique par le fait que le choix de la fonction d'activation pour la couche de sortie d'un réseau neuronal est spécifique au problème à résoudre.

En effet, vous avez pu constater que les fonctions d'activation abordées dans la section précédente ne partagent pas la même plage de valeurs de sortie. Il est donc primordial de choisir une fonction d'activation adéquate pour la couche de sortie, de sorte que notre modèle produise des valeurs cohérentes avec les quantités qu'il est censé prédire.

Si, par exemple, notre modèle est censé être utilisé dans l'ensemble de données sur les logements de Boston dont nous avons parlé [dans le chapitre précédent](#), l'objectif est de prédire les prix des logements, qui sont censés être des quantités non



négligentes. Il serait donc judicieux d'utiliser ReLU (qui peut produire toute valeur positive) comme fonction d'activation pour la couche de sortie dans ce cas.

Comme indiqué précédemment, dans le cas de la classification binaire, le modèle aura un seul neurone de sortie et ce neurone produira la probabilité associée à la classe positive. Cette quantité devra se situer dans l'intervalle $[0, 1]$, et la fonction d'activation sigmoïde est alors le choix par défaut dans ce cas.

Enfin, lorsque la classification multi-classes est en jeu, nous avons un neurone par classe de sortie et chaque neurone est censé fournir la probabilité pour une classe donnée. Dans ce contexte, les valeurs de sortie doivent être comprises entre 0 et 1, et leur somme doit être égale à 1. À cette fin, nous utilisons la fonction d'activation softmax définie comme suit :

$$\forall i, \text{softmax}(o) = \frac{e^{o_i}}{\sum_j e^{o_j}}$$

où, pour tous les i , les o_i sont les valeurs des neurones de sortie avant application de la fonction d'activation.

2.4 Déclarer un MLP en keras

Pour définir un modèle MLP dans keras, il suffit d'empiler des couches. A titre d'exemple, si l'on veut coder un modèle composé de :

- une couche d'entrée avec 10 neurones,
- d'une couche cachée de 20 neurones avec activation ReLU,
- une couche de sortie composée de 3 neurones avec activation softmax,

le code sera le suivant :


```
import keras_core as keras
from keras.layers import Dense, InputLayer from keras.models import Sequential
```

```
model = Sequential([
    InputLayer(input_shape=(10,)),
    Dense(units=20, activation="relu"),
    Dense(units=3, activation="softmax")
])
model.summary()
Using TensorFlow backend
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 20)	220
dense_1 (Dense)	(None, 3)	63

Total params: 283 (1.11 KB)
Trainable params: 283 (1.11 KB)
Non-trainable params: 0 (0.00 Byte)

Notez que `model.summary()` fournit un aperçu intéressant d'un modèle défini et de ses paramètres.

2.4. Déclarer un MLP en keras 1

Exercice #1

En vous basant sur ce que nous avons vu dans ce chapitre, pouvez-vous expliquer le nombre de paramètres retournés par `model.summary()` ci-dessus ?

Solution

Notre couche d'entrée est composée de 10 neurones, et notre première couche est entièrement connectée, donc chacun de ces neurones est connecté à un neurone de la couche cachée par un paramètre, ce qui fait déjà $10 \times 20 = 200$ paramètres. De plus, chacun des neurones de la couche cachée possède son propre paramètre de biais, ce qui fait 20 paramètres supplémentaires. Nous avons donc 220 paramètres, tels que sortis par `model.summary()` pour la couche "dense (Dense)".

De la même manière, pour la connexion des neurones de la couche cachée à ceux de la couche de sortie, le nombre total de paramètres est de $20 \times 3 = 60$ pour les poids plus 3 paramètres supplémentaires pour les biais.

Au total, nous avons $220 + 63 = 283$ paramètres dans ce modèle.

Exercice #2

Déclarez, en keras, un MLP avec une couche cachée composée de 100 neurones et une activation ReLU pour le jeu de données Iris présenté ci-dessus.

Solution

Exercice #3

Même question pour le jeu de données sur le logement à Boston présenté ci-dessous (le but ici est de prédire l'attribut PRICE en fonction des autres).

Solution

```
model = Sequential([
    InputLayer(input_shape=(4,)),
    Dense(units=100, activation="relu"),
    Dense(units=3, activation="softmax")
])
```

```
model = Sequential([
    InputLayer(input_shape=(6,)),
    Dense(units=100, activation="relu"),
    Dense(units=1, activation="relu")
])
```

```
RM  CRIM  INDUS  NOX  AGE  TAX  PRICE
0  6.575  0.00632  2.31  0.538  65.2  296.0  24.0
```

```
1.  1  6.421
2.  2  7.185
3.  3  6.998
4.  4  7.147
...
501 6.593 0.06263 11.93 0.573 69.1 273.0 22.4
502 6.120 0.04527 11.93 0.573 76.7 273.0 20.6
503 6.976 0.06076 11.93 0.573 91.0 273.0 23.9
504 6.794 0.10959 11.93 0.573 89.3 273.0 22.0
505 6.030 0.04741 11.93 0.573 80.8 273.0 11.9
[506 rows x 7 columns]
0.02731 7.07
0.02729 7.07
0.03237 2.18
0.06905 2.18
0.469 78.9
0.469 61.1
0.458 45.8
0.458 54.2
242.0 21.6
242.0 34.7
222.0 33.4
222.0 36.2
...
...
...
```



2.4. Déclarer un MLP en keras 15

Introduction au Deep Learning (notes de cours)



CHAPITRE 3

FONCTIONS DE COÛT

Nous avons maintenant présenté une première famille de modèles, qui est la famille MLP. Afin d'entraîner ces modèles (i.e. d'ajuster leurs paramètres pour qu'ils s'adaptent aux données), nous devons définir une fonction de coût (aussi appelée fonction de perte, ou loss function) à optimiser. Une fois cette fonction choisie, l'optimisation consistera à régler les paramètres du modèle de manière à la minimiser.

Dans cette section, nous présenterons deux fonctions de pertes standard, à savoir l'erreur quadratique moyenne (principalement utilisée pour la régression) et la fonction de perte logistique (utilisée en classification).

Dans ce qui suit, nous supposons connu un ensemble de données $\mathcal{D}$ composé de n échantillons annotés (x_i, y_i) , et nous désignons la sortie du modèle :

$$\forall i, \hat{y} = m_\theta(x_i)$$

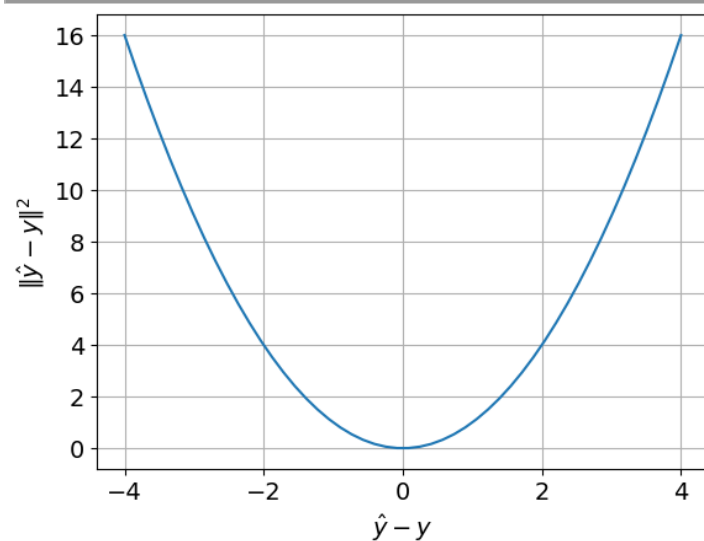
où m_θ est notre modèle et θ est l'ensemble de tous ses paramètres (poids et biais). **3.1 Erreur quadratique moyenne**

L'erreur quadratique moyenne (ou Mean Squared Error, MSE) est la fonction de perte la plus couramment utilisée dans les contextes de régression. Elle est définie comme suit

$$L(\mathcal{D}; m) = \frac{1}{n} \sum_i \| \hat{y}_i - y_i \|^2$$

$$= \frac{1}{n} \sum_i \| m_\theta(x_i) - y_i \|^2$$

Sa forme quadratique tend à pénaliser fortement les erreurs importantes :



3.2 Perte logistique

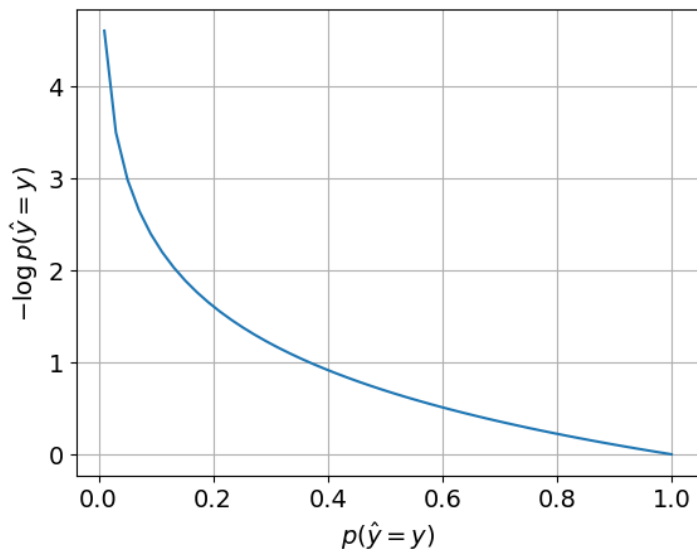
La perte logistique est la fonction de perte la plus largement utilisée pour entraîner des réseaux neuronaux dans des contextes de classification. Elle est définie comme suit

$$L(\mathcal{D}; m) = \frac{1}{n} \sum_i -\log p(\hat{y}_i = y_i; m)$$

où

$p(\hat{y}_i = y_i; m)$ est la probabilité prédite par le modèle m pour la classe correcte y_i .

Sa formulation tend à favoriser les cas où le modèle prédit la classe correcte avec une probabilité proche de 1, comme on peut s'y attendre :



CHAPITRE 4

OPTIMISATION

Dans ce chapitre, nous présenterons des variantes de la stratégie d'optimisation de descente de gradient et montrerons comment elles peuvent être utilisées pour optimiser les paramètres des réseaux de neurones.

Commençons par l'algorithme de base de la descente de gradient et ses limites.

Algorithm 1 (Descente de Gradient)

Entrée: Un jeu de données $\mathcal{D} = (X, y)$

1. Initialiser les paramètres θ du modèle 2. for $e = 1..E$

-
- 
1. for $(x_i, y_i) \in \mathcal{D}$
 1. Calculer la prédiction $\hat{y} = m(x)$

2. Calculer le gradient individuel $\nabla_{\theta} L_i$

2. Calculer le gradient total $\nabla_{\theta} L = \frac{1}{n} \sum_i \nabla_{\theta} L_i$ 3. Mettre à jour les paramètres θ à partir de $\nabla_{\theta} L$

La règle de mise à jour typique pour les paramètres θ à l'itération t est $\theta^{(t+1)} \leftarrow \theta^{(t)} - \rho \nabla_{\theta} L$

où ρ est un hyper-paramètre important de la méthode, appelé le taux d'apprentissage (ou learning rate). La descente de gradient consiste à jour itérativement θ dans la direction de la plus forte diminution de la perte L .

Comme on peut le voir dans l'algorithme précédent, lors d'une descente de gradient, les paramètres du modèle sont mis à jour une fois par epoch, ce qui signifie qu'un passage complet sur l'ensemble des données est nécessaire avant la mise à jour. Lorsque l'on traite de grands jeux de données, cela constitue une forte limitation, ce qui motive l'utilisation de variantes stochastiques.



4.1 Descente de gradient stochastique

L'idée derrière l'algorithme de descente de gradient stochastique (ou Stochastic Gradient Descent, SGD) est d'obtenir des estimations bon marché (au sens de la quantité de calculs nécessaires) pour la quantité

$$\nabla_{\theta} L(\mathcal{D}; m_{\theta}) = \frac{1}{n} \sum_{(x_i, y_i) \in \mathcal{D}} \nabla_{\theta} L(x_i, y_i; m_{\theta})$$

où $\mathcal{D}$ est l'ensemble d'apprentissage. Pour ce faire, on tire des sous-ensembles de données, appelés minibatches, et $\nabla_{\theta} L(B; m_{\theta}) = \frac{1}{b} \sum \nabla_{\theta} L(x_i, y_i; m_{\theta})$

$(x_i, y_i) \in B$

est utilisé comme estimateur de $\nabla_{\theta} L(\mathcal{D}; m_{\theta})$. Il en résulte l'algorithme suivant dans lequel les mises à jour des paramètres

se produisent après chaque minibatch, c'est-à-dire plusieurs fois par epoch. Algorithm 2 (Descente de gradient stochastique)

Input: A dataset $\mathcal{D} = (X, y)$

1. Initialiser les paramètres θ du modèle 2. for $e = 1..E$

1. for $t = 1..n_{\text{minibatches}}$

1. Tirer un échantillon aléatoire de taille b dans $\mathcal{D}$ que l'on appelle minibatch

2. for $(x_i, y_i) \in B$

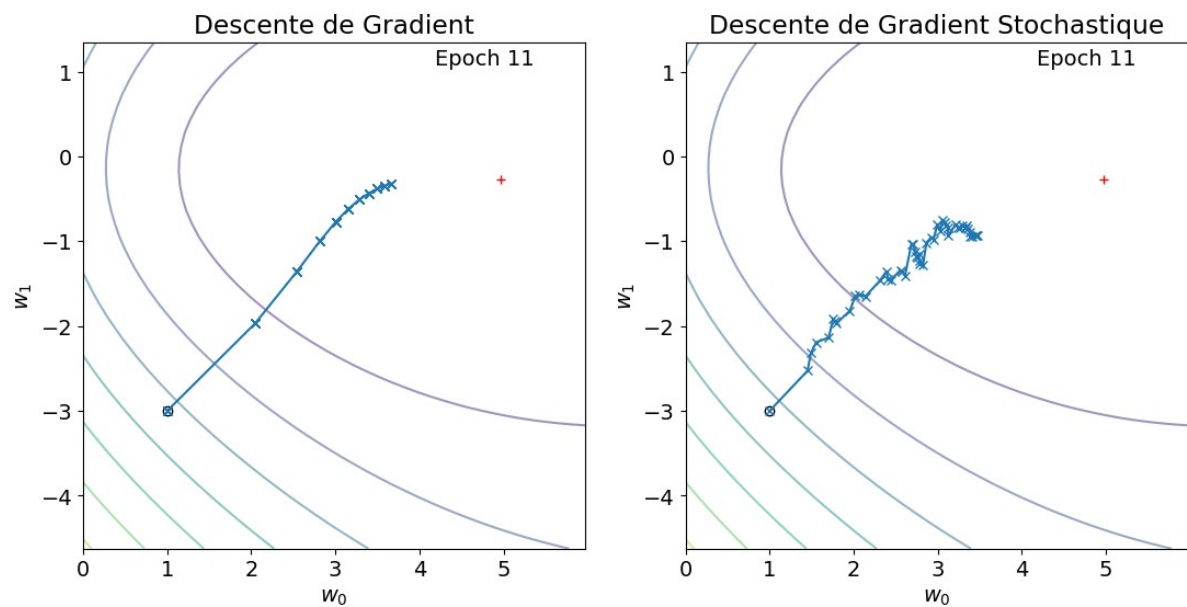
1. Calculer la prédiction $\hat{y} = m(x)$

$i \theta i$

2. Calculer le gradient individuel $\nabla_{\theta} L_i$

3. Calculer le gradient sommé sur le minibatch $\nabla_{\theta} L_B = \frac{1}{b} \sum_i \nabla_{\theta} L_i$ 4. Mettre à jour les paramètres θ à partir de $\nabla_{\theta} L_B$

Par conséquent, lors de l'utilisation de SGD, les mises à jour des paramètres sont plus fréquentes, mais elles sont « bruitées » puisqu'elles sont basées sur une estimation du gradient par minibatch au lieu de s'appuyer sur le vrai gradient, comme illustré ci-dessous :



Outre le fait qu'elle implique des mises à jour plus fréquentes des paramètres, la SGD présente un avantage supplémentaire en termes d'optimisation, qui est essentiel pour les réseaux de neurones. En effet, comme on peut le voir ci-dessous, contrairement à ce que nous avons dans le cas du Perceptron, la perte MSE (et il en va de même pour la perte logistique) n'est plus convexe en les paramètres du modèle dès que celui-ci possède au moins une couche cachée :

La descente de gradient est connue pour souffrir d'optima locaux, et de tels fonctions de pertes constituent un problème sérieux pour la descente de gradient. D'un autre côté, la descente de gradient stochastique est susceptible de bénéficier d'estimations de gradient bruitées pour s'échapper des minima locaux.

4.2 Une note sur Adam

Adam [Kingma and Ba, 2015] est une variante de la méthode de descente de gradient stochastique. Elle diffère dans la règle de mise à jour des paramètres.

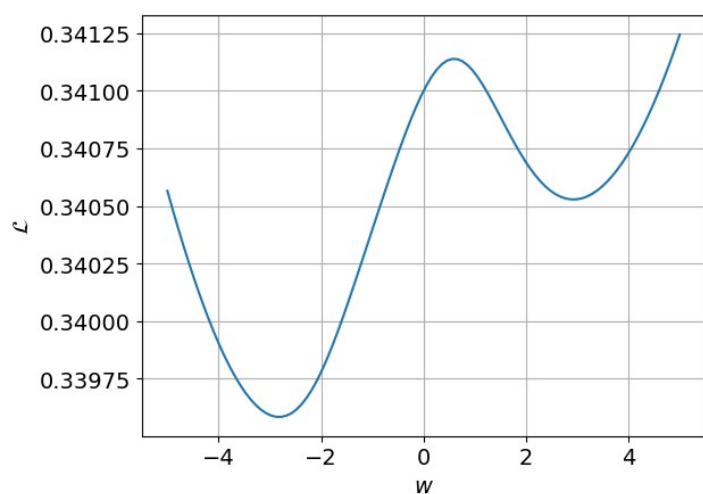
Tout d'abord, elle utilise ce qu'on appelle le momentum, qui consiste essentiellement à s'appuyer sur les mises à jour antérieures du gradient pour lisser la trajectoire dans l'espace des paramètres pendant l'optimisation. Une illustration interactive du momentum peut être trouvée dans [Goh, 2017].

L'estimation du gradient est remplacée par la quantité :

$$m^{(t+1)} \leftarrow \beta m^{(t)} + (1-\beta) \nabla L$$

Lorsque β_1 est égal à zéro, nous avons $m^{(t+1)} = \nabla_{\theta} L$ et pour $\beta_1 \in]0, 1[$, $m^{(t+1)}$ l'estimation courante du gradient utilise l'information sur les estimations passées, stockée dans $m^{(t)}$.

Une autre différence importante entre SGD et la Adam consiste à utiliser un taux d'apprentissage adaptatif. En d'autres termes, au lieu d'utiliser le même taux d'apprentissage ρ pour tous les paramètres du modèle, le taux d'apprentissage pour un paramètre donné θ_i est défini comme :



$\hat{\rho}^{(t+1)}$

$$\rho(\theta_i) = \sqrt{s^{(t+1)}(\theta_i)} + \epsilon$$

où ϵ est une constante petite devant 1 et

$$s^{(t+1)}(\theta) = [\beta s^{(t)}(\theta) + (1-\beta) (\nabla L)^2]$$

$$\frac{1}{1 - \beta_2^{t-2i} \theta_i}$$

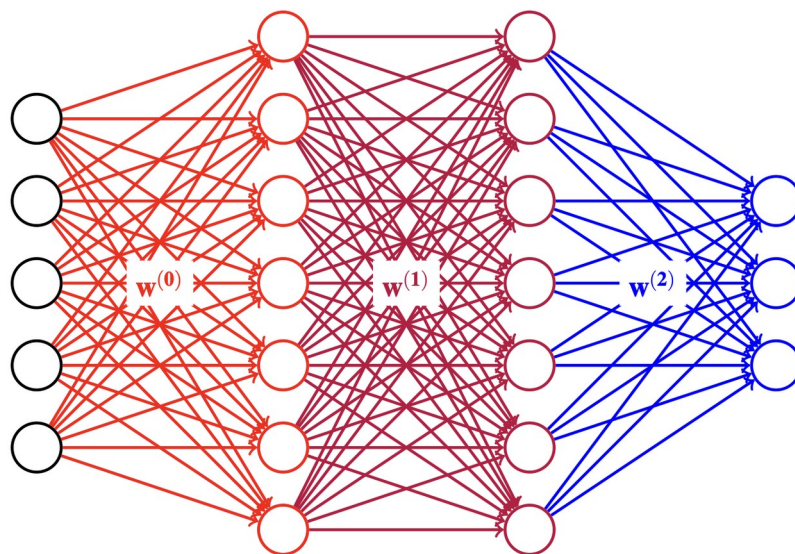
Ici aussi, le terme s utilise le momentum. Par conséquent, le taux d'apprentissage sera réduit pour les paramètres qui ont subi de grandes mises à jour dans les itérations précédentes.

Globalement, la règle de mise à jour d'Adam est la suivante :

$$\theta \leftarrow \theta - \hat{\rho}^{(t+1)} m^{(t+1)}$$

4.3 La malédiction de la profondeur

Considérons le réseau neuronal suivant :



$w^{(2)}$

et rappelons que, pour une couche donnée (l), la sortie de la couche est calculée comme suit $a^{(l)} = \varphi(o^{(l)}) = \varphi(w^{(l-1)} a^{(l-1)})$

où φ est la fonction d'activation pour la couche donnée (nous ignorons les termes de biais dans cet exemple simplifié). Afin d'effectuer une descente de gradient (stochastique), les gradients de la perte par rapport aux paramètres du modèle

doivent être calculés.

En utilisant la règle de la dérivation en chaîne, ces gradients peuvent être exprimés comme suit :

$$\frac{\partial L}{\partial w^{(2)}} = \frac{\partial L}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial w^{(2)}}$$

$$\frac{\partial L}{\partial w^{(1)}} = \frac{\partial L}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial a^{(2)}}{\partial o^{(2)}} \frac{\partial o^{(2)}}{\partial w^{(1)}}$$

$$\frac{\partial L}{\partial w^{(0)}} = \frac{\partial L}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial a^{(2)}}{\partial o^{(2)}} \frac{\partial a^{(1)}}{\partial o^{(1)}} \frac{\partial o^{(1)}}{\partial w^{(0)}}$$

Il y a des idées importantes à saisir ici.

Tout d'abord, il faut remarquer que les poids qui sont plus éloignés de la sortie du modèle héritent de règles de gradient composées de plus de termes. Par conséquent, lorsque certains de ces termes deviennent de plus en plus petits, il y a un risque plus élevé pour ces poids que leurs gradients tombent à 0. C'est ce qu'on appelle l'effet de gradient évanescent (vanishing gradient), qui est un phénomène très courant dans les réseaux neuronaux profonds (c'est-à-dire les réseaux composés de nombreuses couches).

22 Chapitre 4. Optimisation

Deuxièmement, certains termes sont répétés dans ces formules, et en général, des termes de la forme $\partial a^{(l)}$ présents à plusieurs endroits. Ces termes peuvent être développés comme suit :

$$\partial a^{(l)} = \varphi'(o^{(l)}) \partial o^{(l)}$$

$$\partial o^{(l)} = w^{(l-1)} \partial a^{(l-1)}$$

Voyons à quoi ressemblent les dérivées des fonctions d'activation standard :

et

$$\partial o^{(l)} \text{ sont } \partial a^{(l-1)}$$

On peut constater que la dérivée de ReLU possède une plus grande plage de valeurs d'entrée pour lesquelles elle est non

nulle (typiquement toute la plage de valeurs d'entrée positives) que ses concurrentes, ce qui en fait une fonction d'activation

très intéressante pour les réseaux neuronaux profonds, car nous avons vu que le terme $\partial a^{(l)}$ apparaît de manière répétée

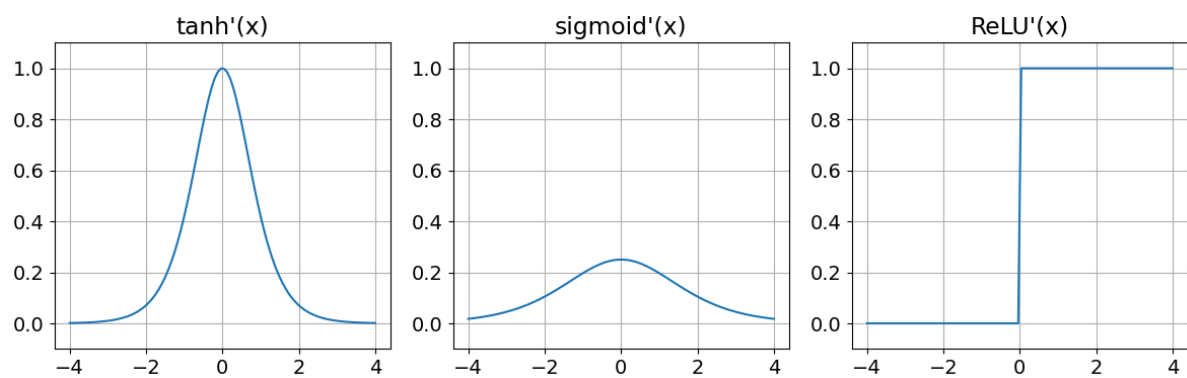
dans les dérivations en chaîne.

4.4 Coder tout cela en keras

Dans keras, les informations sur les pertes et l'optimiseur sont transmises au moment de la compilation

$\partial o^{(l)}$

$\partial o^{(l)}$



```
import keras_core as keras
from keras.layers import Dense, InputLayer from keras.models import Sequential
```

```
model = Sequential([
    InputLayer(input_shape=(10, )),
    Dense(units=20, activation="relu"),
    Dense(units=3, activation="softmax")
])
```

```
model.summary()
Using TensorFlow backend
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 20)	220

(suite sur la page suivante)

4.4. Coder tout cela en keras 23

Introduction au Deep Learning (notes de cours)



En termes de pertes :

- "mse" est la perte d'erreur quadratique moyenne,
- "binary_crossentropy" est la perte logistique pour la classification binaire,
- "categorical_crossentropy" est la perte logistique pour la classification multi-classes.

Les optimiseurs définis dans cette section sont disponibles sous forme de "sgd" et "adam". Afin d'avoir le contrôle sur les hyper-paramètres des optimiseurs, on peut alternativement utiliser la syntaxe suivante :

(suite de la page précédente)

dense_1 (Dense)	(None, 3)	63
-----------------	-----------	----

Total params: 283 (1.11 KB)
Trainable params: 283 (1.11 KB)
Non-trainable params: 0 (0.00 Byte)

```
model.compile(loss="categorical_crossentropy", optimizer="adam")
```

```

from keras.optimizers import Adam, SGD

# Not a very good idea to tune beta_1 # and beta_2 parameters in Adam adam_opt = Adam(learning_rate=0.001,

beta_1=0.9, beta_2=0.9)
# In order to use SGD with a custom learning rate:

sgd_opt = SGD(learning_rate=0.001)
model.compile(loss="categorical_crossentropy", optimizer=adam_opt)

```

4.5 Prétraitement des données

En pratique, pour que la phase d'ajustement du modèle se déroule correctement, il est important de mettre à l'échelle les données d'entrée. Dans l'exemple suivant, nous allons comparer deux entraînements du même modèle, avec une initialisation similaire et la seule différence entre les deux sera de savoir si les données d'entrée sont centrées-réduites ou laissées telles quelles.

```

import pandas as pd
from keras.utils import to_categorical

iris = pd.read_csv("../data/iris.csv", index_col=0)
iris = iris.sample(frac=1)
y = to_categorical(iris["target"])
X = iris.drop(columns=["target"])

from keras.layers import Dense, InputLayer from keras.models import Sequential
from keras.utils import set_random_seed

```

(suite sur la page suivante)

24 Chapitre 4. Optimisation

Standardisons maintenant nos données et comparons les performances obtenues :

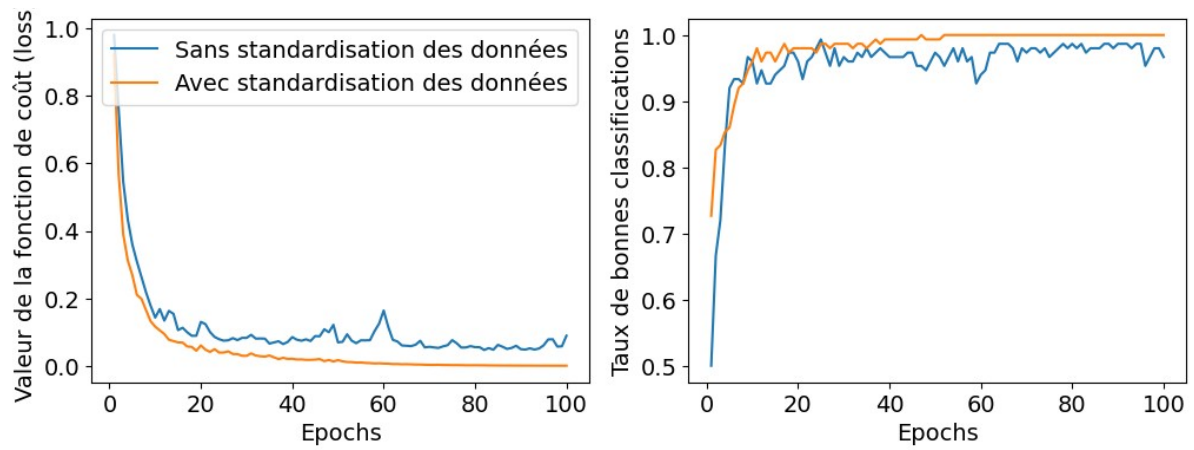
Introduction au Deep Learning (notes de cours)

(suite de la page précédente)

```
set_random_seed(0)
model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=3, activation="softmax")
])

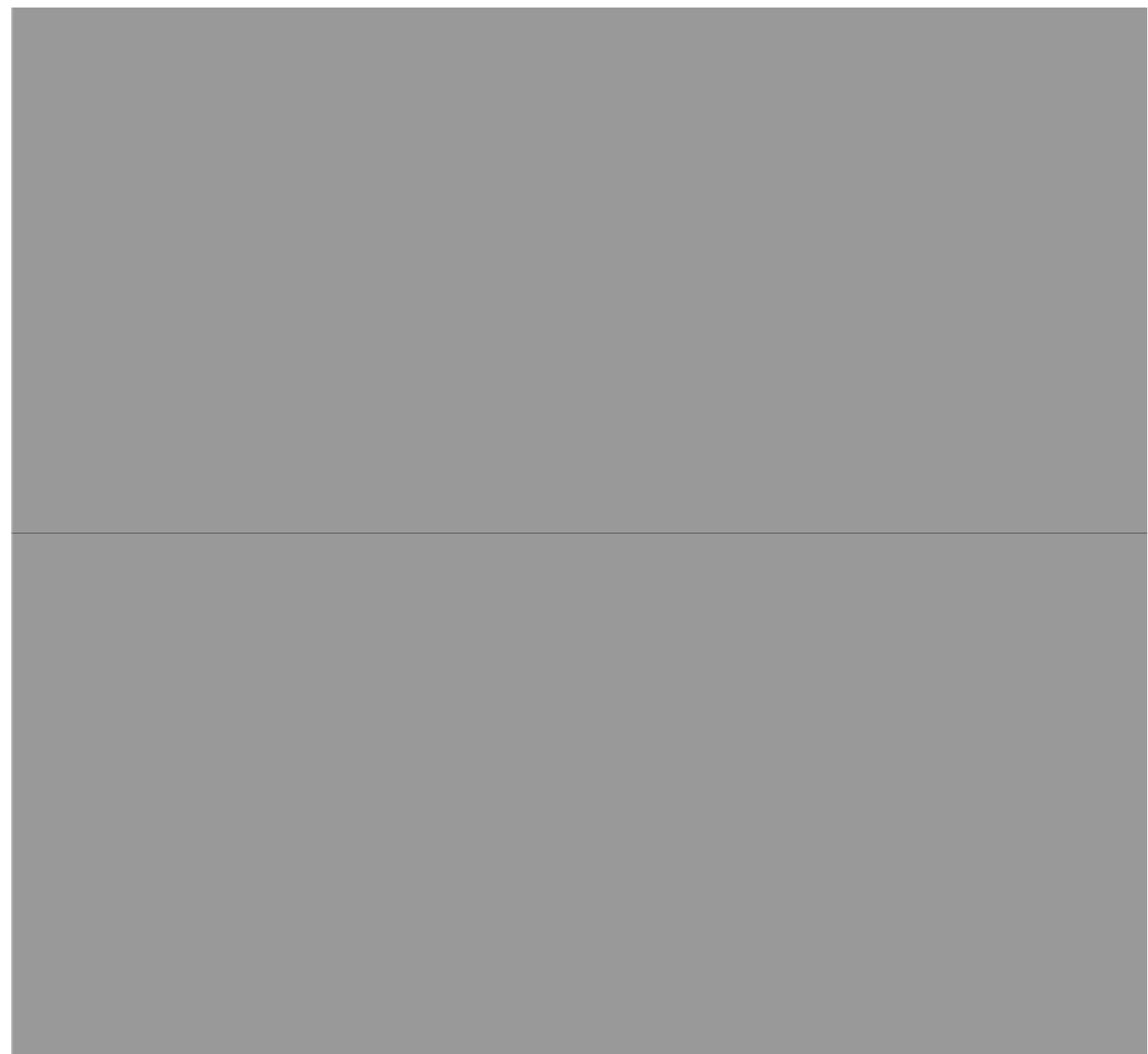
n_epochs = 100
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
h = model.fit(X, y, epochs=n_epochs, batch_size=30, verbose=0)
X -= X.mean(axis=0)
X /= X.std(axis=0)
set_random_seed(0)
model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=3, activation="softmax")
])

n_epochs = 100
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
h_standardized = model.fit(X, y, epochs=n_epochs, batch_size=30, verbose=0)
```



4.5. Prétraitement des données 25

Introduction au Deep Learning (notes de cours)



CHAPITRE 5

RÉGULARISATION

Comme nous l'avons vu dans les chapitres précédents, l'une des forces des réseaux neuronaux est qu'ils peuvent approximer n'importe quelle fonction continue lorsqu'un nombre suffisant de paramètres est utilisé. Lors de l'utilisation d'approximateurs universels dans des contextes d'apprentissage automatique, un risque connexe important est celui du surajustement (overfitting) aux données d'apprentissage. Plus formellement, étant donné un jeu de données d'apprentissage $\mathcal{D}_t$ tiré d'une distribution inconnue $\mathcal{D}$, les paramètres du modèle sont optimisés de manière à minimiser le risque empirique :

$$R_e(\theta) = \frac{1}{|\mathcal{D}_t|} \sum_{(x_i, y_i) \in \mathcal{D}_t} L(x_i, y_i; m_\theta)$$

alors que le véritable objectif est de minimiser le « vrai » risque :

$$R(\theta) = \mathbb{E}_{x, y \sim \mathcal{D}} L(x, y; m_\theta)$$

et les deux objectifs n'ont pas le même minimiseur.

Pour éviter cet écueil, il faut utiliser des techniques de régularisation, telles que celles présentées ci-après.

5.1 Early stopping

Comme illustré ci-dessous, on peut observer que l'entraînement d'un réseau neuronal pendant un trop grand nombre d'époques peut conduire à un surajustement. Notez qu'ici, le risque réel est estimé grâce à l'utilisation d'un ensemble de validation qui n'est pas vu pendant l'entraînement.

Using TensorFlow backend

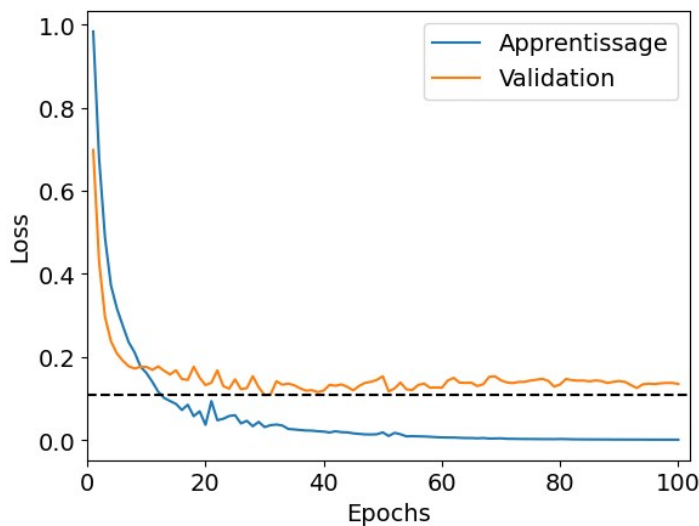
```
iris = pd.read_csv("../data/iris.csv", index_col=0)
iris = iris.sample(frac=1)
y = to_categorical(iris["target"])
X = iris.drop(columns=["target"])
X -= X.mean(axis=0)
X /= X.std(axis=0)
```



```
import keras_core as keras
from keras.layers import Dense, InputLayer from keras.models import Sequential
from keras.utils import set_random_seed

set_random_seed(0)
model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=256, activation="relu"),
    Dense(units=3, activation="softmax")
])

n_epochs = 100
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
h = model.fit(X, y, validation_split=0.3, epochs=n_epochs, batch_size=30, verbose=0)
```

Ici, le meilleur modèle (en termes de capacités de généralisation) semble être le modèle à l'époch 30. En d'autres termes, si nous avions arrêté le processus d'apprentissage après l'époch 30, nous aurions obtenu un meilleur modèle que si nous utilisons le modèle entraîné pendant 70 epochs.

C'est toute l'idée derrière la stratégie d'early stopping, qui consiste à arrêter le processus d'apprentissage dès que la perte de validation cesse de s'améliorer. Cependant, comme on peut le voir dans la visualisation ci-dessus, la perte de validation a tendance à osciller, et on attend souvent plusieurs epochs avant de supposer que la perte a peu de chances de s'améliorer dans le futur. Le nombre d'epochs à attendre est appelé le paramètre de patience.

Dans keras, l'arrêt anticipé peut être configuré via un callback, comme dans l'exemple suivant :

(suite sur la page suivante)

```
from keras.callbacks import EarlyStopping

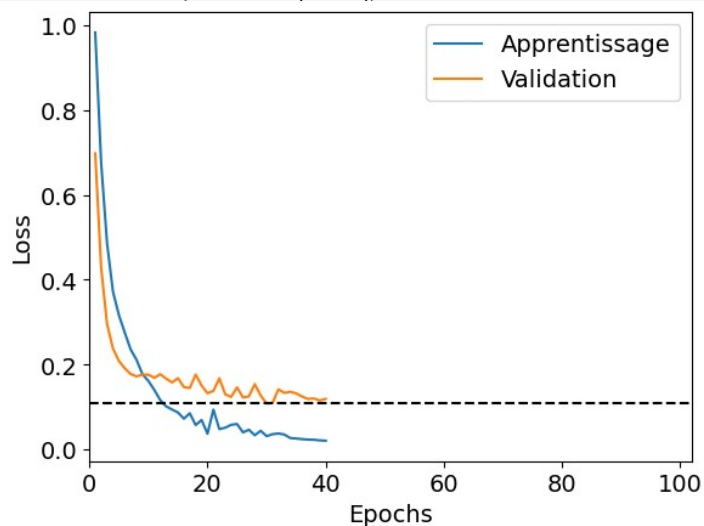
set_random_seed(0)
model = Sequential([
    InputLayer(input_shape=(4,)),
```

(suite de la page précédente)

```
Dense(units=256, activation="relu"),
Dense(units=256, activation="relu"),
Dense(units=256, activation="relu"),
Dense(units=3, activation="softmax")

])
cb_es = EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)

n_epochs = 100
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
h = model.fit(X, y,
              validation_split=0.3, epochs=n_epochs, batch_size=30,
              verbose=0, callbacks=[cb_es])
```



Et maintenant, même si le modèle était prévu pour être entraîné pendant 70 epochs, l'entraînement est arrêté dès qu'il atteint 10 epochs consécutives sans amélioration de la perte de validation, et les paramètres du modèle sont restaurés comme les paramètres du modèle à l'époch 30.

5.2 Pénalisation de la perte

Une autre façon importante d'appliquer la régularisation dans les réseaux neuronaux est la pénalisation des pertes. Un exemple typique de cette stratégie de régularisation est la régularisation L2. Si nous désignons par L_r la perte régularisée par L2, elle peut être exprimée comme suit :

$$L_r(\mathcal{D}; m_\theta) = L(\mathcal{D}; m_\theta) + \lambda \sum \|\theta^{(l)}\|^2$$

où $\theta^{(l)}$ est la matrice de poids de la couche l.

Cette régularisation tend à réduire les grandes valeurs des paramètres pendant le processus d'apprentissage, ce qui est

connu pour aider à améliorer la généralisation. En keras, ceci est implémenté comme :



5.2. Pénalisation de la perte

Introduction au Deep Learning (notes de cours)



```
from keras.regularizers import L2 λ = 0.01
```

```
set_random_seed(0)  
model = Sequential([
```

```

InputLayer(input_shape=(4,)),
Dense(units=256, activation="relu", kernel_regularizer=L2( $\lambda$ )),
Dense(units=256, activation="relu", kernel_regularizer=L2( $\lambda$ )),
Dense(units=256, activation="relu", kernel_regularizer=L2( $\lambda$ )),
Dense(units=3, activation="softmax")

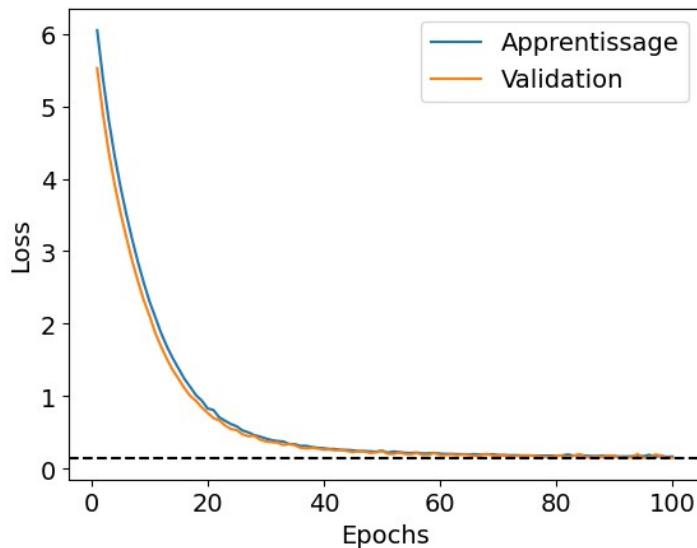
```

)

```

n_epochs = 100
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
h = model.fit(X, y, validation_split=0.3, epochs=n_epochs, batch_size=30, verbose=0)

```



5.3 Dropout

Dans cette section, nous présentons la stratégie Dropout, qui a été introduite dans [Srivastava et al., 2014]. L'idée derrière le Dropout est d'éteindre certains neurones pendant l'apprentissage. Les neurones désactivés changent à chaque minibatch de sorte que, globalement, tous les neurones sont entraînés pendant tout le processus.

Le concept est très similaire dans l'esprit à une stratégie utilisée pour l'entraînement des forêts aléatoires, qui consiste à sélectionner aléatoirement des variables candidates pour chaque division d'arbre à l'intérieur d'une forêt, ce qui est connu pour conduire à de meilleures performances de généralisation pour les forêts aléatoires. La principale différence ici est que l'on peut non seulement désactiver les neurones d'entrée mais aussi les neurones de la couche cachée pendant l'apprentissage.

Dans keras, ceci est implémenté comme une couche, qui agit en désactivant les neurones de la couche précédente dans le réseau :



30 Chapitre 5. Régularisation

Introduction au Deep Learning (notes de cours)



FIG. 5.1 – Illustration du mécanisme de DropOut. Afin d’entraîner un modèle donné (à gauche), à chaque minibatch, une proportion donnée de neurones est choisie au hasard pour être « désactivée » et le sous-réseau

résultat est utilisé pour l'étape d'optimisation en cours (cf. figure de droite, dans laquelle 40% des neurones – colorés en gris – sont désactivés).

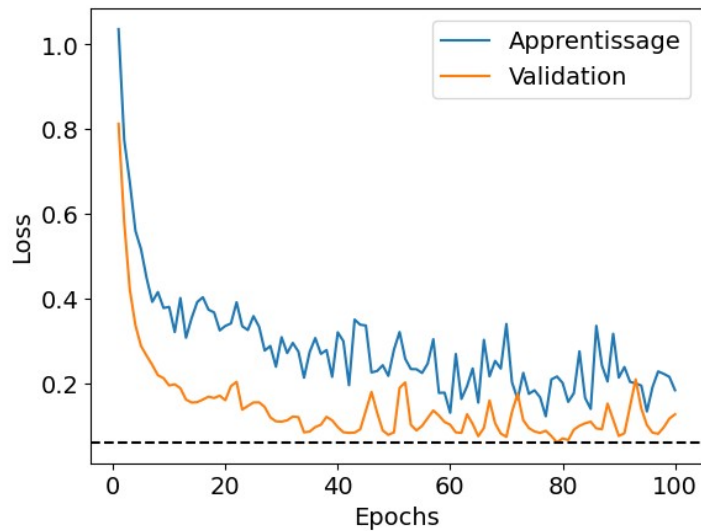
```
from keras.layers import Dropout
```

```
set_random_seed(0)
switchoff_proba = 0.3
model = Sequential([
    InputLayer(input_shape=(4, )),
    Dropout(rate=switchoff_proba),
    Dense(units=256, activation="relu"),
    Dropout(rate=switchoff_proba),
    Dense(units=256, activation="relu"),
    Dropout(rate=switchoff_proba),
    Dense(units=256, activation="relu"),
    Dropout(rate=switchoff_proba),
    Dense(units=3, activation="softmax")
])
```

```
n_epochs = 100
```

```
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
```

```
h = model.fit(X, y, validation_split=0.3, epochs=n_epochs, batch_size=30, verbose=0)
```





Exercice #1

En observant les valeurs de perte dans la figure ci-dessus, pouvez-vous expliquer pourquoi la perte de validation est presque systématiquement inférieure à celle calculée sur le jeu d'apprentissage ?

Solution

En fait, la perte d'apprentissage est calculée comme la perte moyenne sur tous les minibatches d'apprentissage pendant une epoch. Si nous nous rappelons que pendant l'apprentissage, à chaque minibatch, 30% des neurones sont désactivés, on peut voir que seule une sous-partie du modèle complet est utilisée lors de l'évaluation de la perte d'apprentissage alors que le modèle complet est utilisé lors de la prédiction sur l'ensemble de validation, ce qui explique pourquoi la perte de validation mesurée est inférieure à celle de l'apprentissage.



CHAPITRE 6

RÉSEAUX NEURONAUX CONVOLUTIFS

Les réseaux de neurones convolutifs (aussi appelés ConvNets) sont conçus pour tirer parti de la structure des données. Dans ce chapitre, nous aborderons deux types de réseaux convolutifs : nous commencerons par le cas monodimensionnel et verrons comment les réseaux convolutifs à convolutions 1D peuvent être utiles pour traiter les séries temporelles. Nous présenterons ensuite le cas 2D, particulièrement utile pour traiter les données d'image.

6.1 Réseaux de neurones convolutifs pour les séries temporelles

Les réseaux de neurones convolutifs pour les séries temporelles reposent sur l'opérateur de convolution 1D qui, étant donné une série temporelle x et un filtre f , calcule une carte d'activation comme :

où le filtre f est de longueur $(2L + 1)$.

Le code suivant illustre cette notion en utilisant un filtre gaussien :

$$(x * f)(t) = \sum_{k=-L}^L f_k x_{t+k}$$

$k=-L$

(6.1)

Les réseaux de neurones convolutifs sont constitués de blocs de convolution dont les paramètres sont les coefficients des filtres qu'ils intègrent (les filtres ne sont donc pas fixés a priori comme dans l'exemple ci-dessus mais plutôt appris). Ces blocs de convolution sont équivariants par translation, ce qui signifie qu'un décalage (temporel) de leur entrée entraîne le même décalage temporel de leur sortie :

```
/tmp/ipykernel_14514/1028966743.py:32: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.
```

```
plt.tight_layout()
```

```
/tmp/ipykernel_14514/1028966743.py:23: MatplotlibDeprecationWarning: Auto-removal
```

```
of overlapping axes is deprecated since 3.6 and will be removed two minor releases later; explicitly call ax.remove() as needed.
```

```
fig2 = plt.subplot(2, 1, 2)
```

```
/tmp/ipykernel_14514/1028966743.py:32: UserWarning: The figure layout has changed
```

```
to tight plt.tight_layout()
```





Les modèles convolutifs sont connus pour être très performants dans les applications de vision par ordinateur, utilisant des quantités modérées de paramètres par rapport aux modèles entièrement connectés (bien sûr, des contre-exemples existent, et le terme « modéré » est particulièrement vague).

La plupart des architectures standard de séries temporelles qui reposent sur des blocs convolutionnels sont des adaptations directes de modèles de la communauté de la vision par ordinateur ([[Le Guennec et al., 2016](#)] s'appuie sur une alternance entre couches de convolution et couches de pooling, tandis que des travaux plus récents s'appuient sur des connexions résiduelles et des modules d'inception [[Fawaz et al., 2020](#)]). Ces blocs de base (convolution, pooling, couches résiduelles) sont discutés plus en détail dans la section suivante.

Ces modèles de classification des séries temporelles (et bien d'autres) sont présentés et évalués dans [[Fawaz et al., 2019](#)] que nous recommandons au lecteur intéressé.

6.2 Réseaux de neurones convolutifs pour les images

Nous allons maintenant nous intéresser au cas 2D, dans lequel les filtres de convolution ne glissent pas sur un seul axe comme dans le cas des séries temporelles, mais plutôt sur les deux dimensions (largeur et hauteur) d'une image.

6.2.1 Images et convolutions

Comme on le voit ci-dessous, une image est une grille de pixels, et chaque pixel a une valeur d'intensité dans chacun des canaux de l'image. Les images couleur sont typiquement composées de 3 canaux (ici Rouge, Vert et Bleu).

FIG. 6.1 – Une image et ses 3 canaux (intensités de Rouge, Vert et Bleu, de gauche à droite).

La sortie d'une convolution sur une image x est une nouvelle image, dont les valeurs des pixels peuvent être calculées



comme suit :

$KL3$

$$\sum_{k=-K} \sum_{l=-L} \sum_{c=1}^3 f_{k,l,c} x_{i+k,j+l,c} \cdot (6.2)$$

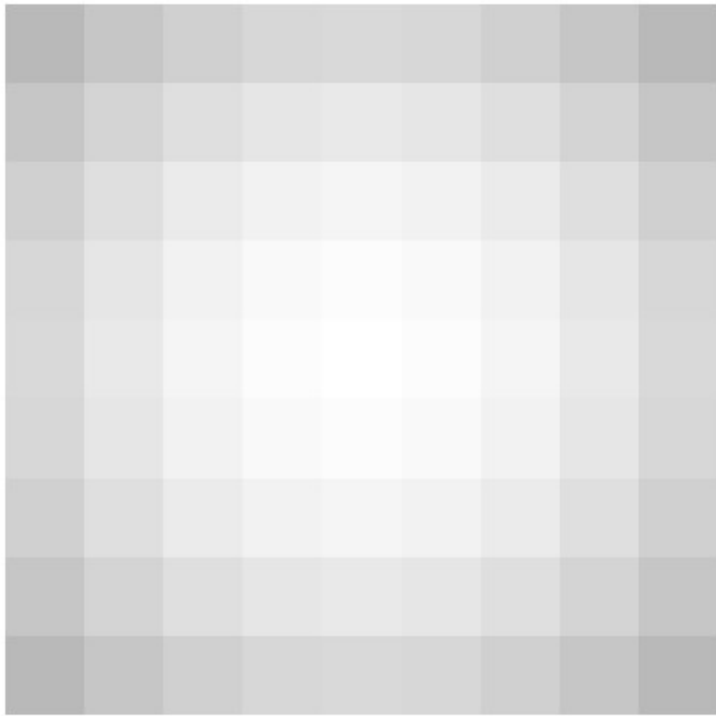
$$(x * f)(i, j) =$$



En d'autres termes, les pixels de l'image de sortie sont calculés comme le produit scalaire entre un filtre de convolution (qui est un tenseur de forme $(2K + 1, 2L + 1, c)$) et un patch d'image centré à la position donnée.

Considérons, par exemple, le filtre de convolution 9x9 suivant :

Le résultat de la convolution de l'image de chat ci-dessus avec ce filtre est l'image suivante en niveaux de gris (c'est-à-dire constituée d'un seul canal) :





6.2. Réseaux de neurones convolutifs pour les images 35

Introduction au Deep Learning (notes de cours)



On peut remarquer que cette image est une version floue de l'image originale. C'est parce que nous avons utilisé un filtre Gaussien. Comme pour les séries temporelles, lors de l'utilisation d'opérations de convolution dans les réseaux neuronaux, le contenu des filtres sera appris, plutôt que défini a priori.

6.2.2 Réseaux convolutifs de type LeNet

Dans [LeCun et al., 1998], un empilement de couches de convolution, de pooling et de couches entièrement connectées est introduit pour une tâche de classification d'images, plus spécifiquement une application de reconnaissance de chiffres. Le réseau neuronal résultant, appelé LeNet, est représenté ci-dessous :

FIG. 6.2 – Modèle LeNet-5

Couches de convolution

Une couche de convolution est constituée de plusieurs filtres de convolution (également appelés kernels) qui opèrent en parallèle sur la même image d'entrée. Chaque filtre de convolution génère une carte d'activation en

sortie et toutes ces cartes sont empilées pour former la sortie de la couche de convolution. Tous les filtres d'une couche partagent la même largeur et la même hauteur. Un terme de biais et une fonction d'activation peuvent être utilisés dans les couches de convolution, comme dans d'autres couches de réseaux neuronaux. Dans l'ensemble, la sortie d'un filtre de convolution est calculée comme suit :

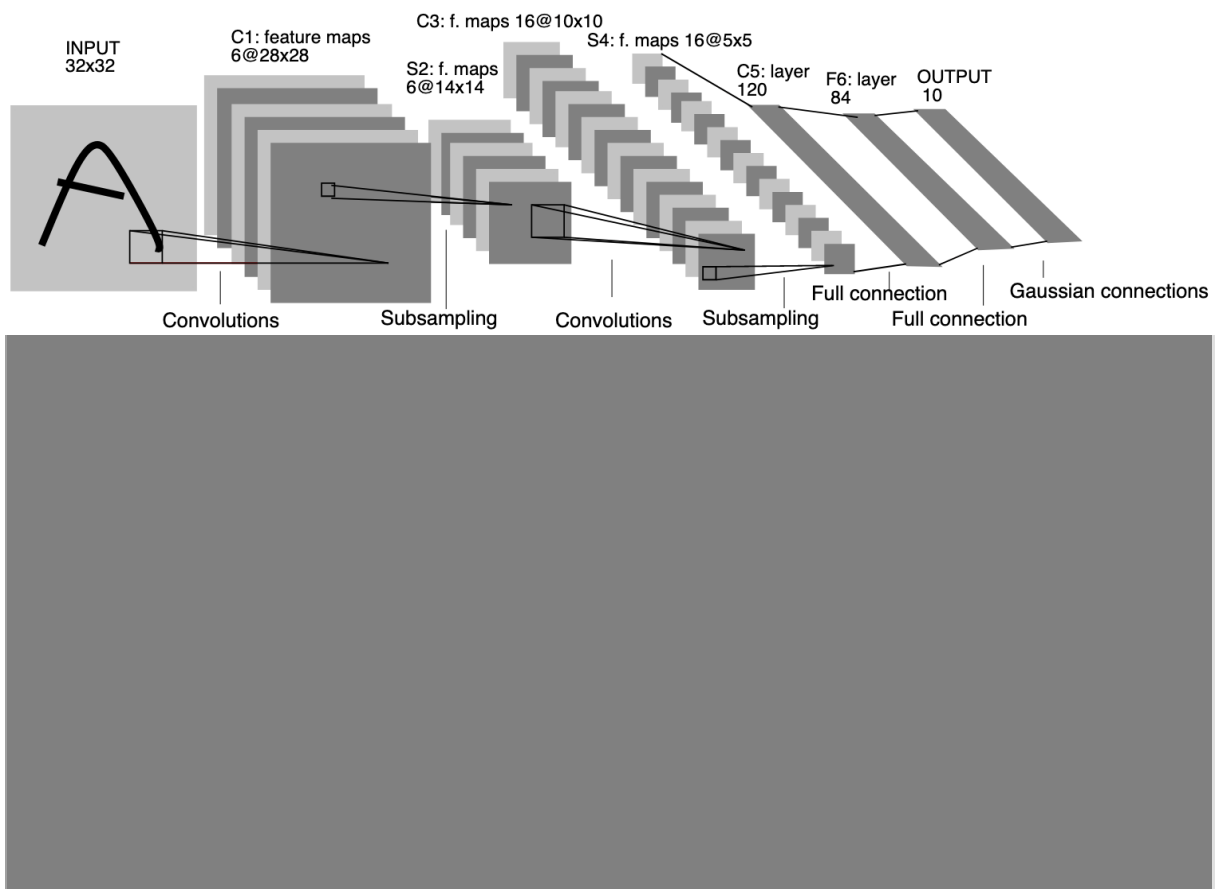
$$(x*f)(i,j,c)=\varphi(\sum_{k=-K}^K \sum_{l=-L}^L \sum_{c'} f_{k,l,c'}^c x_{i+k,j+l,c'} + b_c) \quad (6.3)$$

où c désigne le canal de sortie (notez que chaque canal de sortie est associé à un filtre f^c), b_c est le terme de biais qui lui

est associé et φ est la fonction d'activation utilisée.

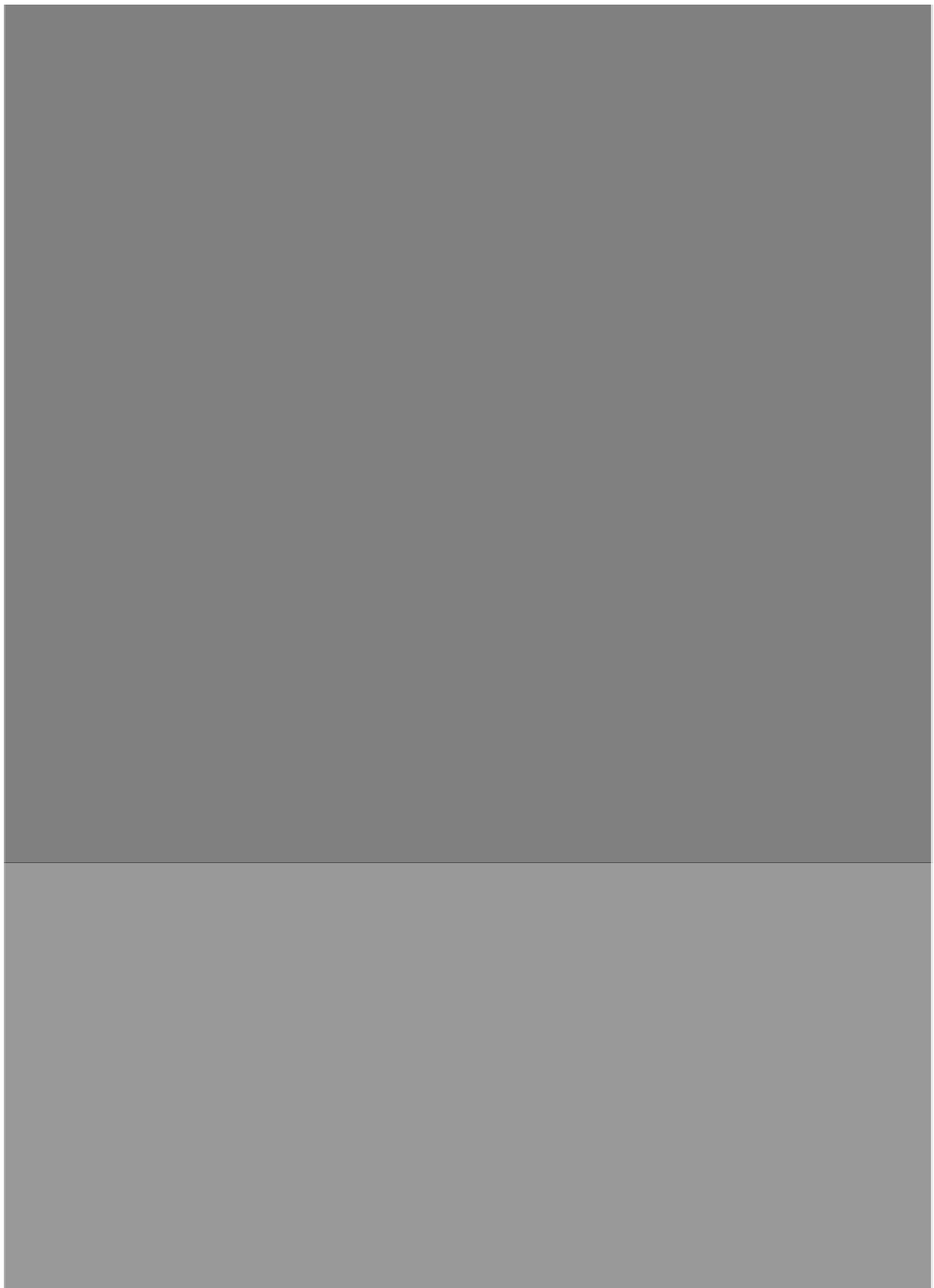
Astuce: En keras, une telle couche est implémentée à l'aide de la classe Conv2D :

Padding



```
import keras_core as keras
from keras.layers import Conv2D
```

```
layer = Conv2D(filters=6, kernel_size=5, padding="valid", activation="relu")
```



Lors du traitement d'une image d'entrée, il peut être utile de s'assurer que la carte de caractéristiques (ou carte d'activation) de sortie a la même largeur et la même hauteur que l'image d'entrée. Cela peut être réalisé en agrandissant artificiellement l'image d'entrée et en remplissant les zones ajoutées avec des zéros, comme illustré dans Fig. 6.3 dans lequel la zone de padding est représentée en blanc.

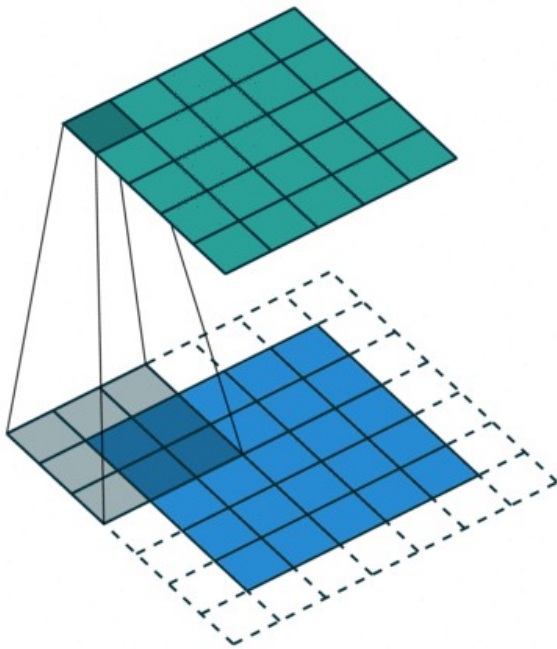
Couches de pooling

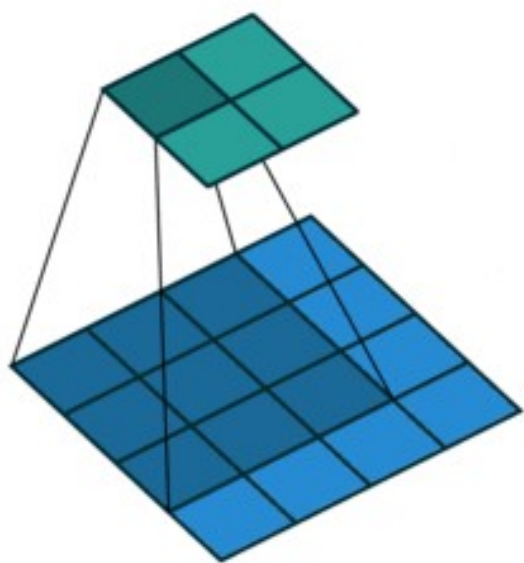
Les couches de pooling effectuent une opération de sous-échantillonnage qui résume en quelque sorte les informations

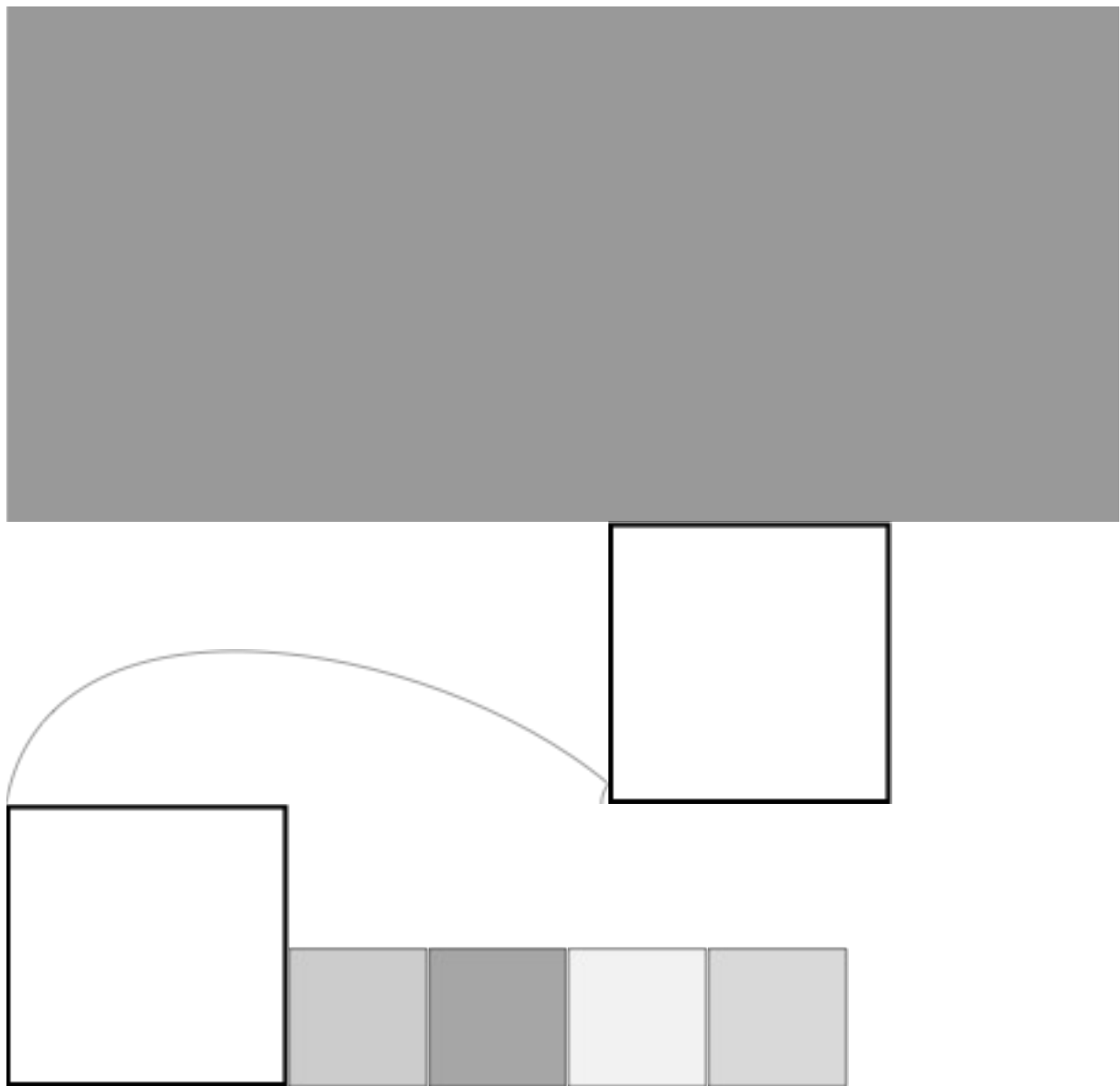
contenues dans les cartes de caractéristiques dans des cartes à plus faible résolution.

L'idée est de calculer, pour chaque parcelle d'image, une caractéristique de sortie qui calcule un agrégat des pixels de la parcelle. Les opérateurs d'agrégation typiques sont les opérateurs de moyenne (dans ce cas, la couche correspondante est appelée average pooling) ou de maximum (pour les couches de max pooling). Afin de réduire la résolution des cartes de sortie, ces agrégats sont généralement calculés sur des fenêtres glissantes qui ne se chevauchent pas, comme illustré ci-dessous, pour un max pooling avec une taille de pooling de 2x2 :

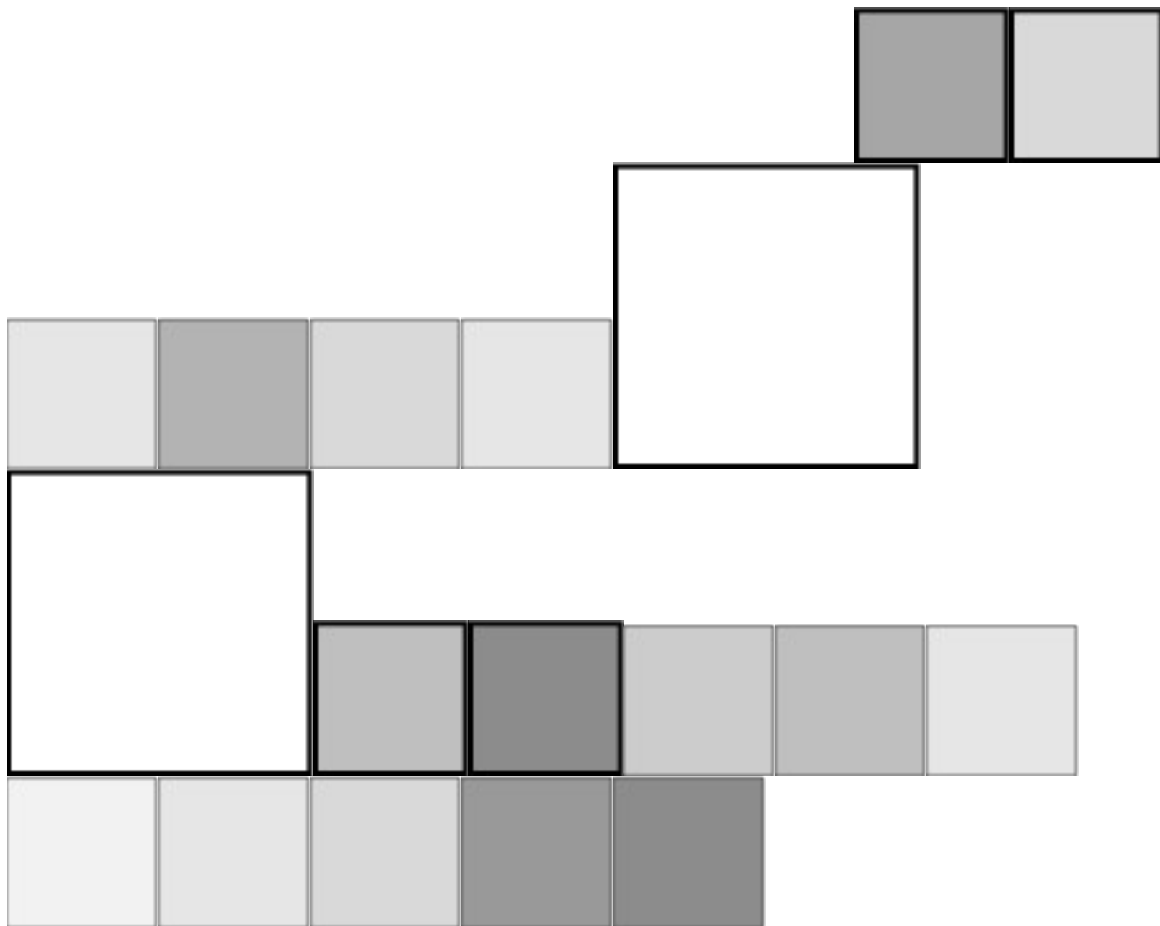
Introduction au Deep Learning (notes de cours)







max



Ces couches étaient largement utilisées historiquement dans les premiers modèles convolutifs et le sont de moins en moins à mesure que la puissance de calcul disponible augmente.

Astuce: En keras, les couches de pooling sont implémentées à travers les classes `MaxPool2D` et `AvgPool2D` :

Ajout d'une tête de classification

Un empilement de couches de convolution et de pooling produit une carte d'activation structurée (qui prend la forme d'une grille 2d avec une dimension supplémentaire pour les différents canaux). Lorsque l'on vise une tâche de classification d'images, l'objectif est de produire la classe la plus probable pour l'image d'entrée, ce qui est généralement réalisé par une tête de classification (classification head) composée de couches entièrement connectées.

Pour que la tête de classification soit capable de traiter une carte d'activation, les informations de cette carte doivent être transformées en un vecteur. Cette opération est appelée `Flatten` dans keras, et le modèle correspondant à [Fig. 6.2](#) peut être implémenté comme :

(suite sur la page suivante)

```
from keras.layers import MaxPool2D, AvgPool2D
```

```
max_pooling_layer = MaxPool2D(pool_size=2)  
average_pooling_layer = AvgPool2D(pool_size=2)
```

```
from keras.models import Sequential  
from keras.layers import InputLayer, Conv2D, MaxPool2D, Flatten, Dense  
  
model = Sequential([
```


38 Chapitre 6. Réseaux neuronaux convolutifs

Introduction au Deep Learning (notes de cours)



(suite de la page précédente)

```
InputLayer(input_shape=(32, 32, 1)),
Conv2D(filters=6, kernel_size=5, padding="valid", activation="relu"),
MaxPool2D(pool_size=2),
Conv2D(filters=16, kernel_size=5, padding="valid", activation="relu"),
MaxPool2D(pool_size=2),
Flatten(),
Dense(120, activation="relu"),
Dense(84, activation="relu"),
Dense(10, activation="softmax")
])
model.summary()
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 6)	156
max_pooling2d (MaxPooling2D)	(None, 14, 14, 6)	0

```
D)
conv2d_1 (Conv2D)      (None, 10, 10, 16)    2416
max_pooling2d_1 (MaxPoolin (None, 5, 5, 16)    0
g2D)
flatten (Flatten)
dense (Dense)
dense_1 (Dense)
dense_2 (Dense)
(None, 400)            0
(None, 120)            48120
(None, 84)             10164
(None, 10)             850
```

Total params: 61706 (241.04 KB)
Trainable params: 61706 (241.04 KB)
Non-trainable params: 0 (0.00 Byte)

6.2. Réseaux de neurones convolutifs pour les images 39

Introduction au Deep Learning (notes de cours)

CHAPITRE 7

RÉSEAUX NEURONAUX RÉCURRENTS

Les réseaux neuronaux récurrents (RNN) traitent les éléments d'une série temporelle un par un. Typiquement, à l'instant t , un bloc récurrent prend en entrée :

— l'entrée courante x_t et

— un état caché h_{t-1} qui a pour but de résumer les informations clés provenant de des entrées passées $\{x_0, \dots, x_{t-1}\}$. Ce bloc retourne un état caché mis à jour h_t :

$$h_{t-1} \quad h_t \quad h_{t+1}$$

.....

$$x_{t-1} \quad x_t \quad x_{t+1}$$

Il existe différentes couches récurrentes qui diffèrent principalement par la façon dont h_t est calculée.





7.1 Réseaux récurrents standard

La formulation originale d'une RNN est la suivante :

$$\forall t, h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad (7.1)$$

où W_h est une matrice de poids associée au traitement de l'état caché précédent, W_x est une autre matrice de poids associée au traitement de la l'entrée actuelle et b est un terme de biais.

On notera ici que W_h , W_x et b ne sont pas indexés par t , ce qui signifie que qu'ils sont partagés entre tous les temps.

Une limitation importante de cette formule est qu'elle échoue à capturer les dépendances à long terme. Pour mieux comprendre pourquoi, il faut se rappeler que les paramètres de ces réseaux sont optimisés par des algorithmes de descente de gradient stochastique.

Pour simplifier les notations, considérons un cas simplifié dans lequel h_t et x_t sont tous deux des valeurs scalaires, et regardons ce que vaut le gradient de la sortie h_t par rapport à W_h (qui est alors aussi un scalaire) :

où $o_t = W_h h_{t-1} + W_x x_t + b$, donc:

$$\nabla_W (h_t) = \tanh'(o_t) \cdot \frac{\partial o_t}{\partial h} \frac{\partial h}{\partial W_h}$$

$$\frac{\partial o_t}{\partial h_{t-1} + W_h} \cdot \frac{\partial h_{t-1}}{\partial W_h} \cdot \frac{\partial W_h}{\partial W_h}$$

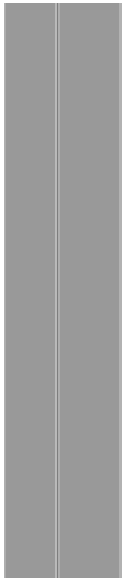
(7.2)

(7.3)

(7.4)

(7.5) (7.6)

(7.7)



Ici, la forme de $\frac{\partial h_{t-1}}{\partial W_h}$

sera similaire à celle de $\nabla(h)$ ci-dessus, et, au final, on obtient : W_h^T



$\nabla_W(\mathbf{h}_t) =_{\mathbf{h}}$

$=$

$=$

$\tanh'(o_t) \cdot [\mathbf{h}_{t-1} + W_{\mathbf{h}} \cdot \frac{\partial \mathbf{h}_{t-1}}{\partial W_{\mathbf{h}}}] \partial W_{\mathbf{h}}$

$\tanh'(o_t) \cdot [\mathbf{h}_{t-1} + W_{\mathbf{h}} \cdot \tanh'(o_{t-1}) \cdot [\mathbf{h}_{t-2} + W_{\mathbf{h}} \cdot [\dots]]]$

$\mathbf{h}_{t-1} \tanh'(o_t) + \mathbf{h}_{t-2} W_{\mathbf{h}} \tanh'(o_t) \tanh'(o_{t-1}) + \dots$

$t-1$

$\sum \mathbf{h} \cdot [W^{t-t'-1} \tanh'(o_{\cdot}) \cdot \dots \cdot \tanh'(o_{\cdot})]_{th \ t+1 \ t}$

$t'=1$



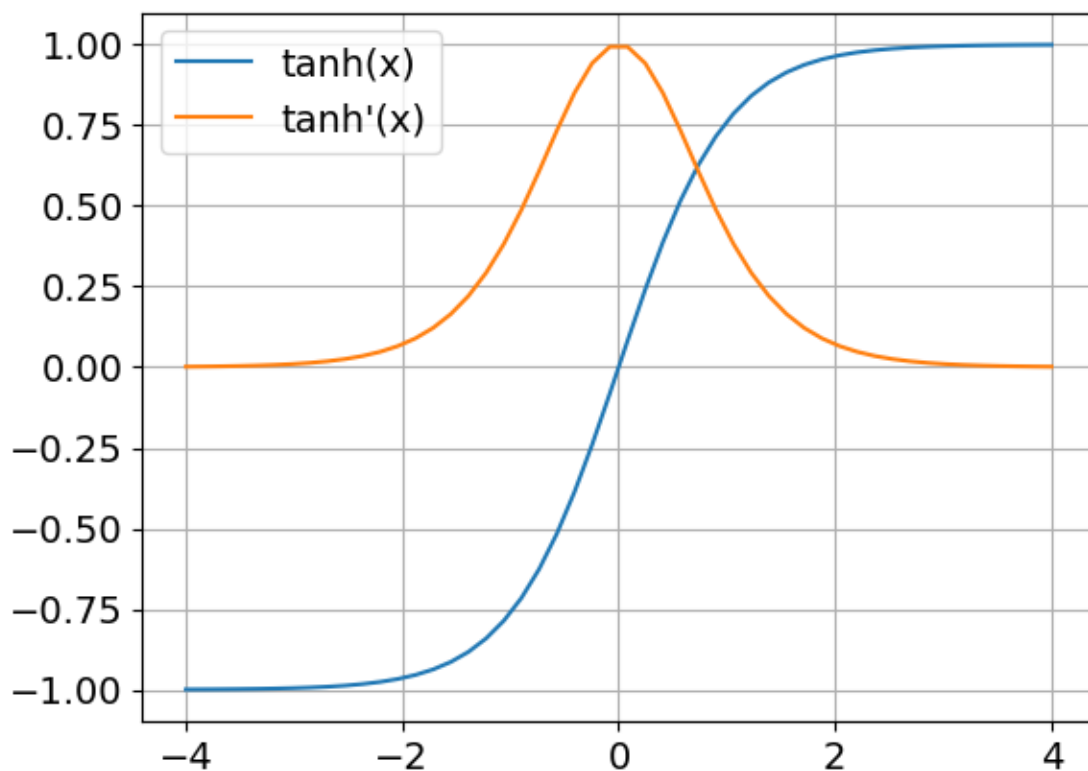
sera atténuée par un facteur $W^{t-t'-1} \tanh'(o) \cdot \dots \cdot \tanh'(o)$. Rappelons maintenant à quoi ressemblent la fonction $\tanh$ et sa dérivée :

En d'autres termes, l'influence de h_{t-1} sur h_t est :



42 Chapitre 7. Réseaux neuronaux récurrents

Introduction au Deep Learning (notes de cours)



On peut voir à quel point les gradients se rapprochent rapidement de 0 pour des entrées plus grandes (en valeur absolue) que 2, et avoir plusieurs termes de ce type dans une dérivation en chaîne fera tendre les termes correspondants vers 0.

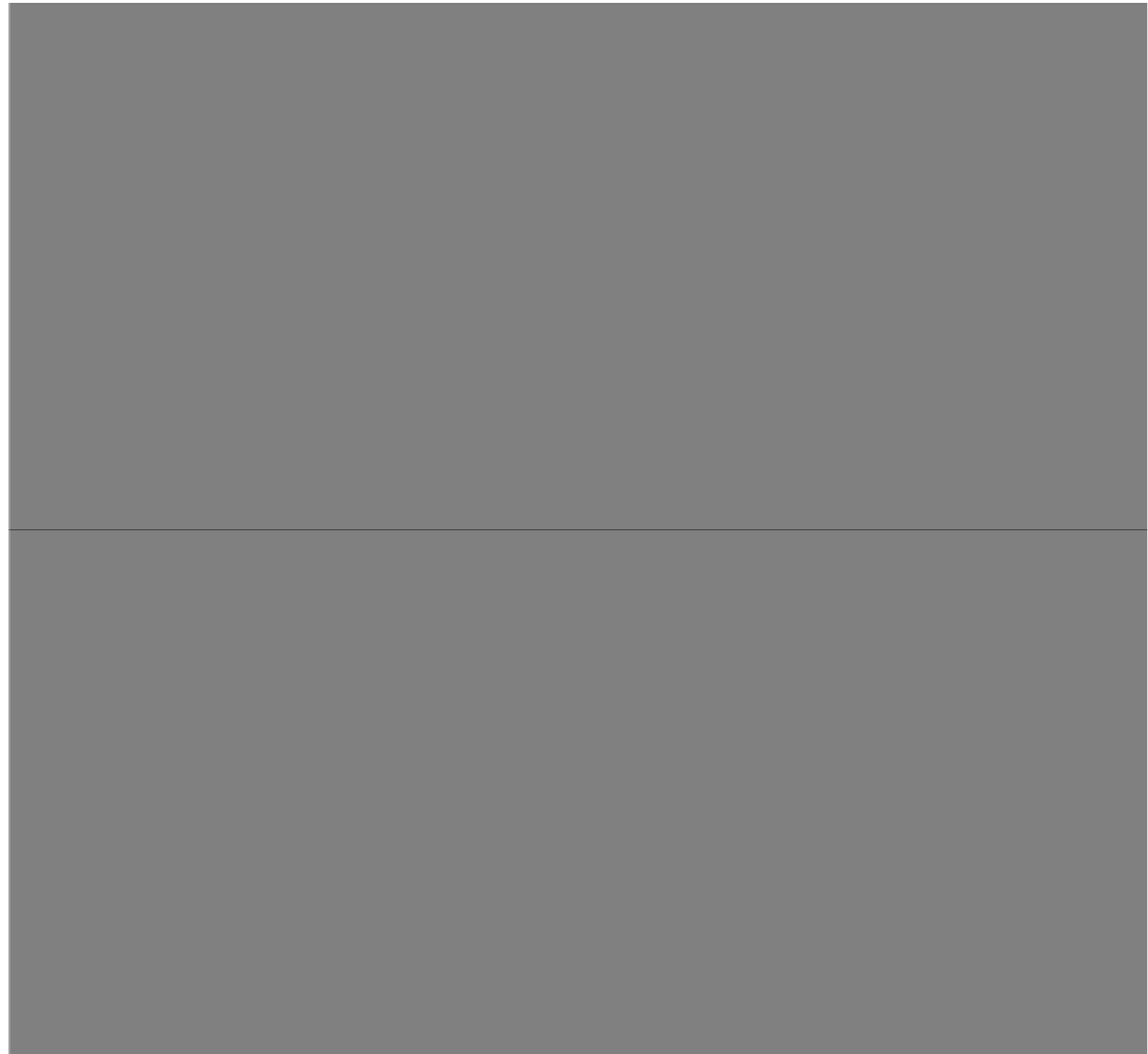
En d'autres termes, le gradient de l'état caché au temps t sera seulement influencé par quelques uns de ses prédécesseurs $\{h_{t-1}, h_{t-2}, \dots\}$ et les dépendances à long terme seront ignorées lors de l'actualisation des paramètres du modèle par descente de gradient. Il s'agit d'une occurrence d'un phénomène plus général connu sous le nom de vanishing gradient.

7.2 Long Short Term Memory

Les blocs Long Short Term Memory (LSTM, [Hochreiter and Schmidhuber, 1997]) ont été conçus comme une alternative à aux blocs récurrents classiques. Ils visent à atténuer l'effet de vanishing gradient par l'utilisation de portes qui codent explicitement quelle partie de l'information doit (resp. ne doit pas) être utilisée.

Les portes dans les réseaux neuronaux

Dans la terminologie des réseaux de neurones, une porte $g \in [0, 1]^d$ est un vecteur utilisé pour filtrer les informations d'un vecteur caractéristique entrant $v \in \mathbb{R}^d$ de telle sorte que le résultat de l'application de la porte est : $g \odot v$, où $\odot$ est le produit élément-par-élément. La porte g aura donc tendance à supprimer une partie des caractéristiques de v . (celles qui correspondent à des valeurs très faibles de g).



Dans ces blocs, un état supplémentaire est utilisé, appelé état de la cellule C_t . Cet état est calculé comme suit :

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (7.8)$$

où f_t est appelée forget gate (elle pousse le réseau à oublier les parties inutiles de l'état passé de la cellule), i_t est l'input

gate et $\tilde{C}_t$ est une version actualisée de l'état de la cellule (qui, à son tour, peut être partiellement censurée par l'input i_t

gate).



Introduction au Deep Learning (notes de cours)



Laissons de côté pour l'instant les détails concernant le calcul de ces 3 termes et concentrons-nous plutôt sur la façon dont la formule ci-dessus est significativement différente de la règle de mise à jour de l'état caché dans le modèle classique. En effet, dans ce cas, si le réseau l'apprend (par l'intermédiaire de f_t), l'information complète de l'état précédent de la cellule C_{t-1} peut être récupérée, ce qui permet aux gradients de se propager à rebours de l'axe du temps (et de ne plus disparaître).

Alors, le lien entre l'état de la cellule et l'état caché est :
$$h_t = o_t \odot \tanh(C_t). \quad (7.9)$$

En d'autres termes, l'état caché est la version transformée (par la fonction $\tanh$) de l'état de la cellule, encore censuré par une porte de sortie (output gate) o_t .

Toutes les portes utilisées dans les formules ci-dessus sont définies de manière similaire :

- $f_t =$
- $i_t =$
- $o_t =$

$$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

(7.10) (7.11) (7.12)

où σ est la fonction d'activation sigmoïde (dont les valeurs sont comprises dans $[0, 1]$) et $[h_{t-1}, x_t]$ la concaténation des caractéristiques h_{t-1} et x_t .

Enfin, l'état de cellule mis à jour $\tilde{C}_t$ est calculé comme suit :

$$\tilde{C}_t = \tanh(W \cdot [h_{t-1}, x_t] + b). \quad (7.13)$$

Il existe dans la littérature de nombreuses variantes de ces blocs LSTM qui reposent toujours sur les mêmes principes de base.

7.3 Gated Recurrent Unit

Une paramétrisation légèrement différente d'un bloc récurrent est utilisée dans les Gated Recurrent Units (GRU, [Cho et al., 2014]).

Les GRUs reposent également sur l'utilisation de portes pour laisser (de manière adaptative) l'information circuler à travers le temps. Une première différence significative entre les GRUs et les LSTMs est que les GRUs n'ont pas recours à l'utilisation d'un état de cellule. Au lieu de cela, la règle de mise à jour de l'état caché est la suivante :

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (7.14)$$

où z_t est une porte qui équilibre (par caractéristique) la quantité d'informations qui est conservée de l'état caché précédent

avec la quantité d'informations qui doit être mise à jour en utilisant le nouvel état caché candidat $\tilde{h}_t$, calculé comme suit :

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b), \quad (7.15)$$

où r_t est une porte supplémentaire qui peut cacher une partie de l'état caché précédent.

Les formules pour les portes z_t et r_t sont similaires à celles fournies pour f_t , i_t et o_t dans le cas des LSTMs.

Une étude graphique de la capacité de ces variantes de réseaux récurrents à apprendre des dépendances à long terme est fournie dans [Madsen, 2019].



7.4 Conclusion

Dans ce chapitre et le précédent, nous avons passé en revue les architectures de réseaux de neurones qui sont utilisées pour apprendre à partir de données temporelles ou séquentielles. En raison de contraintes de temps, nous n'avons pas abordé les modèles basés sur l'attention dans ce cours. Nous avons présenté les modèles convolutifs qui visent à extraire des formes locales discriminantes dans les séries et les modèles récurrents qui exploitent plutôt la notion de séquence. Concernant ces derniers, des variantes visant à faire face à l'effet de gradient évanescant ont été introduites. Il est à noter que les modèles récurrents sont connus pour nécessiter plus de données d'entraînement que leurs homologues convolutifs.

